Glow65 Manual

Learn the 6502 by reading, and by running

Glow65: a 6502 emulator for learning

https://github.com/Masakai/Glow65

2026-10-15 / e528ca9

MIT License

Copyright and License

Glow65 Manual (the manual and the tutorial)
Copyright (c) 2026 masanorisakai

This document is part of the Glow65 repository (https://github.com/Masakai/Glow65). It is released under the MIT License given in the `LICENSE` file at the top of the repository. The license text is at the end of this page.

Third-party works

- Microsoft BASIC for 6502 v1.1: Copyright (c) Microsoft Corporation (MIT License). It is bundled with Glow65 and converted for use. https://github.com/microsoft/BASIC-M6502
- The GAME language: published by Hiroshi Ohnishi in ASCII magazine (1978). GAME65G is a separate implementation written from the published specification. It does not use any GAME65 code.
- Klaus Dormann's 6502 functional test (GPLv3): used to verify the CPU. It is not included in the repository or in this document.

Company and product names in this document are trademarks or registered trademarks of their owners.

MIT License

```
MIT License

Copyright (c) 2026 masanorisakai

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Contents

Part 1: Manual

About this manual

Chapter 1: The History of the 6502

1.1 1975: The $25 CPU
1.2 Specifications of the 6502
1.3 Apple I and Apple II
1.4 Commodore PET 2001
1.5 The Family Computer
1.6 Other 6502-Family Chips
1.7 A 6502 You Can Buy Today
1.8 What Glow65 Has and Does Not Have
Sources

Chapter 2: Computer Basics

2.1 Bits and Bytes
2.2 Binary
2.3 Hexadecimal
2.4 Range of Numbers
2.5 Negative Numbers and Two's Complement
2.6 Character Codes (ASCII)
2.7 Memory and Addresses
2.8 ROM, RAM, and I/O
2.9 Practice Problems
2.10 Answers and Solutions

Chapter 3: Inside the CPU

3.1 The CPU and Memory
3.2 Registers
3.3 Buses
3.4 Fetch and Execute
3.5 Instruction Format
3.6 An Example of Executing Instructions
3.7 Vectors for Reset and Interrupts
3.8 Practice Problems
3.9 Answers and Explanations
Sources

Chapter 4 Addressing Modes

4.1 What Is an Addressing Mode
4.2 Modes Without an Operand
4.3 Immediate
4.4 Zero Page Modes
4.5 Absolute Modes
4.6 Indirect
4.7 Indexed Indirect
4.8 Indirect Indexed
4.9 Relative

(c) 2026 masanorisakai / MIT License

4.10 Summary of All Modes
4.11 Modes Added by the 65C02
4.12 Practice Problems
4.13 Answers and Explanations
Sources

Chapter 5 Instruction Reference

How to Read This Chapter
Groups
List of All 56 Instructions
Undefined Opcodes
Sources

5.1 Load and Store

5.2 Register Transfer

5.3 Arithmetic and Comparison

5.4 Logic, Shift, and Bit Test

5.5 Branch and Jump

Sources

5.6 Stack and Flag Operations

5.7 Instructions Added by the 65C02

Differences from the 6502
Running the 65C02 in Glow65
List of New Instructions
New Instructions
New Modes of 6502 Instructions
Opcodes That Were Undefined
A Difference in Decimal Mode
Practice Problems
Answers and Explanations

Chapter 6 Flags, the Stack, and Interrupts

6.1 Flags (the P Register)
6.2 The Stack
6.3 Interrupts
6.4 Reset
6.5 Practice Problems
6.6 Answers and Explanations
Sources

Chapter 7 The Memory Map and I/O

7.1 The Memory Map
7.2 Memory-Mapped I/O
7.3 UART
7.4 Monitor Routines
7.5 The Vector Area
7.6 Practice Problems
7.7 Answers and Explanations

Chapter 8 Glossary and Index

(c) 2026 masanorisakai / MIT License

8.3 Correspondence with the Tutorial

8.1 Glossary

A
B
C
D
E
F
H
I
L
M
N
O
P
R
S
T
U
V
Z

8.2 Index

8.2.1 Instruction Index
8.2.2 Addressing Mode Index
8.2.3 Flag and Register Index
8.2.4 Monitor Command and Routine Index
8.2.5 Important Address Index
8.2.6 List of Figures

Part 2: Tutorial

Tutorial: your first program, an LED counter

1. The program
2. Get it into the computer
3. Run it
4. Slow it down and watch
5. Look at the result
6. Experiments

Tutorial 2: addition and the carry flag

1. The program
2. Load it
3. Run it
4. Watch the carry
5. Experiments

Tutorial 3: talking to the UART

1. The UART is two addresses
2. The program
3. Load and run it
4. See it on the panel
5. A shorter way: use the monitor's routine

(c) 2026 masanorisakai / MIT License

6. Experiments

Tutorial 4: reading the keyboard, an echo program

1. Why the screen needs an echo
2. The program
3. Load and run it
4. Watch it on the panel
5. Experiments

Tutorial 5: calling machine code from BASIC (USR)

1. How USR works
2. The program
3. Load it
4. Call it
5. The limits of the routine
6. Printing from machine code
7. Experiments
8. Things to remember

Tutorial 6: interrupts

1. What the CPU does on an interrupt
2. The vectors on this board
3. NMI: count button presses
4. Watch an interrupt on the panel
5. IRQ: echo by interrupt
6. BRK is an interrupt too
7. Things to try
8. Things to remember

Tutorial 7: Switching to the 65C02

1. What the 65C02 adds
2. The same program twice
3. Assemble them
4. Run it on a 65C02
5. Run it on a 6502
6. The CPU switch on the panel
7. Things to try
8. Things to remember

(c) 2026 masanorisakai / MIT License

About this manual

This manual is for reading and understanding. It explains the history of the 6502, how the CPU works, the instructions, and how memory is used, in that order.

For learning by doing, see the tutorial, which is a separate document. In the tutorial you load short programs into Glow65, run them, and check the results on the LEDs and in the monitor. The manual covers the ideas and specifications behind them. You can read either one first.

(c) 2026 masanorisakai / MIT License

Chapter 1: The History of the 6502

The 6502 is an 8-bit CPU (central processing unit, the part that executes a program's instructions) announced in 1975. The CPUs of the Apple II, the Commodore PET 2001, and the Family Computer are either a 6502 or a derivative of it.

Numbers such as [1] refer to the sources at the end of the chapter.

1.1 1975: The $25 CPU

MOS Technology of the United States announced the 6502 in September 1975 at the WESCON trade show in San Francisco [3]. The price was $25 per chip [3].

The chip was designed by a team that included Chuck Peddle and Bill Mensch [2][3]. Peddle is said to have led the team, and Mensch to have designed the chip [3].

The two had been engineers at Motorola, which sold a CPU called the 6800. Peddle's proposal for a cheaper CPU was not accepted. In August 1974, Peddle and several other engineers left Motorola and moved to MOS Technology [3][4].

In November 1975, Motorola sued MOS Technology, claiming that former employees had used Motorola's technology. The case was settled in the spring of 1976, and MOS Technology stopped selling the 6501, which had the same pin layout as the 6800 [4]. Sales of the 6502 continued.

1.2 Specifications of the 6502

The specifications of the 6502 are as follows [1].

Item	Description
Data width	8 bits. It handles one byte at a time
Address (memory location) width	16 bits. 2 to the 16th power, so it can specify 65,536 bytes (64KB)
Registers	A (accumulator), X, Y, S (stack pointer), and P (status) are 8 bits. PC (program counter) is 16 bits
Zero page	The first 256 bytes, where the upper byte of the address is 00
Clock	1 MHz (the assumption behind the timing tables in this manual)

1 MHz means the clock ticks 1 million times per second. Later versions ran at 2 MHz or 3 MHz [3]. The chip has about 3,500 transistors [3][5].

(c) 2026 masanorisakai / MIT License

Column: How to Count the Transistors

"About 3,500" is the number usually given for the 6502's transistors. To be exact, there are 3,510 transistors that form logic. There are also 1,018 transistors used as loads. Together they make 4,528. The difference in the numbers comes from where you draw the line on what counts as a transistor [5].

What you can try in Glow65

The panel LEDs show the 16-bit address bus (A0 to A15) and the 8-bit data bus (D0 to D7). There are also LEDs for PC, SP, A, X, Y, and the status flags. The monitor command `D 0000 00FF` displays the whole zero page. `L 0300 0308` reads the contents of memory as instructions (disassembles them). The steps are in Tutorial Chapter 1.

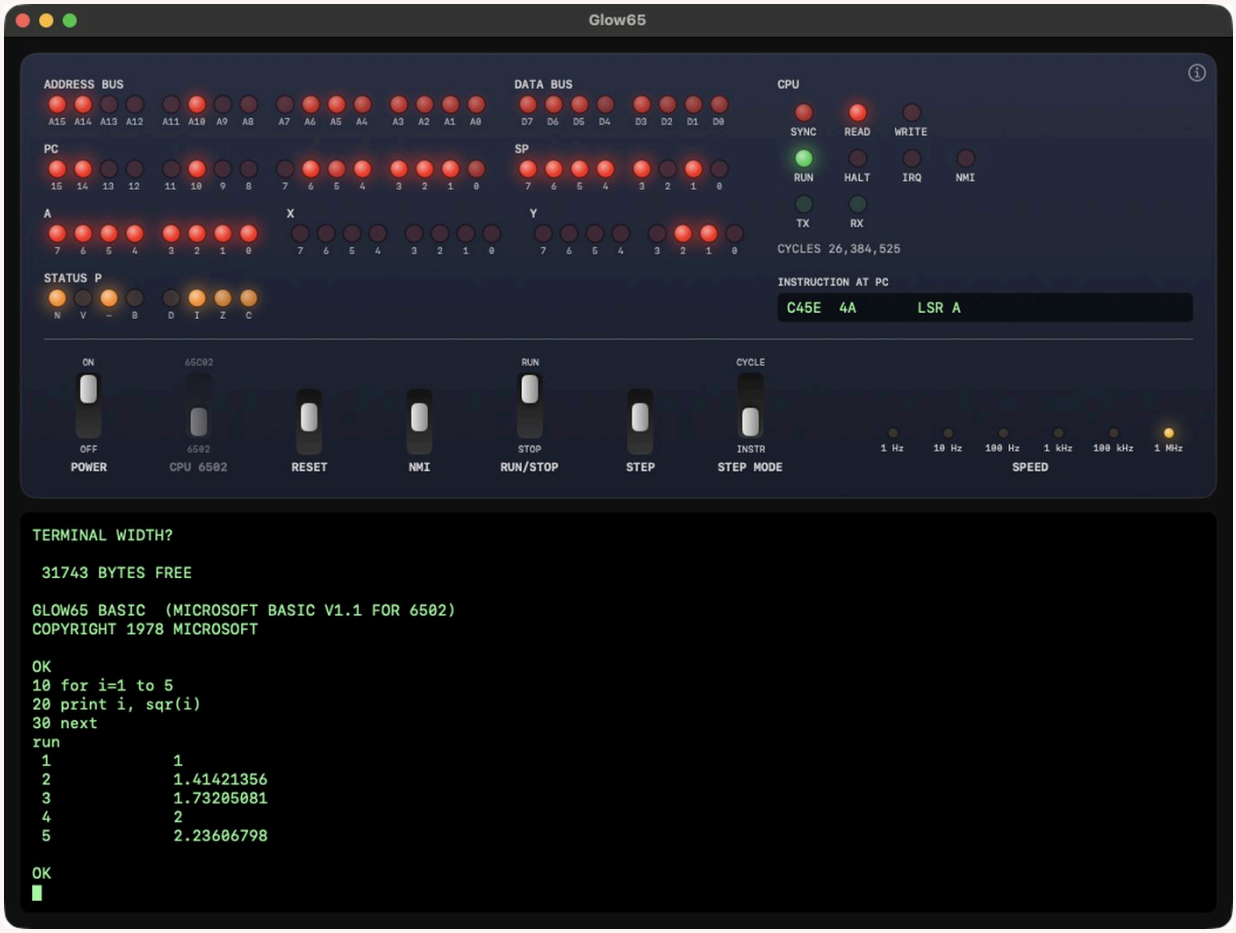


Figure 1-1 The Glow65 front panel while BASIC is running. The top section has the LEDs for the address bus, data bus, PC, SP, A, X, Y, and the status flags

1.3 Apple I and Apple II

The Apple I went on sale in July 1976. Its CPU was a 6502 running at about 1 MHz, and the price was $666.66 [6].

(c) 2026 masanorisakai / MIT License

The Apple II began shipping on June 10, 1977 [7]. Its CPU was a 6502 at 1.023 MHz [7]. At launch the price was $1,298 for the 4KB RAM configuration and $2,638 for the configuration with up to 48KB [7].

The Apple II outputs video as an NTSC composite signal. Text is 40 columns by 24 rows. The keyboard has uppercase letters only, and sound comes from toggling a 1-bit speaker. It can save data to cassette tape. The ROM (memory that keeps its contents when the power is off) holds Integer BASIC and a machine-code monitor [8].

> **Column: The Odd Clock of 1.023 MHz**
>
> The Apple II clock is not exactly 1 MHz. An NTSC television signal contains a color signal of about 3.579545 MHz. The Apple II clock is 2/7 of that value. To be exact it is 1.022727 MHz, which is rounded to 1.023 MHz. The value is derived from the same reference as the video circuit [6].

1.4 Commodore PET 2001

The PET 2001 is a computer from Commodore. Its CPU was a 6502 at 1 MHz, and the price was $795 [9].

The answer to "when did it go on sale?" depends on the standard you use.

Standard	Date
Exhibition	January 1977, a prototype shown at the Winter CES (a consumer electronics trade show)
Shipping	The first 100 or so units in October 1977. Ordinary buyers received it in December 1977

The Apple II shipped in June 1977. Compared with it, the PET was shown first, but the Apple II shipped first [7][9].

The PET has the screen, keyboard, and cassette tape unit built into one case. The screen is monochrome, 40 columns by 25 rows. Some models had a small buzzer for sound, and some did not. The ROM holds BASIC, the KERNAL (which handles basic input and output), and the character shapes [9].

1.5 The Family Computer

Nintendo's Family Computer (Famicom) went on sale in 1983 [12]. The release date was July 15, and the price was 14,800 yen [10]. Its CPU is the Ricoh RP2A03, a chip that combines a core compatible with the 6502 and circuits such as the one that produces sound [10][11].

In an interview with a Ricoh engineer (Toyofumi Takahashi), the 2A03 is described as "a chip with a 6502 CPU core and sound" [14]. A different group built the CPU core, and Takahashi was in charge of the sound circuit. Ricoh made the CPU together with the 2C02, a chip for image processing, and supplied them to Nintendo [14].

The Famicom CPU does not have the decimal mode of the 6502 (a feature that calculates numbers while keeping them in decimal) [15].

(c) 2026 masanorisakai / MIT License

The CPU clock is about 1.79 MHz on the NTSC version for Japan. For regions using the PAL system, the chip is the RP2A07 and the clock is about 1.66 MHz [15].

> **What you can try in Glow65**
>
> The Glow65 CPU runs the decimal mode in addition to the 6502 instructions. It passes Klaus Dormann's functional test, including the decimal mode (see the "Tests" section of the README). The Famicom chip does not have this feature.

> **Column: Why the Famicom Chose the 6502**
>
> Masayuki Uemura, who was involved in developing the Famicom, describes how it came about in an official Nintendo interview. It was Ricoh, a maker of semiconductors, that recommended the 6502. At the time, the 6502 was almost unused in Japan. Uemura says that few people could understand its inner workings, so it was hard for other companies to analyze, and that was the reason for choosing it. He says engineers inside the company objected [13].

1.6 Other 6502-Family Chips

Here are examples of machines and chips that used the 6502 or its derivatives.

Machine or chip	Year	Description
Atari 2600	About 1977	Uses the 6507. About 1.19 MHz [16]
VIC-20	1980 (Japan) / 1981 (worldwide)	6502, about 1.02 MHz (NTSC) [17]
BBC Micro	December 1981	6502A, 2 MHz [18]
Commodore 64	August 1982	6510, about 1.023 MHz (NTSC) [19]
65C02	1983	A CMOS version with lower power consumption. Developed by WDC [20]
65816	About 1985	16 bits inside, 24-bit addresses (16MB) [21]
PC Engine	October 30, 1987	Hudson Soft HuC6280 (an extended 65C02) [22][23]
Super Famicom	November 21, 1990	Ricoh 5A22 (a derivative of the 65816) [24]

CMOS is a way of making transistors that uses little power.

> **Column: The 6507 with Fewer Address Pins**
>
> The CPU of the Atari 2600 is the 6507, a 6502 with some of its pins removed. The 6502 has 40 pins, and the 6507 has 28. The address pins drop from 16 to 13, so it can specify only up to 2 to the 13th power, or 8,192 bytes (8KB). It also has no interrupt pins. It was a CPU made cheaply by cutting pins. The inside of the chip is the same as the 6502 [16][27].

(c) 2026 masanorisakai / MIT License

1.7 A 6502 You Can Buy Today

The 6502 family is still on sale. WDC (Western Design Center) lists the W65C02S on its official page. It comes in three packages: 40-pin DIP, 44-pin PLCC, and 44-pin QFP, and the official page points to distributors such as Mouser and Jameco [25].

The 6502 is an inexpensive CPU that was announced at $25. It has about 3,500 transistors [3][5], and MOS Technology published a hardware manual [1]. Derivatives were also made to suit different uses, including the 6507, 6510, 2A03, 65C02, 65816, and HuC6280.

1.8 What Glow65 Has and Does Not Have

Glow65 is an emulator of a fictional single-board computer, made for learning the 6502. The differences from three real machines are shown in the table below.

Item	Apple II (1977)	PET 2001 (1977)	Famicom (1983)	Glow65
CPU	6502, 1.023 MHz [7]	6502, 1 MHz [9]	RP2A03 (6502-compatible, no decimal mode), about 1.79 MHz [15]	NMOS 6502 (default). The SPEED control on the panel switches from 1 Hz to 1 MHz. The CPU switch can also select the 65C02
Memory	RAM 4KB to 48KB [7]	RAM 4KB or 8KB [9]	2KB of RAM for the CPU, 2KB of RAM for video [26]	32KB of RAM (`$0000–$7FFF`)
Display	NTSC composite output, 40 columns by 24 rows [8]	Built-in monochrome screen, 40 columns by 25 rows [9]	Image processing chip 2C02, 256 x 240 dots [26]	Only a terminal connected through the UART. No dedicated graphics screen. A front panel (LEDs) shows the CPU state
Input	Keyboard (uppercase only) [8]	Built-in keyboard [9]	Game pads (two, fixed to the console) [26]	Keyboard input from the terminal (through the UART)
Sound	1-bit speaker [8]	A small buzzer only (some models have none) [9]	5 channels [26]	None
External storage	Cassette tape [8]	Built-in cassette unit [9]	None built in [26]	None. BASIC `SAVE` and `LOAD` cannot be used. File loading (File → Load Program) is a feature where the emulator writes directly into RAM
Software in ROM	Integer BASIC, machine-code monitor (8KB) [8]	BASIC, KERNAL, character shapes [9]	Game cartridges (the program chip is up to 32KB) [26]	Monitor (`$F000–$FFFF`), Microsoft BASIC (`$C000–$EFFF`), and GAME65G (`$8000–$9FFF`)

The BASIC in Glow65 is Microsoft's BASIC for the 6502 (v1.1 from 1978) [28], converted for this board.

The goal of Glow65 is to let you watch how the CPU works, not to reproduce real hardware. It does not reproduce the extra dummy reads and dummy writes of the real chip, so the bus display differs slightly from an actual 6502.

(c) 2026 masanorisakai / MIT License

Sources

1. MOS Technology, "MCS6500 Microcomputer Family Hardware Manual", January 1976 edition: https://www.commodore.ca/wp-content/uploads/2018/11/kim1_hardware.txt
2. IEEE Spectrum, "Chip Hall of Fame: MOS Technology 6502 Microprocessor": https://spectrum.ieee.org/chip-hall-of-fame-mos-technology-6502-microprocessor
3. Wikipedia, "MOS Technology 6502": https://en.wikipedia.org/wiki/MOS_Technology_6502
4. Wikipedia, "Motorola 6800": https://en.wikipedia.org/wiki/Motorola_6800
5. Ken Shirriff, "Intel x86 documentation has more pages than the 6502 has transistors": https://www.righto.com/2013/09/intel-x86-documentation-has-more-pages.html
6. Wikipedia, "Apple I": https://en.wikipedia.org/wiki/Apple_I
7. Wikipedia, "Apple II": https://en.wikipedia.org/wiki/Apple_II
8. Wikipedia, "Apple II (original)": https://en.wikipedia.org/wiki/Apple_II_(original)
9. Wikipedia, "Commodore PET": https://en.wikipedia.org/wiki/Commodore_PET
10. Japanese Wikipedia, "ファミリーコンピュータ" (Family Computer): https://ja.wikipedia.org/wiki/ファミリーコンピュータ
11. Wikipedia, "Ricoh 2A03": https://en.wikipedia.org/wiki/Ricoh_2A03
12. 任天堂 沿革 (Nintendo company history): https://www.nintendo.co.jp/corporate/history/index.html
13. 任天堂「社長が訊く マリオ25周年 Vol.2」(Nintendo, "Iwata Asks: Mario's 25th Anniversary, Vol. 2"): https://www.nintendo.co.jp/n10/interview/mario25th/vol2/index2.html
14. 高橋豊文氏オーラルヒストリー(第2回前半)、一橋大学 IIR Working Paper WP#19-17、2019年 (Toyofumi Takahashi oral history, part 2 first half, Hitotsubashi University IIR Working Paper WP#19-17, 2019): https://pubs.iir.hit-u.ac.jp/admin/ja/pdfs/file/2267
15. NESdev Wiki, "CPU": https://www.nesdev.org/wiki/CPU
16. Wikipedia, "Atari 2600": https://en.wikipedia.org/wiki/Atari_2600
17. Wikipedia, "VIC-20": https://en.wikipedia.org/wiki/VIC-20
18. Wikipedia, "BBC Micro": https://en.wikipedia.org/wiki/BBC_Micro
19. Wikipedia, "Commodore 64": https://en.wikipedia.org/wiki/Commodore_64
20. Wikipedia, "WDC 65C02": https://en.wikipedia.org/wiki/WDC_65C02
21. Wikipedia, "WDC 65C816": https://en.wikipedia.org/wiki/WDC_65C816
22. Wikipedia, "TurboGrafx-16": https://en.wikipedia.org/wiki/TurboGrafx-16
23. Wikipedia, "Hudson Soft HuC6280": https://en.wikipedia.org/wiki/Hudson_Soft_HuC6280
24. Wikipedia, "Super Nintendo Entertainment System": https://en.wikipedia.org/wiki/Super_Nintendo_Entertainment_System
25. Western Design Center, "W65C02S": https://www.westerndesigncenter.com/wdc/w65c02s-chip.php
26. Wikipedia, "Nintendo Entertainment System": https://en.wikipedia.org/wiki/Nintendo_Entertainment_System
27. Wikipedia, "MOS Technology 6507": https://en.wikipedia.org/wiki/MOS_Technology_6507
28. Microsoft, "BASIC-M6502" (source code of Microsoft BASIC for the 6502): https://github.com/microsoft/BASIC-M6502

(c) 2026 masanorisakai / MIT License

Chapter 2: Computer Basics

All a CPU handles is strings of 0s and 1s. This chapter explains how people read and write those strings. It covers bits and bytes, binary and hexadecimal, negative numbers, characters, and memory and addresses, in that order. The math you need is powers of 2 (2, 4, 8, 16 ...) and basic arithmetic (addition, subtraction, multiplication, and division).

At the end of each section there is something you can try in Glow65. How to use the monitor is described in "Monitor commands" in the README. Practice problems and their answers are at the end of the chapter.

2.1 Bits and Bytes

A **bit** is the smallest unit. It holds either 0 or 1, like a switch that is ON or OFF.

A **byte** is a unit of 8 bits grouped together. The 6502 handles data one byte at a time. Eight bits can form 2 to the 8th power, or 256, combinations of 0s and 1s.

The 8 bits are numbered starting from 0 at the right end, then 1, 2, ..., 7 going left. Bit 0 is called the **least significant bit** (LSB), and bit 7 is called the **most significant bit** (MSB).

2.2 Binary

In the decimal numbers we use every day, each digit is worth 10 times the digit to its right. 345 is 3 x 100 + 4 x 10 + 5 x 1.

Binary is a way of writing numbers in which each digit is worth 2 times the digit to its right. The only digits are 0 and 1. The weights of the 8 bits, from the left, are as follows.

Bit number	7	6	5	4	3	2	1	0
Weight	128	64	32	16	8	4	2	1

In this manual, a binary number starts with `%`. In `%01101001`, the bits with weights 64, 32, 8, and 1 are 1, so the value is 64 + 32 + 8 + 1 = 105.

To **convert binary to decimal**, add up the weights of the bits that are 1.

To **convert decimal to binary**, go from the largest weight down and check whether you can subtract it. For 200:

1. You can subtract 128 from 200. 72 remains. Bit 7 is 1.
2. You can subtract 64 from 72. 8 remains. Bit 6 is 1.
3. 8 is smaller than 32, so bit 5 is 0. It is smaller than 16, so bit 4 is 0.
4. You can subtract 8 from 8. 0 remains. Bit 3 is 1.
5. Nothing remains, so bits 2, 1, and 0 are 0.

The result is `%11001000`.

(c) 2026 masanorisakai / MIT License

2.3 Hexadecimal

Binary takes many digits and is hard to read. It takes 8 digits for 8 bits and 16 digits for 16 bits. So we use **hexadecimal**, which groups 4 bits at a time and writes them as one digit.

Four bits have 16 combinations, from 0 to 15. One hexadecimal digit stands for 0 to 15. For 10 to 15, it uses the letters A to F.

Decimal	Binary	Hex		Decimal	Binary	Hex
0	%0000	0		8	%1000	8
1	%0001	1		9	%1001	9
2	%0010	2		10	%1010	A
3	%0011	3		11	%1011	B
4	%0100	4		12	%1100	C
5	%0101	5		13	%1101	D
6	%0110	6		14	%1110	E
7	%0111	7		15	%1111	F

In the 6502 world, a hexadecimal number starts with `$`. `$A0` is the hexadecimal number A0. This manual follows that convention.

To **convert between binary and hexadecimal**, split the number into groups of 4 bits and look each group up in the table.

- Split `%10100000` into groups of 4 bits, and you get `1010` and `0000`. From the table, these are A and 0. The result is `$A0`.
- In `$3C`, 3 is `0011` and C is `1100`. The result is `%00111100`.

To **convert hexadecimal to decimal**, multiply the upper digit by 16 and add the lower digit. For `$A0`, A is 10, so it is 10 x 16 + 0 = 160.

To **convert decimal to hexadecimal**, divide by 16. The quotient is the upper digit and the remainder is the lower digit. Dividing 200 by 16 gives a quotient of 12 and a remainder of 8. 12 is C, so the result is `$C8`.

A 16-bit value is written with 4 hexadecimal digits. Think of `$1234` as the upper 2 digits `$12` and the lower 2 digits `$34`.

> **Column: Different Ways to Write Hexadecimal**
>
> The way to write hexadecimal differs by language and tool. In 6502 assemblers, a `$` prefix is common. In C and similar languages, you add `0x` and write `0xA0`. In Intel-style assemblers, you add an `h` at the end and write `0A0h`. All of them mean the same number.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

The data bus LEDs on the panel are 8 LEDs, D7 to D0. An LED lights when the corresponding bit of the value on the bus is 1. When the value is `$A0` (`%10100000`), two LEDs light, D7 and D5. The address bus has 16 LEDs, A15 to A0.

To see `$A0` on the LEDs, run `LDA #$A0` and read the A register LEDs.

1. In the monitor, enter `E 0300 A9 A0 8D F0 03 4C 00 03` and run `G 0300`. The program repeats `LDA #$A0`, `STA $03F0`, and `JMP $0300`.
2. Set **RUN/STOP** to STOP to stop the CPU. Set **STEP MODE** to INSTR.
3. Press **STEP** until INSTRUCTION AT PC shows `LDA #$A0`, then press it once more.

Right after `LDA #$A0` runs, A7 and A5 are lit on the A register LEDs (Figure 2-1). On the DATA BUS, D7 and D5 are lit too, because the operand just read was `$A0`. The ADDRESS BUS shows `$0301`, the address the operand was read from. The most significant bit is 1, so the N flag LED is lit as well.



Figure 2-1 After `LDA #$A0`. The A LEDs are A7 and A5, the DATA BUS is D7 and D5, and the ADDRESS BUS is `$0301`

2.4 Range of Numbers

The numbers that 8 bits can represent run from 0, when all bits are 0, to 255, when all bits are 1. In hexadecimal, that is `$00` to `$FF`.

With 16 bits, the range is 0 to 65535, or `$0000` to `$FFFF`. 65535 is 2 to the 16th power minus 1.

Bits	Range (decimal)	Range (hex)
8	0 to 255	`$00` to `$FF`
16	0 to 65535	`$0000` to `$FFFF`

If a calculation goes beyond the range, the digits that do not fit are discarded. In 8 bits, 255 + 1 gives 0, not 256. The fact that a digit overflowed is recorded in a marker (a flag) called the **carry**. Chapter 6 explains the carry.

(c) 2026 masanorisakai / MIT License

2.5 Negative Numbers and Two's Complement

All a CPU has is strings of bits. Negative numbers are also represented as strings of bits. The 6502 uses a method called **two's complement**.

In two's complement, if the most significant bit (bit 7) is 1, the value is read as negative, and if it is 0, as zero or positive. This bit is called the **sign bit**. The range that 8 bits can represent is -128 to 127.

Binary	Hex	Read as unsigned	Read as signed
%00000000	`$00`	0	0
%00000001	`$01`	1	1
%01111111	`$7F`	127	127
%10000000	`$80`	128	-128
%11111111	`$FF`	255	-1

The same `$FF` is either 255 or -1, depending on how you read it. The CPU does not know which way you are reading it. The person writing the program decides.

To make a negative number, follow two steps. For -5:

1. Flip every 0 and 1 in the binary form of 5, `%00000101`. This gives `%11111010`.
2. Add 1. This gives `%11111011`, which is `$FB`.

To check the result, add 256. `$FB` is 251, and 251 - 256 is -5. Also, adding 5 to `$FB` gives 256, which wraps back to 0 in 8 bits.

Two's complement has the advantage that signed numbers can be calculated with the same addition circuit as unsigned numbers.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

The branch instruction `BNE` holds its destination as a distance (a 1-byte signed number) from the next instruction. With `L` in the monitor, you can see how the distance byte turns into a destination address.

```
*E 0300 D0 FB
*L 0300 0301
0300  D0 FB     BNE   $02FD
```

`BNE` is a 2-byte instruction, so the next instruction is at `$0302`. The distance `$FB` is -5. Going back 5 from 0302 gives `$02FD`. With `D0 7F` (distance +127) the display is `BNE $0381`, and with `D0 F0` (distance -16) it is `BNE $02F2`.

You can also check two's complement on the A LEDs. With the same steps as in 2.3, enter `E 0300 A9 FB 8D F0 03 4C 00 03`, run it, and stop after `LDA #$FB`. On the A LEDs, only A2 is off, so the pattern is `%11111011` (`$FB`, -5) (Figure 2-2). The most significant bit is 1, so the N flag is lit too. The CPU can read this value as unsigned 251 or as signed -5.

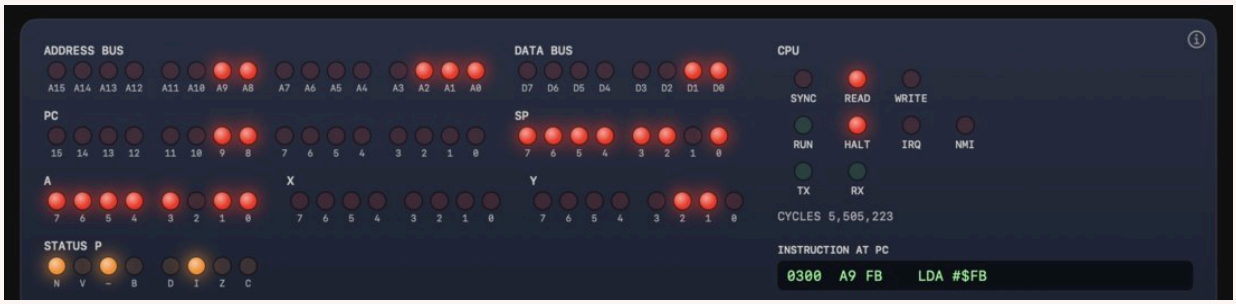


Figure 2-2 After `LDA #$FB`. Every A LED except A2 is lit

2.6 Character Codes (ASCII)

Computers also represent characters with numbers. A table that decides which number is assigned to which character is called a **character code**. For letters, digits, and symbols, an assignment called ASCII is widely used. ASCII is what the Glow65 UART sends and receives.

The main ASCII assignments are as follows.

Character	Code	Note
`0` to `9`	`$30` to `$39`	Digit characters. `0` is `$30` and `9` is `$39`
`A` to `Z`	`$41` to `$5A`	Uppercase. `A` is `$41`
`a` to `z`	`$61` to `$7A`	Lowercase. `a` is `$61`
CR	`$0D`	Line break (carriage return). Sent by the Return key

The digit character `7` is not the number 7. It is the code `$37`. To get the number 7 from the character `7`, subtract `$30`.

(c) 2026 masanorisakai / MIT License

Uppercase and lowercase letters differ in code by exactly `$20`. `A` (`$41`) and `a` (`$61`) are an example.

ASCII fits in 7 bits. The most significant bit of an 8-bit byte is not used. The `CHROUT` routine of the Glow65 monitor discards the most significant bit of the byte it outputs (see the jump table in the README).

> **What you can try in Glow65**
>
> You write UART data to `$A001`. If you build a program with `LDA #$48` and `STA $A001`, `H` (`$48`) appears on the terminal. The following is a 16-byte program that sends `H`, `I`, and CR in order. Run it with `G`, and the final `60` (`RTS`) returns to the monitor.
>
> ```
> *E 0300 A9 48 8D 01 A0 A9 49 8D 01 A0 A9 0D 8D 01 A0 60
> *G 0300
> HI
> ```
>
> Sending `$0D` makes the terminal display move to a new line.

2.7 Memory and Addresses

Programs and data are placed in **memory**. Memory is a shelf with a row of boxes, each holding one byte. The boxes are numbered in order from the start. This number is the **address**.

A 6502 address is 16 bits. The numbers it can express run from `$0000` to `$FFFF`, 65,536 in all. Each box is 1 byte, so the total is 65,536 bytes. If 1 KB (kilobyte) is 1,024 bytes, this is 64 KB. This whole range is called the **address space**.

Placing a 16-bit address in memory takes 2 bytes. The 6502 places the **lower byte first** and the upper byte next. This ordering is called **little-endian**.

For example, if you place the address `$1234` starting at `$0300`, it looks like this.

Address	`$0300`	`$0301`
Content	`$34` (low)	`$12` (high)

The order is the reverse of how it looks when written, so be careful when you read it.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

You can check reading and writing memory with these three commands.

- `E` writes bytes in order, starting from the address you give.
- `D` displays the contents of memory in hexadecimal. The display is in units of 16 bytes, showing 16 from the start address.
- `L` reads the contents of memory as instructions.

Here is an example that places `$1234` low byte first.

```
*E 0300 34 12
*D 0300 0301
0300: 34 12 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

`$0300` holds `34` and `$0301` holds `12`. The first 2 bytes are in the order low, high.

You can check the same thing in BASIC. Enter `B` in the monitor to start BASIC, press Return twice, and type the following.

```
POKE 768,52 : POKE 769,18
PRINT PEEK(769)*256+PEEK(768)
 4660
```

768 is `$0300`, 52 is `$34`, and 18 is `$12`. Multiply the upper byte by 256 and add the lower byte, and you get 4660. 4660 is the decimal form of `$1234`.

There is also an example of a 16-bit value stored in 2 bytes inside the 6502 itself. When power is turned on, or on an interrupt, the CPU reads the 6 bytes from address `$FFFA` to `$FFFF`. There, three destination addresses are lined up in little-endian order. Chapter 6 explains this in detail.

What you can try in Glow65

You can see the last 6 bytes with the monitor command `D FFF0 FFFF`.

```
*D FFF0 FFFF
FFF0: FF FF FF FF FF FF FF FF FF FF 1E F4 2A F1 21 F4
```

The last 6 bytes are three pairs: `1E F4`, `2A F1`, and `21 F4`. Because it is little-endian, they read as `$F41E`, `$F12A`, and `$F421`. The `$F12A` at `$FFFC` and `$FFFD` is the address the CPU jumps to first when power is turned on.

(c) 2026 masanorisakai / MIT License

2.8 ROM, RAM, and I/O

An address space is not always made of only one kind of part. Parts with different roles are connected using the same way of assigning addresses.

- **RAM** is memory you can read and write. Its contents disappear when the power is turned off. It holds program variables and working data.
- **ROM** is read-only memory. Its contents remain when the power is turned off. It holds the program that runs at startup.
- **I/O** (input and output) is the window to devices outside. A device is assigned to some of the memory addresses. Reading or writing those addresses means exchanging data with the device. This method is called memory-mapped I/O.

The Glow65 address space is used as follows (the same content as the "Memory map" in the README).

Address	Type	Content
`$0000–$7FFF`	RAM	32KB
`$0200–$027F`	Inside RAM	The monitor's line input buffer
`$0300–$03F7`	Inside RAM	Area left free for your own programs
`$03F8–$03F9`	Inside RAM	Address of the NMI handler
`$03FA–$03FB`	Inside RAM	Address of the IRQ and BRK handler
`$0400–$7FFF`	Inside RAM	BASIC or GAME65G programs and variables
`$8000–$9FFF`	ROM	GAME65G
`$A000`	I/O	UART status and control
`$A001`	I/O	UART data
`$C000–$EFFF`	ROM	Microsoft BASIC (about 8.6KB used)
`$F000–$FFFF`	ROM	Monitor and vectors (`$FFFA–$FFFF`)

`$A000–$A0FF` is assigned to the UART. Even addresses work as status and control, and odd addresses work as data. The ones normally used are `$A000` and `$A001`. Nothing is connected to the addresses not listed in the table above. Reading them returns `$FF`. Writing to ROM does not change its contents.

The UART is a device for exchanging data with a terminal. A byte written to `$A001` goes out to the terminal, and a byte typed on the terminal can be read from `$A001`. `$A000` is the status that tells whether a byte can be received or sent. Chapter 7 explains how to use it in detail.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

An address with nothing connected reads as `$FF`.

```
*D B000 B00F
B000: FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
```

A write to ROM is ignored. `$F000` is ROM and holds the monitor's first `JMP` (`$4C`).

```
*E F000 00
*D F000 F00F
F000: 4C 0F F0 4C 28 F0 4C 37 F0 4C 4B F1 4C 1A F0 AD
```

After `E F000 00`, `$F000` is still `4C`. If you do the same operation on RAM at `$0300`, the value you wrote stays (the example in 2.7).

2.9 Practice Problems

The answers and how to solve them are in the next section.

Problem 1 Convert the following binary numbers to decimal. (1) `%10110101` (2) `%01100110`

Problem 2 Convert the following decimal numbers to 8-bit binary. (1) 200 (2) 77

Problem 3 Convert the following hexadecimal numbers to decimal. (1) `$2F` (2) `$C8`

Problem 4 Convert the following decimal numbers to hexadecimal. (1) 100 (2) 173

Problem 5 Convert the following hexadecimal numbers to binary. (1) `$5A` (2) `$E3`

Problem 6 Convert `%10010110` to hexadecimal.

Problem 7 Write the following numbers in 8-bit two's complement, in hexadecimal. (1) -3 (2) -100

Problem 8 Read the following 8-bit values as signed numbers and give them in decimal. (1) `$F0` (2) `$80`

Problem 9 Adding `$01` to `$7F` in 8 bits gives `$80`. (1) Read as unsigned numbers, it changes from what value to what value? (2) Read as signed numbers, it changes from what value to what value?

Problem 10 Give the ASCII codes of the string `HI` in hexadecimal. The code of the character `7` is `$37`. Give the calculation that gets the number 7 from this character.

Problem 11 `$0300` holds `5A` and `$0301` holds `02`. If these 2 bytes are a 16-bit address, what is it? Also write how to check your answer with the Glow65 monitor.

Problem 12 How many bytes are in each of the following address ranges? (1) `$0400` to `$7FFF` (2) `$0300` to `$03F7`

(c) 2026 masanorisakai / MIT License

2.10 Answers and Solutions

Problem 1 (1) 181 (2) 102

- (1) 128 + 32 + 16 + 4 + 1 = 181
- (2) 64 + 32 + 4 + 2 = 102

Problem 2 (1) `%11001000` (2) `%01001101`

- (1) See 2.2 in the text. 128 + 64 + 8 = 200.
- (2) Subtracting 64 from 77 leaves 13. Subtracting 8, 4, and 1 from 13 leaves 0. Bits 6, 3, 2, and 0 are 1.

Problem 3 (1) 47 (2) 200

- (1) 2 x 16 + 15 = 47
- (2) 12 x 16 + 8 = 200

Problem 4 (1) `$64` (2) `$AD`

- (1) Dividing 100 by 16 gives a quotient of 6 and a remainder of 4.
- (2) Dividing 173 by 16 gives a quotient of 10 and a remainder of 13. 10 is A and 13 is D.

Problem 5 (1) `%01011010` (2) `%11100011`

- (1) 5 is `0101` and A is `1010`.
- (2) E is `1110` and 3 is `0011`.

Problem 6 `$96`

- `1001` is 9 and `0110` is 6.

Problem 7 (1) `$FD` (2) `$9C`

- (1) 3 is `%00000011`. Flipping gives `%11111100`, and adding 1 gives `%11111101`, which is `$FD`.
- (2) 100 is `$64`, or `%01100100`. Flipping gives `%10011011`, and adding 1 gives `%10011100`, which is `$9C`. Check: `$9C` is 156, and 156 - 256 is -100.

Problem 8 (1) -16 (2) -128

- (1) `$F0` is 240. 240 - 256 = -16.
- (2) `$80` is 128. 128 - 256 = -128. The value with only the most significant bit set to 1 is the smallest number in 8 bits.

Problem 9 (1) From 127 to 128 (2) From 127 to -128

- The same change in bits is only an increase of 1 when read as unsigned numbers. When read as signed numbers, the value jumps from the largest number to the smallest number.

Problem 10 `HI` is `$48 $49`. The number 7 is `$37 - $30 = 7`.

- `H` is `$48` and `I` is `$49`. Subtracting `$30` from a digit character gives the number.

(c) 2026 masanorisakai / MIT License

Problem 11 `$025A` (602 in decimal)

- The lower byte comes first, so the upper byte is `$02` and the lower byte is `$5A`.
- To check in the monitor, enter `E 0300 5A 02`, then display the contents with `D 0300 0301`. The display starts with `5A 02`.
- In BASIC, `POKE 768,90 : POKE 769,2` followed by `PRINT PEEK(769)*256+PEEK(768)` gives 602.

Problem 12 (1) 31,744 bytes (2) 248 bytes

- (1) `$7FFF – $0400 + 1`. `$7FFF` is 32767 and `$0400` is 1024. 32767 - 1024 + 1 = 31744.
- (2) `$03F7 – $0300 + 1`. 1015 - 768 + 1 = 248.

(c) 2026 masanorisakai / MIT License

Chapter 3: Inside the CPU

The CPU takes instructions from memory and executes them one at a time. This chapter explains the storage areas inside the 6502 (registers), the lines that carry signals between the CPU and memory (buses), and the steps for executing an instruction (fetch and execute).

3.1 The CPU and Memory

A program is a sequence of bytes placed in memory. The CPU reads the instruction bytes from the start of the program in order and acts on them: adding numbers, writing a value to memory, jumping to another place, and so on.

A register called the **PC** (program counter) remembers where the CPU reads next. Each time the CPU reads an instruction, the PC moves forward by the length of that instruction.

3.2 Registers

A **register** is a small storage area of a few bytes inside the CPU. It can be read and written faster than memory. The 6502 has the following six registers [1].

Register	Width	Role
A (accumulator)	8 bits	The center of calculation. Used for addition and comparison
X	8 bits	Index register. Used, for example, to add an offset to an address
Y	8 bits	A register with the same role as X
SP (stack pointer)	8 bits	The current position in the stack. The stack uses `$0100–$01FF`
PC (program counter)	16 bits	The address of the next instruction to execute
P (status register)	8 bits	A group of flags that describe the result of the previous instruction

A, X, and Y can be used freely by a program. SP is covered in Chapter 6. How to use X and Y is covered in Chapter 4.

Flags

Each bit of P has its own meaning. One such bit is called a **flag**. From the most significant bit down, they are arranged as follows.

Bit	7	6	5	4	3	2	1	0
Name	N	V	-	B	D	I	Z	C
Meaning	Negative. The most significant bit of the result is 1	Overflow. A signed result went out of range	Unused	Break	Decimal mode	Interrupt disable	Zero. The result is 0	Carry. A carry out of the top bit

(c) 2026 masanorisakai / MIT License

Chapter 6 explains what each flag does. For now, remember **N and Z**. Most instructions that put a value into A or X set Z to 1 if the value is 0, and set N to 1 if the most significant bit of the value is 1.

Values After Reset

After a reset (power-on or the RESET switch), the CPU starts from a fixed state. In Glow65, the values after reset are as follows.

Register	Value after reset
A, X, Y	0
SP	`$FD`
P	`$24` (`%00100100`). The I flag is 1, and bit 5 is 1
PC	The 2 bytes at addresses `$FFFC` and `$FFFD` (low byte first)

Section 3.7 explains how the PC is determined from `$FFFC` and `$FFFD`. On a real 6502, the internal state of the registers after reset is not defined, so the program must initialize them [1].

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

The panel has LEDs for PC, SP, A, X, Y, and the eight status flags. The status LEDs are arranged from left to right as `N`, `V`, `-`, `B`, `D`, `I`, `Z`, `C` (bit 7 down to bit 0). An LED is lit for a bit that is 1 and dark for a bit that is 0.

You can see the state right after reset with these steps.

1. Turn **POWER** ON.
2. Set **RUN/STOP** to STOP to halt the CPU.
3. Press **RESET**.

The CPU stays halted after the reset and shows the following state.

Display	LEDs	Value read
PC	15, 14, 13, 12, 8, 5, 3, 1 lit	`$F12A`
SP	7, 6, 5, 4, 3, 2, 0 lit	`$FD`
A, X, Y	all dark	0
STATUS P	`-` and `I` lit	`$24`
ADDRESS BUS	all lit except A1	`$FFFD`
DATA BUS	D7, D6, D5, D4, D0 lit	`$F1`
CYCLES	7	
INSTRUCTION AT PC	`F12A 78 SEI`	

The last byte read was at `$FFFD`, and its value `$F1` became the high byte of the PC. The READ and HALT LEDs are lit, and the RUN LED is dark. The PC value `$F12A` was determined from `2A` at `$FFFC` and `F1` at `$FFFD` (section 3.7).



Figure 3-1 The panel right after reset

3.3 Buses

The CPU and memory are connected by many lines. A bundle of such lines is called a **bus**.

(c) 2026 masanorisakai / MIT License

Bus / signal	Lines	Role
Address bus	16 lines (A0 to A15)	The CPU tells memory which address it reads or writes
Data bus	8 lines (D0 to D7)	The byte being read or written flows here
R/W	1 line	1 for a read, 0 for a write
SYNC	1 line	1 while the CPU reads the first byte of an instruction (the opcode)

The 16 address bus lines correspond to the 16-bit addresses from Chapter 2. The 8 data bus lines correspond to the 8 bits of one byte [1].

R/W shows whether the CPU is **reading** or **writing** memory. To read, the CPU puts out the address and the memory puts the byte on the data bus. To write, the CPU puts out both the address and the byte.

SYNC shows that the byte being read is the first byte of an instruction. SYNC stays 0 when the CPU reads a byte in the middle of an instruction, and when it writes [1].

In one clock (cycle), the CPU accesses the bus exactly once. It either reads or writes [1].

> **What you can try in Glow65**
>
> The panel has 16 ADDRESS BUS LEDs (A0 to A15) and 8 DATA BUS LEDs (D0 to D7). It also has READ, WRITE, and SYNC LEDs. READ lights while the CPU reads memory, WRITE while it writes, and SYNC while it reads an opcode. At a SPEED of 1 kHz or lower, the bus accesses are shown one at a time.

3.4 Fetch and Execute

The CPU runs a program by repeating the same steps.

1. **Fetch**: Read the first byte of the instruction (the **opcode**) from the address in the PC. Advance the PC by 1.
2. **Fetch the operand**: If the opcode needs more bytes, read them next. Advance the PC by 1 for each byte read. An operand is a byte that the instruction uses, either as a value or as part of an address.
3. **Execute**: Carry out the action of the instruction. For example, change a register, write to memory, or set the PC to another place.

Once the CPU has read the opcode, it knows how many bytes the instruction needs and what it does. Below is a table of the bus accesses when the five-line program from Tutorial Chapter 1 runs.

```
0300  A9 00        LDA #$00
0302  8D F0 03     STA $03F0
0305  18           CLC
0306  69 01        ADC #$01
0308  4C 02 03     JMP $0302
```

(c) 2026 masanorisakai / MIT License

#	Address	Data	Read/Write	SYNC	Instruction
1	`$0300`	`A9`	Read	yes	`LDA #$00`
2	`$0301`	`00`	Read		
3	`$0302`	`8D`	Read	yes	`STA $03F0`
4	`$0303`	`F0`	Read		
5	`$0304`	`03`	Read		
6	`$03F0`	`00`	Write		
7	`$0305`	`18`	Read	yes	`CLC`
8	`$0306`	`69`	Read	yes	`ADC #$01`
9	`$0307`	`01`	Read		
10	`$0308`	`4C`	Read	yes	`JMP $0302`
11	`$0309`	`02`	Read		
12	`$030A`	`03`	Read		

SYNC is 1 only when the first byte of an instruction is read. `STA $03F0` reads three times (the opcode and the 2-byte address), then writes once.

Cycles and the Clock

The CPU moves forward on a **clock**, a signal that ticks at a fixed interval. One period of the clock is one **cycle**. With a 1 MHz clock, there are 1 million cycles per second, and one cycle takes 1 microsecond.

The number of cycles an instruction takes depends on the instruction. According to MOS Technology's documentation, instructions take from 2 to 7 cycles [1]. `LDA #$00` takes 2 cycles and `STA $03F0` takes 4 cycles.

On a real 6502, even a 1-byte instruction such as `CLC` takes 2 cycles, and the CPU reads the next byte once more. That extra read is not used [1]. Glow65 does not reproduce this extra read, which is why `CLC` has only one access in the table. The cycle counter still advances by 2, as on the real chip.

> **Column: Single-Stepping with SYNC**
>
> MOS Technology's documentation describes a use for the SYNC signal. SYNC is 1 in the cycle that reads an opcode. If the CPU's RDY pin is set to 0 at that moment, the CPU stops in that state. Setting RDY back to 1 resumes it. This mechanism makes it possible to build single-stepping, which executes one instruction at a time [1]. The STEP switch in Glow65 is also a feature for advancing one instruction or one bus access at a time.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

You can follow the action cycle by cycle on the panel. After resetting as in section 3.2, set **STEP MODE** to CYCLE and press **STEP** once for each step. The CPU runs the program at the start of the monitor (`SEI`, `CLD`, `LDX #$FF`, `TXS`).

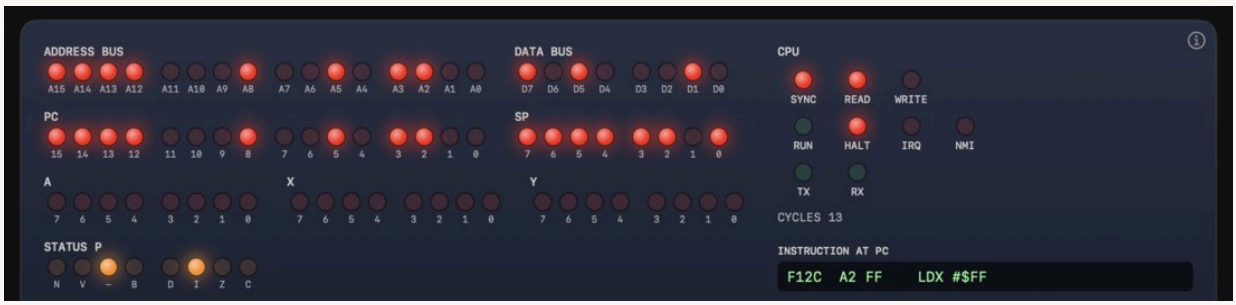
STEP	ADDRESS BUS	DATA BUS	SYNC	What happened in that STEP
Before pressing	`$FFFD`	`$F1`		The read made by the reset (3.2)
1st	`$F12A`	`$78`	lit	The opcode of `SEI` was read. It is an instruction that needs only one read, so the PC advances to `$F12B` and INSTRUCTION AT PC becomes `CLD`
2nd	`$F12B`	`$D8`	lit	The opcode of `CLD` was read. The PC is `$F12C`
3rd	`$F12C`	`$A2`	lit	The opcode of `LDX #$FF` was read
4th	`$F12D`	`$FF`	dark	The operand of `LDX` was read
5th	`$F12D`	`$FF`	dark	The bus does not change, and the result of `LDX` appears in the display

Between the 3rd and 4th STEP, the PC stays at `$F12C` and X stays 0. With a 2-byte instruction, **until the instruction has been read completely, the PC, registers, and flags show their values from before the instruction**. At the 5th STEP, X becomes `$FF`, the N flag lights, and the PC becomes `$F12E` (INSTRUCTION AT PC is `TXS`).

CYCLES advances by the instruction's cycle count at the first bus access of the instruction. It goes from 7 to 9 at the STEP for `SEI`, to 11 at `CLD`, and to 13 at the 3rd STEP of `LDX #$FF`.



Figure 3-2 1st STEP. The opcode of `SEI` has just been read



(c) 2026 masanorisakai / MIT License

Figure 3-3 3rd STEP. The opcode of `LDX #$FF` has just been read. The PC is `$F12C` and X is still 0



Figure 3-4 4th STEP. The operand of `LDX` has just been read. SYNC is dark and X is still 0



Figure 3-5 5th STEP. The buses do not change. X becomes `$FF`, N lights, and the PC becomes `$F12E`

If you set SPEED to 10 Hz and switch to RUN, the same actions run continuously. The low LEDs of the address bus change each time the PC advances.

If you set STEP MODE to INSTR, each press of STEP advances one instruction. The bus keeps the last bus access of that instruction. The detailed steps are in Tutorial Chapter 1.

3.5 Instruction Format

An instruction consists of a 1-byte opcode and 0 to 2 bytes of operand. An instruction is 1 to 3 bytes long.

Length	Structure	Example
1 byte	Opcode only	`AA` (`TAX`)
2 bytes	Opcode + 1 byte	`A9 00` (`LDA #$00`)
3 bytes	Opcode + 2 bytes (low byte first)	`8D F0 03` (`STA $03F0`)

The opcode determines the length. Even for the same `LDA`, `A9` is 2 bytes and `AD` is 3 bytes. The meaning of the operand (a value itself, or an address) is determined by the **addressing mode**. Chapter 4 covers this in detail.

In a 3-byte instruction, the address is stored in little-endian order, as in Chapter 2. In `8D F0 03`, `F0 03` is the address `$03F0`.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

Write bytes with `E` in the monitor and read them back as instructions with `L`.

```
*E 0300 A9 00 8D F0 03
*L 0300 0304
0300  A9 00     LDA   #$00
0302  8D F0 03  STA   $03F0
```

`L 0300 0304` disassembles the range from `$0300` to `$0304`. You can see that `LDA` is 2 bytes and `STA` is 3 bytes from how the addresses increase.

The same byte sequence becomes different instructions if you change where you start reading.

```
*E 0300 AD 34 12 A9 10 8D 00 04 AA
*L 0300 0308
0300  AD 34 12  LDA   $1234
0303  A9 10     LDA   #$10
0305  8D 00 04  STA   $0400
0308  AA        TAX
```

`AD 34 12` is the 3-byte `LDA $1234`. `34 12` becomes `$1234` in little-endian order.

3.6 An Example of Executing Instructions

Follow the next program through its bus accesses and its changes to registers and flags.

```
0300  A9 80        LDA #$80
0302  AA           TAX
0303  8D F0 03     STA $03F0
0306  8E F1 03     STX $03F1
0309  60           RTS
```

Instruction	Bus accesses	A	X	N	Z	Change in memory
`LDA #$80`	2 reads (opcode, value)	`$80`	unchanged	1	0	none
`TAX`	1 read (opcode)	`$80`	`$80`	1	0	none
`STA $03F0`	3 reads, 1 write	`$80`	`$80`	1	0	`$03F0` becomes `$80`
`STX $03F1`	3 reads, 1 write	`$80`	`$80`	1	0	`$03F1` becomes `$80`

- `LDA #$80` puts `$80` into A. The most significant bit of `$80` is 1, so N becomes 1. The value is not 0, so Z is 0.
- `TAX` copies the value of A into X. A stays as it is. N and Z are set according to the `$80` that went into X.

(c) 2026 masanorisakai / MIT License

- `STA` and `STX` write a register's value to memory. The registers and flags do not change.
- `RTS` is the instruction that returns to the monitor that called the program with `G`. Chapter 6 covers it.

When the value is 0, Z becomes 1 and N becomes 0. After `LDA #$00`, only Z is 1.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65 (panel)

Reset as in section 3.2, set STEP MODE to INSTR, and press STEP to advance one instruction at a time. With the program at the start of the monitor, you can watch the registers and flags change.

STEP	Instruction executed	Change in registers and flags	ADDRESS BUS / DATA BUS	Next instruction
1st	`SEI`	No change (the I flag was already 1)	`$F12A` / `$78` (SYNC lit)	`CLD` (`$F12B`)
2nd	`CLD`	No change (the D flag was already 0)	`$F12B` / `$D8` (SYNC lit)	`LDX #$FF` (`$F12C`)
3rd	`LDX #$FF`	X becomes `$FF` and N lights	`$F12D` / `$FF` (operand of `LDX`)	`TXS` (`$F12E`)
4th	`TXS`	SP becomes `$FF` (all SP LEDs lit). N stays lit	`$F12E` / `$9A` (SYNC lit)	`LDA #$03` (`$F12F`)
5th	`LDA #$03`	A becomes `$03` (bits 1 and 0 lit). N goes dark	`$F130` / `$03`	`STA $A000` (`$F131`)

The `$FF` of `LDX #$FF` has its most significant bit set, so N lights. `TXS` only copies the value of X into SP and does not change the flags. `LDA #$03` has a most significant bit of 0 and is not 0, so neither N nor Z lights.

The STEP counts start from right after the reset in section 3.2.



Figure 3-6 After the 3rd STEP. `LDX #$FF` has run, X is `$FF`, and N is lit

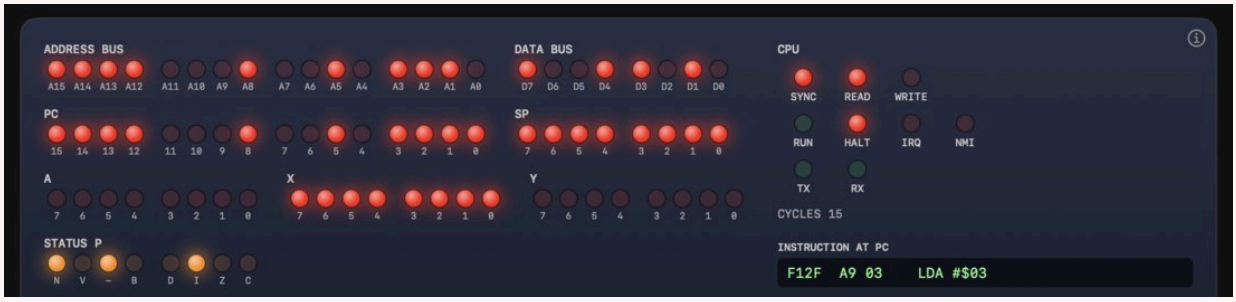


Figure 3-7 After the 4th STEP. `TXS` has run, and all SP LEDs are lit

(c) 2026 masanorisakai / MIT License

What you can try in Glow65 (monitor)

The monitor has no command to display the registers. Instead, the program writes its result to memory and you read it with `D`.

First, run the five instructions above.

```
*E 0300 A9 80 AA 8D F0 03 8E F1 03 60
*L 0300 0309
0300  A9 80     LDA   #$80
0302  AA        TAX
0303  8D F0 03  STA   $03F0
0306  8E F1 03  STX   $03F1
0309  60        RTS
*G 0300
*D 03F0 03F1
03F0: 80 80 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

Both `$03F0` and `$03F1` are `80`. `D` shows 16 bytes at a time, so the following `24 F4 24 F4` are the monitor's interrupt vectors. Chapter 6 explains them.

Next, check the flags. `PHP` pushes P onto the stack, and `PLA` pulls the pushed value into A. `PHP` pushes with bit 4 (B) and bit 5 set to 1. Ignore those two and look at N and Z.

```
*E 0300 A9 80 08 68 8D F0 03 A9 00 08 68 8D F1 03 60
*G 0300
*D 03F0 03F1
03F0: B4 36 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

After `LDA #$80`, P is `$B4` (`%10110100`), and N is 1. After `LDA #$00`, P is `$36` (`%00110110`), and Z is 1. In both, the I flag (bit 2) is 1.

3.7 Vectors for Reset and Interrupts

On a reset or an interrupt, the CPU reads the address to jump to from a fixed place in memory. This place is called a **vector**. The vectors are in the last 6 bytes of the address space [1].

Address	Use
`$FFFA` `$FFFB`	NMI (non-maskable interrupt)
`$FFFC` `$FFFD`	Reset
`$FFFE` `$FFFF`	IRQ and BRK

Each vector is 2 bytes, low byte first. After a reset, the CPU reads `$FFFC` and `$FFFD` and puts that 16-bit address into the PC. The program starts from that address. Chapter 6 explains interrupts in detail.

(c) 2026 masanorisakai / MIT License

In Glow65, the vectors are in the monitor's ROM (see the `D FFF0 FFFF` example in Chapter 2).

3.8 Practice Problems

The answers and explanations are in the next section.

Problem 1 How many bits wide are the PC and A registers?

Problem 2 What are the values of SP and P right after a reset in Glow65, in hexadecimal? Which flags are 1 in P?

Problem 3 After a reset, where is the address of the first instruction the CPU executes taken from? If `$FFFC` and `$FFFD` of Glow65 contain `2A F1`, what is the PC?

Problem 4 When `STA $03F0` (`8D F0 03`) is executed once, how many bus accesses are there? Give the numbers of reads and writes separately. How many times is SYNC 1?

Problem 5 Split the byte sequence `A9 10 8D 00 04 AA` into instructions. Give the length of each instruction and the total number of bytes.

Problem 6 After `LDA #$00` is executed, which of N and Z is 1? Which one after `LDA #$FF`?

Problem 7 In Glow65, you enter `E 0300 A2 05 8E F0 03 60`, run it with `G 0300`, and enter `D 03F0 03F0`. What is the value of `$03F0`?

Problem 8 You read P with the method in 3.6 and got `$B4`. Which flags are 1? List all of them.

Problem 9 After running `A9 7F 08 68 8D F0 03 60`, what is the value of `$03F0`?

3.9 Answers and Explanations

Problem 1 The PC is 16 bits and A is 8 bits.

- The PC holds a 16-bit address. A handles one byte of data.

Problem 2 SP is `$FD` and P is `$24`. The flags that are 1 are the I flag (bit 2) and bit 5.

- `$24` is `%00100100`. Bit 5 is unused and always 1. Bit 2 is I.

Problem 3 It is taken from `$FFFC` and `$FFFD`. The PC is `$F12A`.

- The low byte comes first, so `2A` at `$FFFC` is the low byte and `F1` at `$FFFD` is the high byte.

Problem 4 3 reads and 1 write. SYNC is 1 once.

- Reads: the opcode `8D`, the low byte `F0`, and the high byte `03`. Write: once, to `$03F0`. SYNC is 1 only when the opcode is read.

Problem 5 `A9 10` = `LDA #$10` (2 bytes), `8D 00 04` = `STA $0400` (3 bytes), `AA` = `TAX` (1 byte). 6 bytes in total.

(c) 2026 masanorisakai / MIT License

- `A9` is followed by 1 value byte, `8D` is followed by a 2-byte address, and `AA` has no operand.

Problem 6 After `LDA #$00`, Z. After `LDA #$FF`, N.

- `$00` is 0, so Z is 1. `$FF` has its most significant bit set, so N is 1.

Problem 7 `$05`.

- `A2 05` is `LDX #$05`, `8E F0 03` is `STX $03F0`, and `60` is `RTS`. The 5 in X is written to `$03F0`.

Problem 8 N and I. In addition, bit 4 (B) and bit 5, which PHP sets to 1 when it pushes.

- `$B4` is `%10110100`. Bits 7 (N), 5, 4 (B), and 2 (I) are 1.

Problem 9 `$34`.

- `LDA #$7F` has a most significant bit of 0 and is not 0, so N and Z are both 0. I (bit 2) is 1, and PHP sets bits 4 and 5 to 1. That gives `%00110100`, which is `$34`.

Sources

1. MOS Technology, "MCS6500 Microcomputer Family Hardware Manual", January 1976 edition: https://www.commodore.ca/wp-content/uploads/2018/11/kim1_hardware.txt

(c) 2026 masanorisakai / MIT License

Chapter 4 Addressing Modes

The operand of an instruction decides which value in memory the instruction works on. The way it decides is called the **addressing mode**. The 6502 has 13 modes.

This chapter explains, for each mode, how it is written, how many bytes it takes, and how the effective address is found, using diagrams.

4.1 What Is an Addressing Mode

Even for the same `LDA` (the instruction that loads a value into the A register), a different mode means a different action and a different number of bytes. As seen in section 3.5 of Chapter 3, the opcode determines the length of an instruction. The mode is part of the opcode.

Notation	Mode	Opcode	Bytes
`LDA #$10`	Immediate	`A9`	2
`LDA $10`	Zero page	`A5`	2
`LDA $10,X`	Zero page,X	`B5`	2
`LDA $0400`	Absolute	`AD`	3
`LDA $0410,X`	Absolute,X	`BD`	3
`LDA $0420,Y`	Absolute,Y	`B9`	3
`LDA ($20,X)`	Indexed indirect	`A1`	2
`LDA ($30),Y`	Indirect indexed	`B1`	2

The address in memory that the operand points to is called the **effective address**. The CPU first works out the effective address, then reads or writes that address. The difference between addressing modes is the difference in how the effective address is found.

How to Read the Diagrams

All the diagrams in this chapter are built from the same parts.

- Top row: the bytes of the instruction. The leftmost is the opcode, and the blue boxes are the operand.
- Left: the registers. The orange box is the register that receives the result.
- White box in the center: the calculation that finds the effective address.
- Right: memory. Addresses and contents are lined up, and the blue box is the target byte.
- Arrow numbers: the order of the steps the CPU takes. The orange arrow shows the result going into a register.

The Form of the Examples

The examples in this chapter all have the same form. The result of the instruction is written to memory with `STA $03F0`, and `RTS` returns to the monitor. You run it with `G 0300` and read the value of `$03F0` with `D`

(c) 2026 masanorisakai / MIT License

`03F0 03F0`. `D` shows 16 bytes at a time, so the first byte of the line is the result. The following `24 F4 24 F4` are the monitor's interrupt vectors.

The program is placed at `$0300`. Data is written to RAM with `E`.

> **What you can try in Glow65**
>
> If you write the following bytes at `$0300` and read them with `L`, the notation of each mode is shown as it is.

```
*E 0300 E8 0A A9 10 A5 10 B5 10 B6 10 AD 00 04 BD 10 04 B9 20 04 6C 30 04 A1 20 B1
30 D0 FE
*L 0300 0319
0300  E8        INX
0301  0A        ASL   A
0302  A9 10     LDA   #$10
0304  A5 10     LDA   $10
0306  B5 10     LDA   $10,X
0308  B6 10     LDX   $10,Y
030A  AD 00 04  LDA   $0400
030D  BD 10 04  LDA   $0410,X
0310  B9 20 04  LDA   $0420,Y
0313  6C 30 04  JMP   ($0430)
0316  A1 20     LDA   ($20,X)
0318  B1 30     LDA   ($30),Y
```

4.2 Modes Without an Operand

Implied

These instructions have no operand, only an opcode. The name of the instruction already contains its target (a register or a flag). Examples are `INX` (add 1 to X), `TAX`, and `CLC`. The length is 1 byte.

(c) 2026 masanorisakai / MIT License

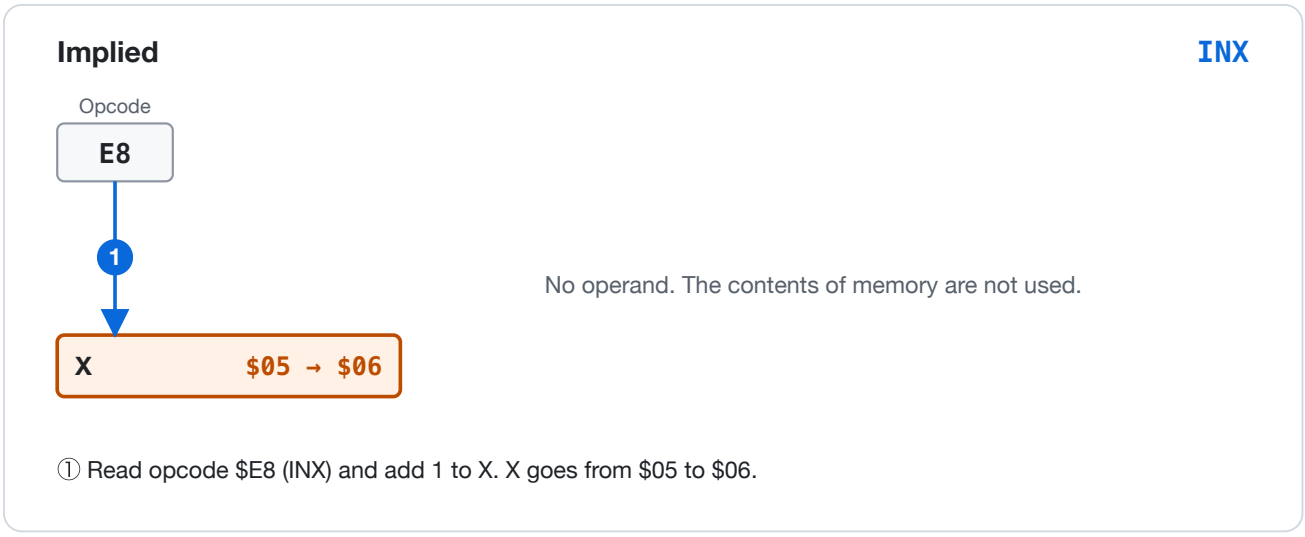


Figure 4-1 Implied (`INX`). The contents of memory are not used.

What you can try in Glow65

```
*E 0300 A2 05 E8 8E F0 03 60
*G 0300
*D 03F0 03F0
03F0: 06 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`LDX #$05` makes X 5, and `INX` makes it 6.

Accumulator

The target of the operation is the A register itself. Four instructions have this mode: `ASL A` (shift left by 1 bit), `LSR A`, `ROL A`, and `ROR A` (the 65C02 adds `INC A` and `DEC A`). The length is 1 byte.

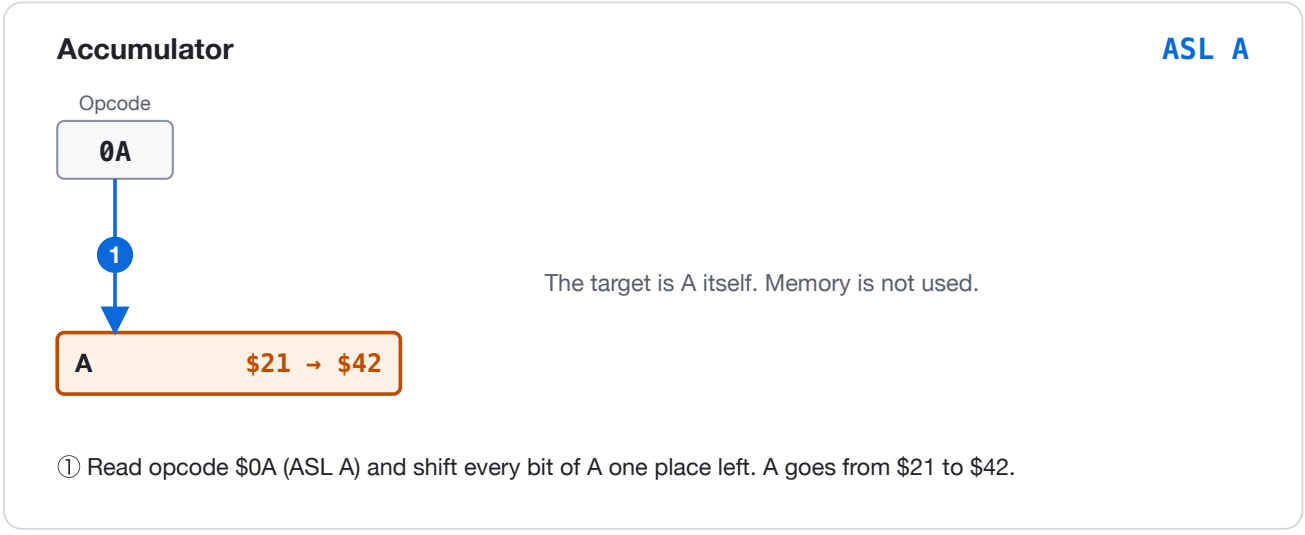


Figure 4-2 Accumulator (`ASL A`).

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

```
*E 0300 A9 21 0A 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 42 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

Shifting `$21` (`%00100001`) left by 1 bit gives `$42` (`%01000010`).

4.3 Immediate

The value in the operand is used as it is. It is not a memory address. You write it with `#` in front of the value.

Item	Content
Notation	`LDA #$10`
Machine code	`A9 10`
Bytes	2
Effective address	None. The operand is the value itself

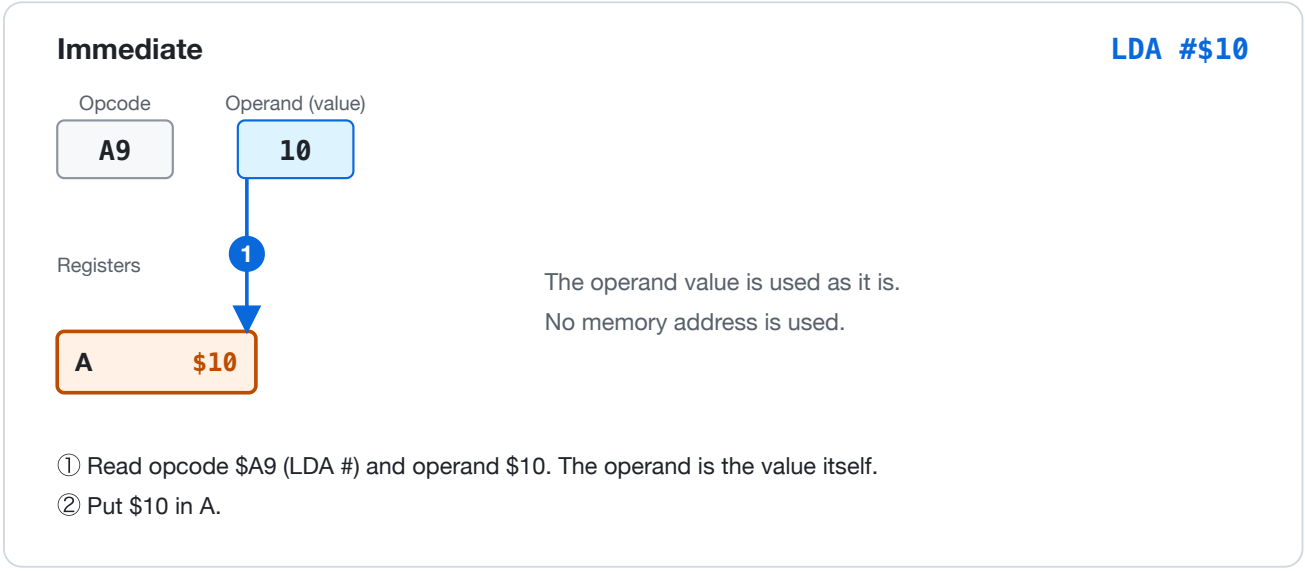


Figure 4-3 Immediate (`LDA #$10`).

`LDA #$10` means "put the value `$10` into A", and `LDA $10` means "put the contents of address `$0010` into A". The meaning changes depending on whether `#` is there.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

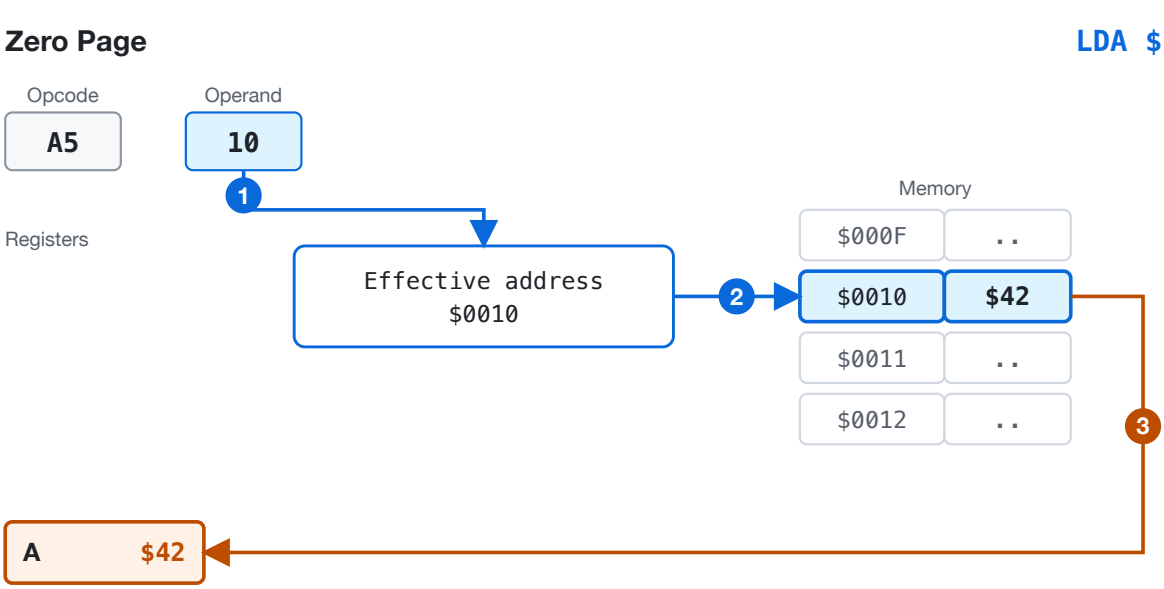
```
*E 0300 A9 10 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 10 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

4.4 Zero Page Modes

The zero page (the first 256 bytes, `$0000–$00FF`) can be specified with a 1-byte operand. This makes instructions shorter and faster to execute.

Zero Page

Item	Content
Notation	`LDA $10`
Machine code	`A5 10`
Bytes	2
Effective address	`$0010` (the high byte is `$00`)



① Read opcode $A5 and operand $10.
The operand itself is the effective address $0010.
② Read the contents of $0010: $42.
③ Put $42 in A.

Figure 4-4 Zero page (`LDA $10`).

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

```
*E 0010 42
*E 0300 A5 10 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 42 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

Zero Page,X

The address used is the operand plus the value of X. Use it to read data lined up like an array, one item after another.

Item	Content
Notation	`LDA $10,X`
Machine code	`B5 10`
Bytes	2
Effective address	Operand + X. The result is the remainder after dividing by 256 (if it goes past `$FF`, it wraps around to `$00`)

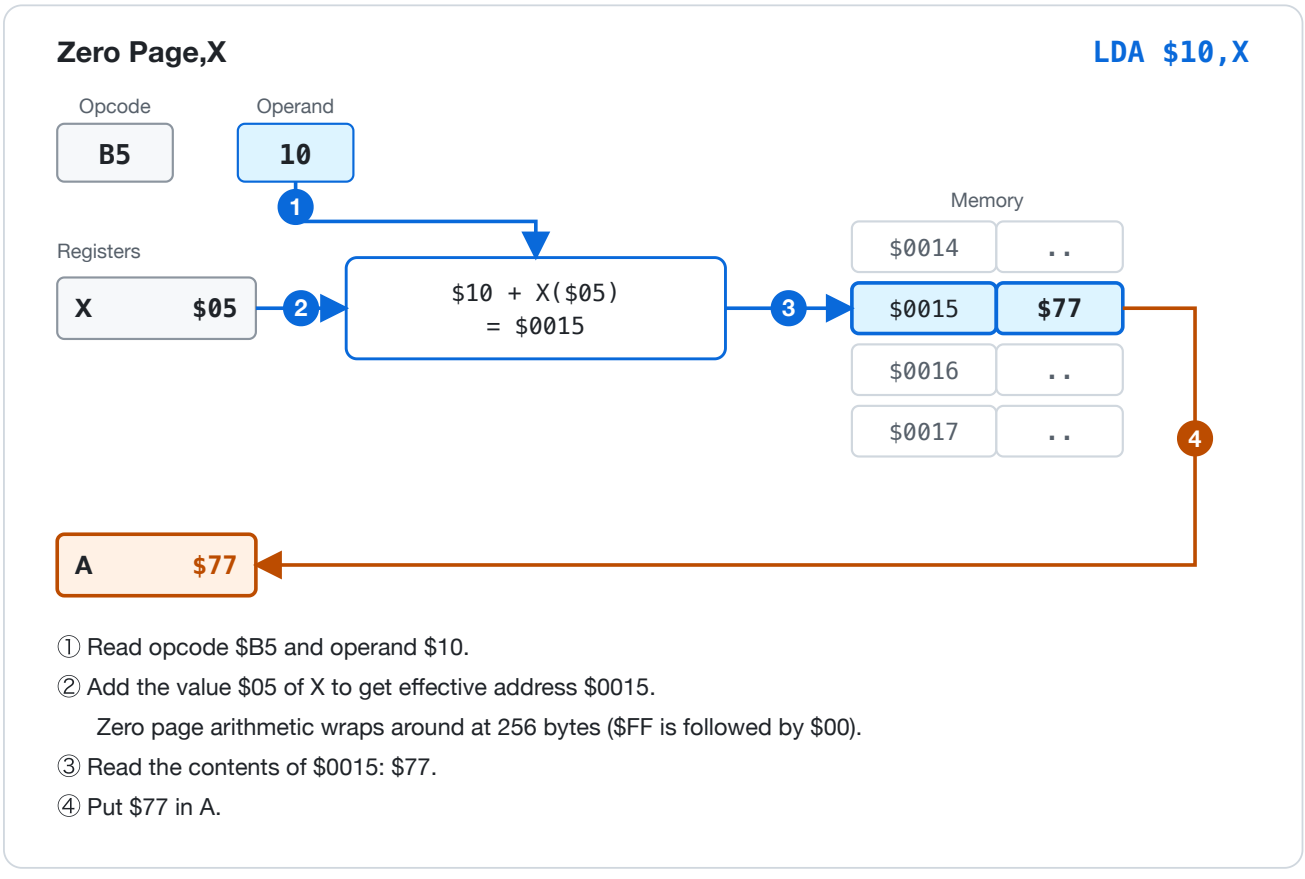


Figure 4-5 Zero page,X (`LDA $10,X`). When X is `$05`, it reads `$0015`.

(c) 2026 masanorisakai / MIT License

The result of the calculation stays inside the zero page. For `LDA $FF,X` with X equal to `$02`, the effective address is `$0001`, not `$0101`. If the sum goes past `$FF`, counting starts again from `$00` [1].

What you can try in Glow65

```
*E 0015 77
*E 0300 A2 05 B5 10 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 77 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

You can check the wrap-around as follows. `LDA $FF,X` reads `$0001`, while the absolute,X `LDA $00FF,X` (whose second byte is `$00`) reads `$0101`.

```
*E 0001 7B
*E 0101 3D
*E 0300 A2 02 B5 FF 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 7B 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
*E 0300 A2 02 BD FF 00 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 3D 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

Zero Page,Y

This works like zero page,X, except that the register added is Y. Only `LDX` and `STX` have this mode.

Item	Content
Notation	`LDX $10,Y`
Machine code	`B6 10`
Bytes	2
Effective address	Operand + Y. Wraps around at 256

(c) 2026 masanorisakai / MIT License

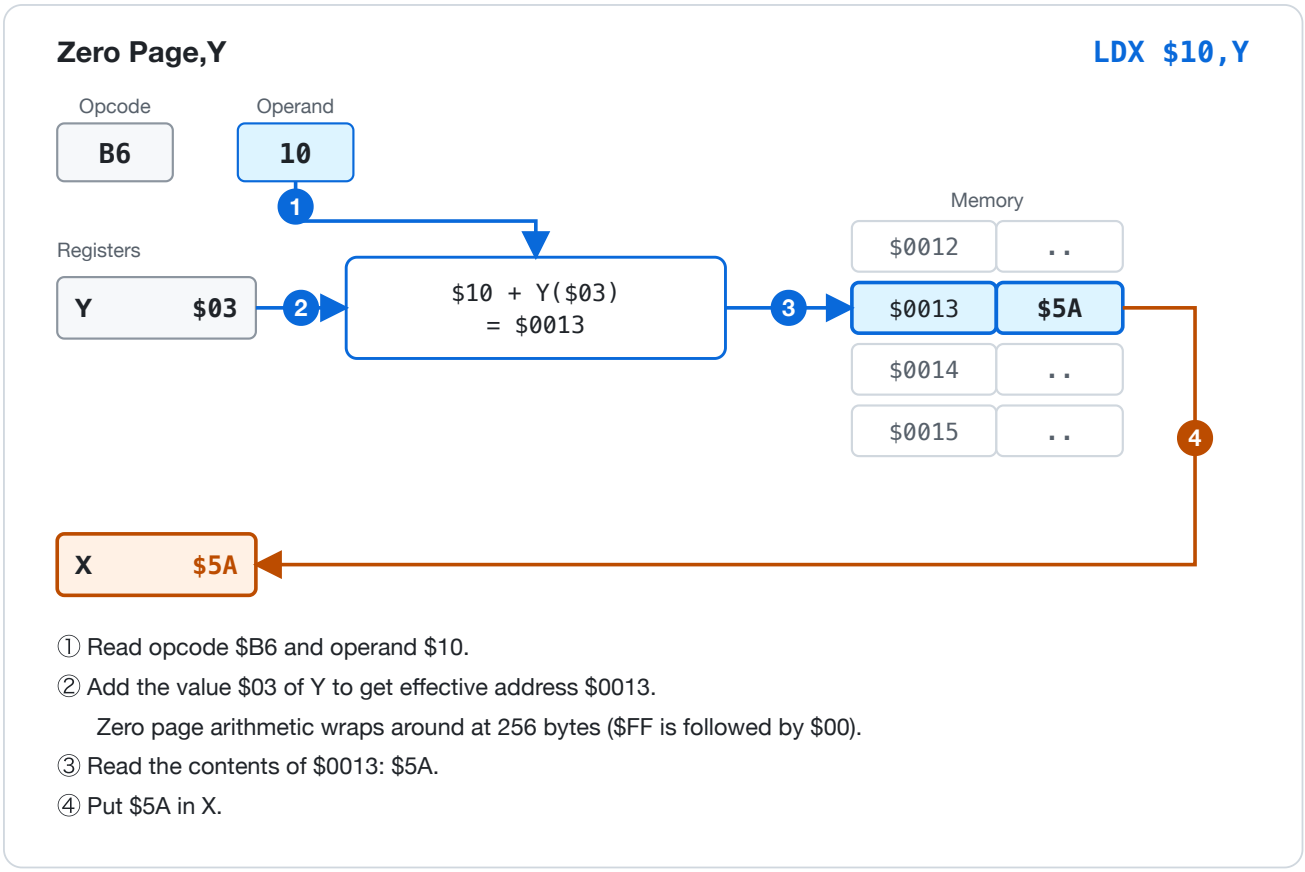


Figure 4-6 Zero page,Y (`LDX $10,Y`). The result goes into X.

What you can try in Glow65

```
*E 0013 5A
*E 0300 A0 03 B6 10 8E F0 03 60
*G 0300
*D 03F0 03F0
03F0: 5A 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

4.5 Absolute Modes

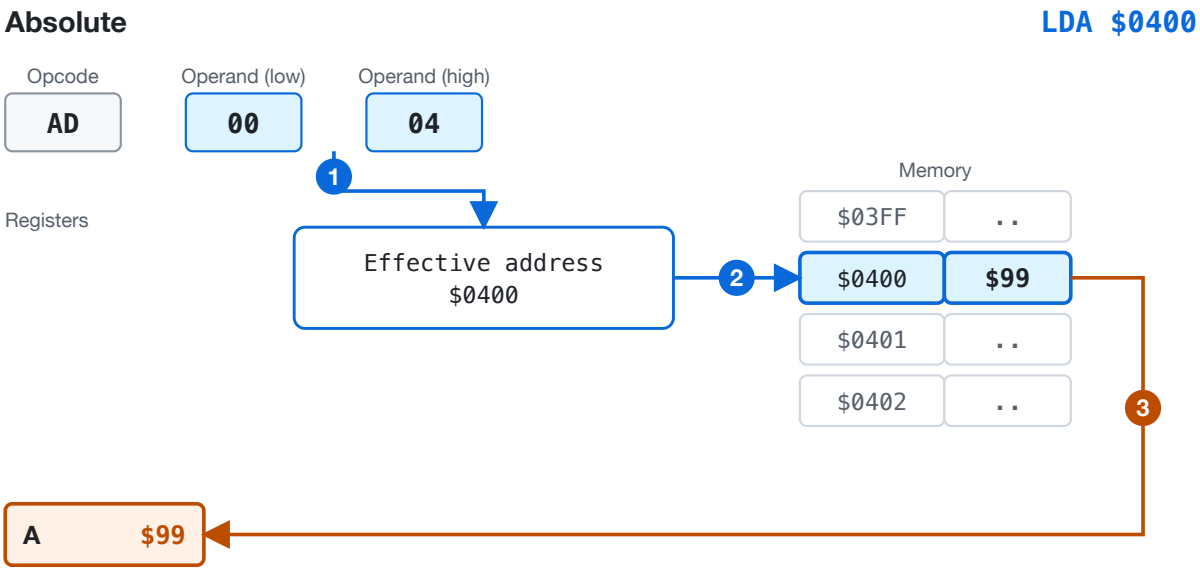
Any address in the 64 KB address space can be specified with a 2-byte operand. The operand is little-endian (low byte first).

Absolute

Item	Content
Notation	`LDA $0400`
Machine code	`AD 00 04`
Bytes	3

(c) 2026 masanorisakai / MIT License

Item	Content
Effective address	The 2 bytes of the operand (`$0400`)



① Read opcode $AD and operand $0400.
The operand itself is the effective address $0400.
② Read the contents of $0400: $99.
③ Put $99 in A.

Figure 4-7 Absolute (`LDA $0400`).

What you can try in Glow65

```
*E 0400 99
*E 0300 AD 00 04 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 99 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

Absolute,X

The address used is the 2-byte operand plus the value of X.

Item	Content
Notation	`LDA $0410,X`
Machine code	`BD 10 04`
Bytes	3
Effective address	Operand + X (a 16-bit addition)

(c) 2026 masanorisakai / MIT License

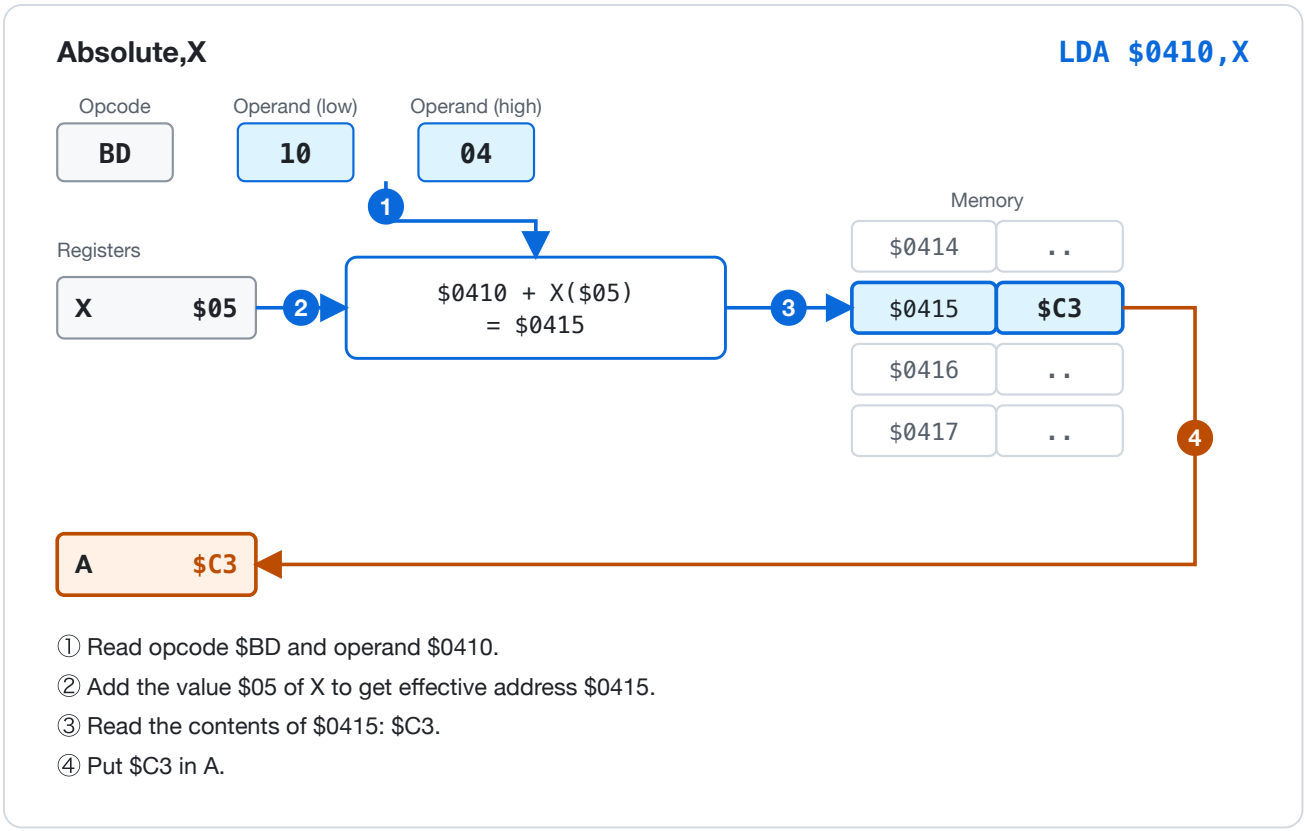


Figure 4-8 Absolute,X (`LDA $0410,X`).

When the sum lands on a different page from the original address (an address with a different high byte), this is called **crossing a page**. For `LDA $04FF,X` with X equal to `$01`, the effective address is `$0500`, which crosses a page. On a real 6502, a read instruction such as `LDA` takes 1 extra cycle when it crosses a page[1]. A write instruction such as `STA` always takes the longer cycle count, even when it does not cross a page.

What you can try in Glow65

```
*E 0415 C3
*E 0300 A2 05 BD 10 04 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: C3 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

The value read is found in the same way when a page is crossed.

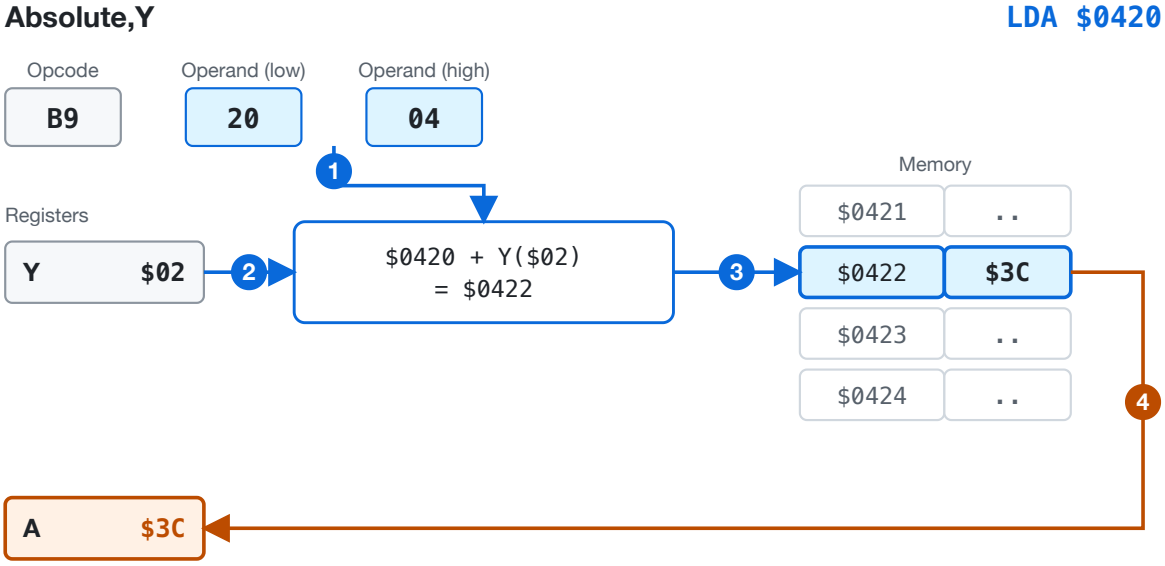
```
*E 0500 5E
*E 0300 A2 01 BD FF 04 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 5E 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

(c) 2026 masanorisakai / MIT License

Absolute,Y

This works like absolute,X, except that the register added is Y.

Item	Content
Notation	`LDA $0420,Y`
Machine code	`B9 20 04`
Bytes	3
Effective address	Operand + Y



① Read opcode $B9 and operand $0420.
② Add the value $02 of Y to get effective address $0422.
③ Read the contents of $0422: $3C.
④ Put $3C in A.

Figure 4-9 Absolute,Y (`LDA $0420,Y`).

What you can try in Glow65

```
*E 0422 3C
*E 0300 A0 02 B9 20 04 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 3C 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

The effective address appears on the panel as the address on the bus. Use this program:

```
*E 0410 00 00 00 00 00 C3
*E 0010 42
*E 0300 A2 05 BD 10 04 A5 10 A9 10 4C 00 03
*G 0300
```

It repeats `LDX #$05`, `LDA $0410,X`, `LDA $10`, `LDA #$10`, and `JMP $0300`. Set **RUN/STOP** to STOP and **STEP MODE** to INSTR, then press STEP once at a time. After each instruction, its last bus access stays on the ADDRESS BUS and DATA BUS.

Instruction run	ADDRESS BUS	DATA BUS	A
`LDA $0410,X` (X = `$05`)	`$0415`	`$C3`	`$C3`
`LDA $10`	`$0010`	`$42`	`$42`
`LDA #$10`	`$0308`	`$10`	`$10`

The absolute,X instruction `LDA $0410,X` reads `$0415`, the operand `$0410` plus the `$05` in X (Figure 4-14). The zero page instruction `LDA $10` reads `$0010` (Figure 4-15). The immediate instruction `LDA #$10` reads the byte right after the instruction, at `$0308` (Figure 4-16).



Figure 4-14 After `LDA $0410,X`. The ADDRESS BUS shows the effective address `$0415`, the DATA BUS and A show `$C3`, and X is `$05`



Figure 4-15 After `LDA $10`. The ADDRESS BUS is `$0010`, and the DATA BUS and A are `$42`

(c) 2026 masanorisakai / MIT License



Figure 4-16 After `LDA #$10`. The ADDRESS BUS is `$0308`, just after the instruction, and the DATA BUS and A are `$10`

4.6 Indirect

The 2 bytes stored at the address in the operand are used as the jump target. Only `JMP` has this mode.

Item	Content
Notation	`JMP ($0430)`
Machine code	`6C 30 04`
Bytes	3
Jump target	The byte at `$0430` is the low byte, and the byte at `$0431` is the high byte

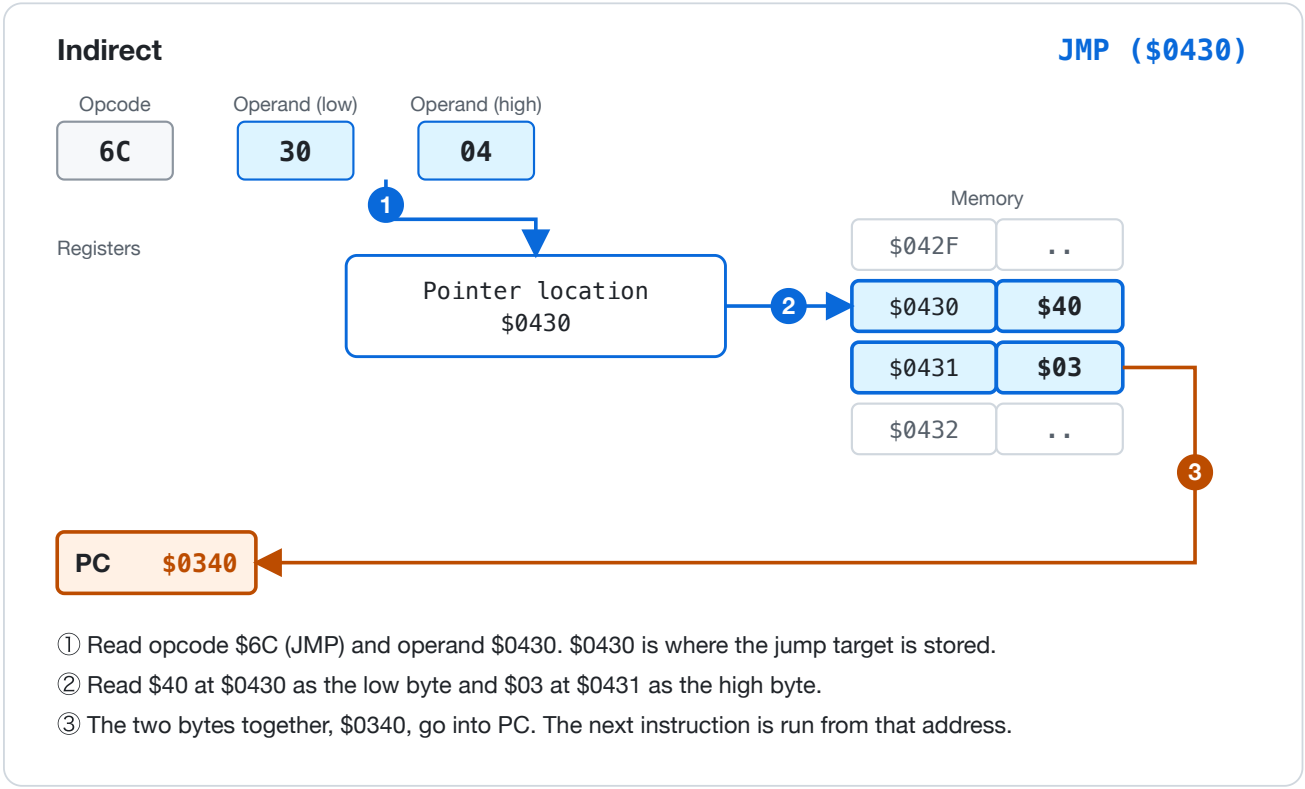


Figure 4-10 Indirect (`JMP ($0430)`). The result goes into the PC.

The NMOS 6502 has a quirk in this mode. It appears when the pointer is at the last byte of a page (`$xxFF`). The high byte is read not from the start of the next page but from **the start of the same page**. For `JMP`

(c) 2026 masanorisakai / MIT License

`($04FF)`, the low byte is read from `$04FF` and the high byte from `$0400`, not from `$0500`. The 65C02 has fixed this quirk.

What you can try in Glow65

```
*E 0430 40 03
*E 0340 A9 AB 8D F0 03 60
*E 0300 6C 30 04
*G 0300
*D 03F0 03F0
03F0: AB 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

The program starting at `$0340` writes `$AB`.

You can check the quirk at the last byte of a page as follows. Put `40` at `$04FF`, `05` at `$0500`, and `03` at `$0400`. If the high byte is read from `$0500`, the jump goes to `$0540`. If it is read from `$0400`, the jump goes to `$0340`.

```
*E 04FF 40
*E 0500 05
*E 0400 03
*E 0340 A9 AB 8D F0 03 60
*E 0540 A9 CD 8D F0 03 60
*E 0300 6C FF 04
*G 0300
*D 03F0 03F0
03F0: AB 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

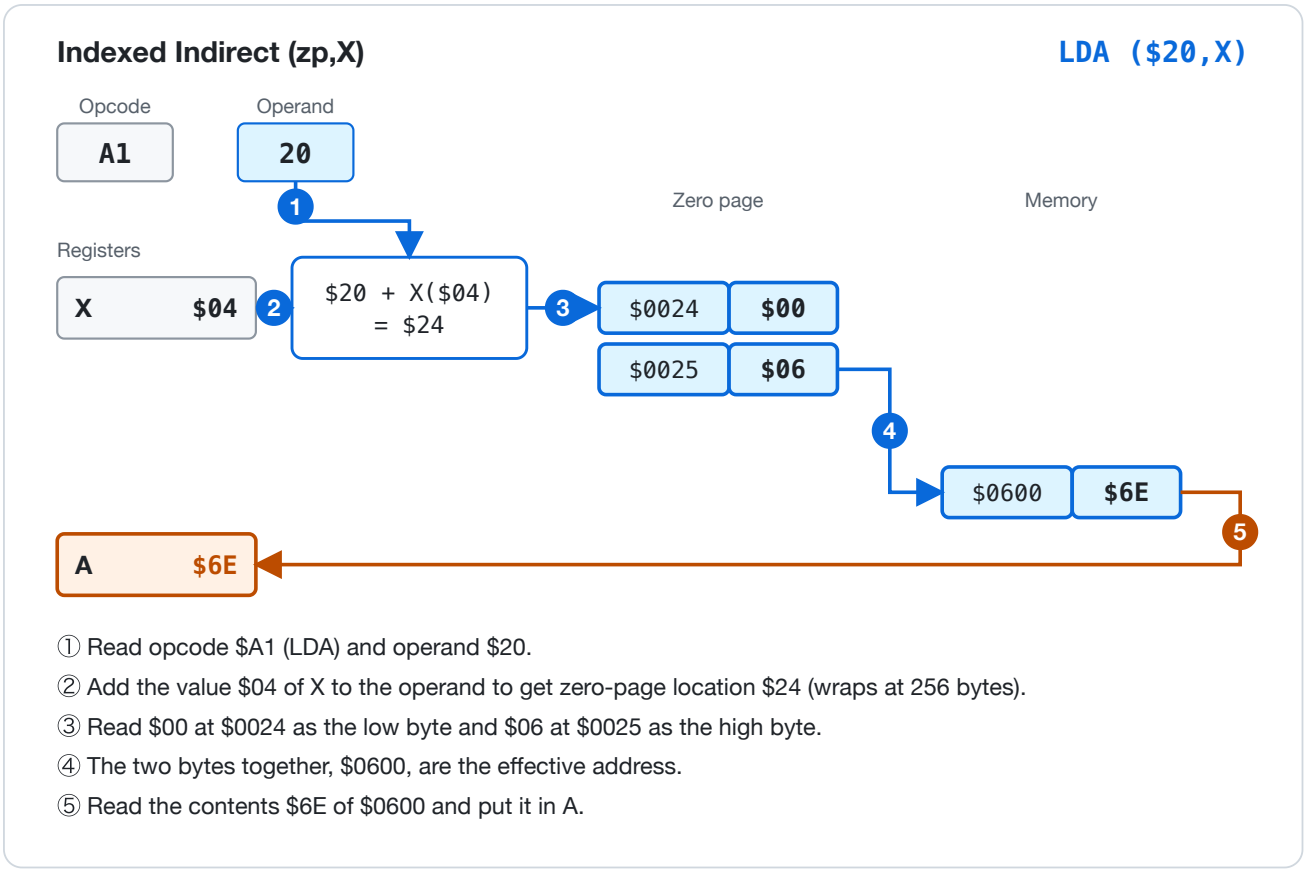
With the default NMOS 6502, the jump goes to `$0340` and `AB` is written. If you do the same with `glow65-cli --cpu 65c02`, the jump goes to `$0540` and `CD` is written.

4.7 Indexed Indirect

You place a table of addresses (2 bytes each) in the zero page. The instruction takes the address that X selects from the table and uses the contents of that address. It is written `(zp,X)`.

Item	Content
Notation	`LDA ($20,X)`
Machine code	`A1 20`
Bytes	2
Effective address	The 2 bytes (low byte first) at `$20 + X` in the zero page. The sum wraps around at 256

(c) 2026 masanorisakai / MIT License



① Read opcode $A1 (LDA) and operand $20.
② Add the value $04 of X to the operand to get zero-page location $24 (wraps at 256 bytes).
③ Read $00 at $0024 as the low byte and $06 at $0025 as the high byte.
④ The two bytes together, $0600, are the effective address.
⑤ Read the contents $6E of $0600 and put it in A.

Figure 4-11 Indexed indirect (`LDA ($20,X)`). The address `$0600` is stored in zero page `$0024` and `$0025`.

X selects which entry of the table to use. An address is 2 bytes, so X is increased in steps of 2.

What you can try in Glow65

```
*E 0024 00 06
*E 0600 6E
*E 0300 A2 04 A1 20 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 6E 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

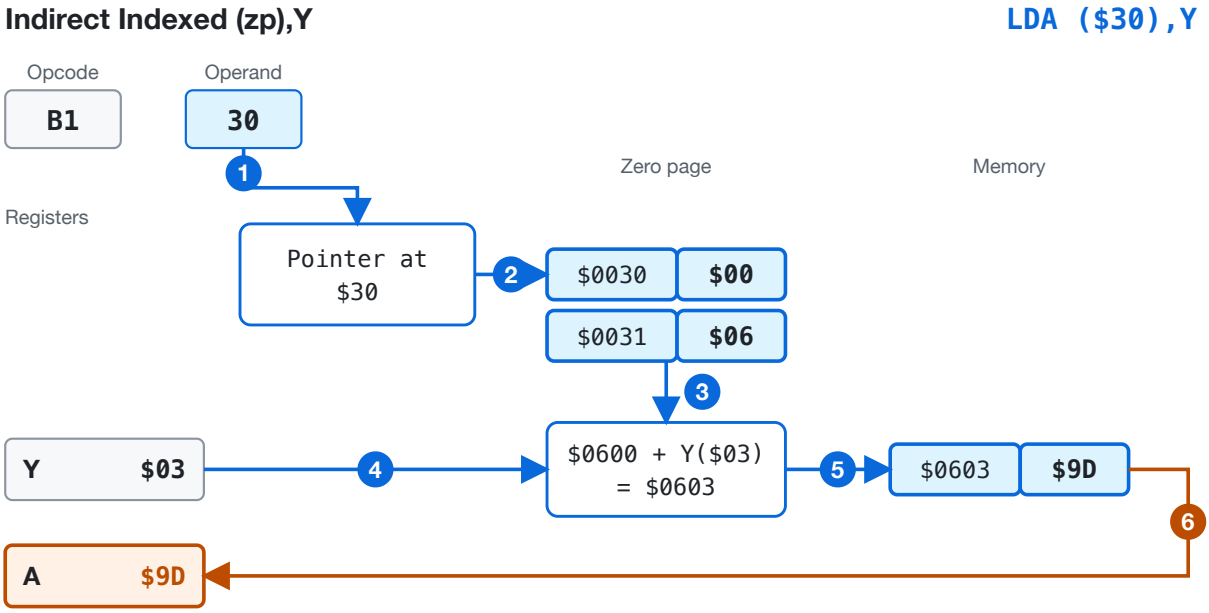
4.8 Indirect Indexed

The 2-byte address in the zero page is the base, and the value of Y is added to it. It is written `(zp),Y`. For example, put the start address of an array in memory into the zero page, and choose an element with Y.

Item	Content
Notation	`LDA ($30),Y`
Machine code	`B1 30`

(c) 2026 masanorisakai / MIT License

Item	Content
Bytes	2
Effective address	The 2 bytes (low byte first) at `$30` in the zero page are the base address, and Y is added to it



① Read opcode $B1 (LDA) and operand $30.
② Read $00 at $0030 as the low byte and $06 at $0031 as the high byte to get base address $0600.
③④ Add the value $03 of Y to the base address to get effective address $0603.
⑤⑥ Read the contents $9D of $0603 and put it in A.

Figure 4-12 Indirect indexed (`LDA ($30),Y`). Y (`$03`) is added to the base address `$0600`.

When adding Y to the base address lands on a different page, a real 6502 takes 1 extra cycle[1].

What you can try in Glow65

```
*E 0030 00 06
*E 0603 9D
*E 0300 A0 03 B1 30 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 9D 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

4.9 Relative

This is the mode used by branch instructions (`BNE`, `BEQ`, `BCC`, and others). The operand is a signed 1-byte value that gives the **distance from the address of the next instruction**. The range is -128 to +127. A

(c) 2026 masanorisakai / MIT License

negative number is written in two's complement, as in Chapter 2.

Item	Content
Notation	`BNE $0306` (the assembler calculates the distance)
Machine code	`D0 02`
Bytes	2
Jump target	The PC after the instruction has been read + the distance

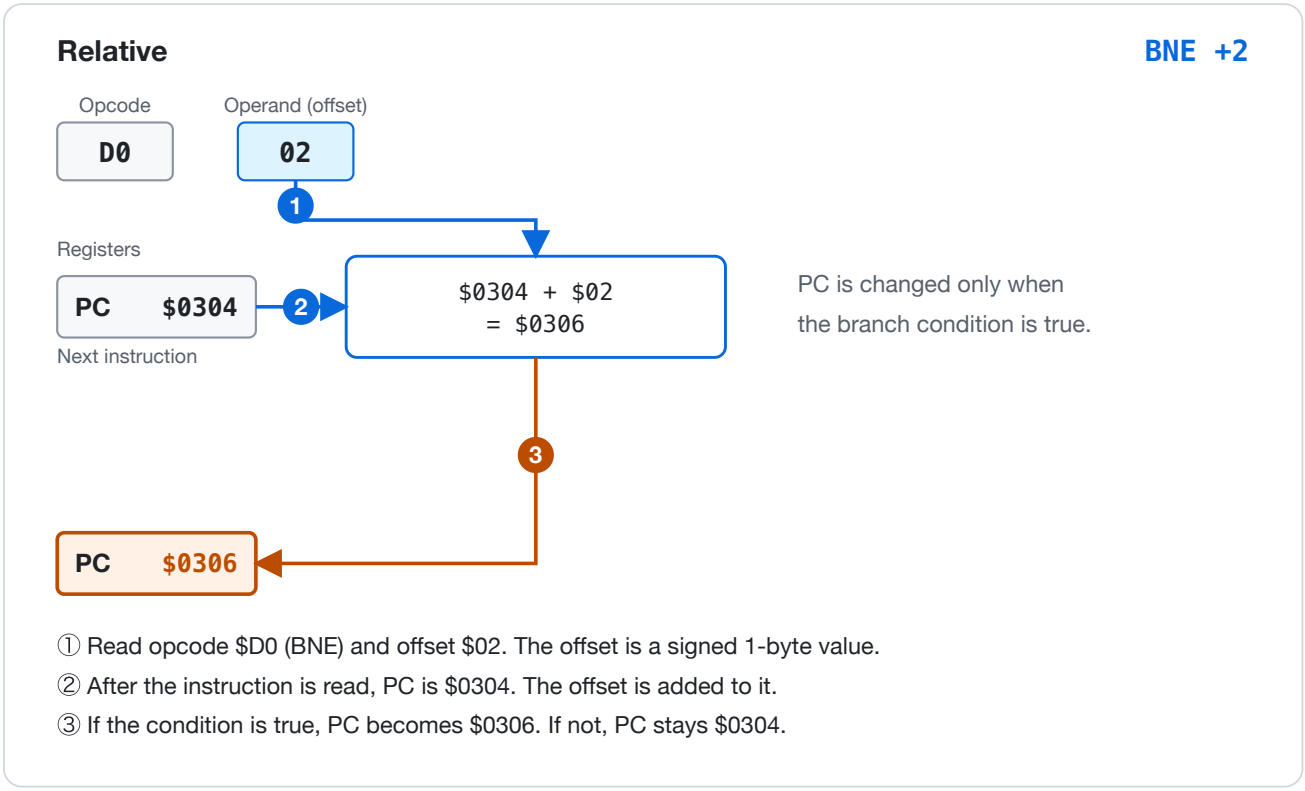


Figure 4-13 Relative (`BNE`). With a distance of `$02`, the jump target is `$0306`.

The distance is measured not from the start of the instruction but from the position after the instruction has been read. `D0 02` is a 2-byte instruction, so if it is at `$0302`, the PC after reading it is `$0304`. If the distance is `$FB` (-5), going back 5 from `$0304` gives `$02FF`.

If the condition does not hold, the CPU does not branch and goes on to the next instruction. The instruction takes 2 cycles when the condition does not hold, 3 when it holds, and 4 when it holds and the jump goes to another page.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

```
*E 0300 A9 01 D0 02 A9 EE 8D F0 03 60
*L 0300 0309
0300  A9 01     LDA   #$01
0302  D0 02     BNE   $0306
0304  A9 EE     LDA   #$EE
0306  8D F0 03  STA   $03F0
0309  60        RTS
*G 0300
*D 03F0 03F0
03F0: 01 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

After `LDA #$01`, the Z flag is 0. `BNE` branches, skips `LDA #$EE`, and goes to `$0306`. `01` is written to `$03F0`.

A branch can also go backward. In `BNE $0302`, the distance is `$FD` (-3).

```
*E 0300 A2 03 CA D0 FD 8E F0 03 60
*L 0300 0308
0300  A2 03     LDX   #$03
0302  CA        DEX
0303  D0 FD     BNE   $0302
0305  8E F0 03  STX   $03F0
0308  60        RTS
```

`DEX` decreases X by 1 each time, and the program goes back to `$0302` until X becomes 0.

4.10 Summary of All Modes

The cycle counts are those of a typical instruction that has the mode, such as `LDA`. A `+` means 1 more cycle when a read instruction crosses a page.

Mode	Notation	Bytes	Cycles	Example
Implied	(none)	1	2	`INX`
Accumulator	`A`	1	2	`ASL A`
Immediate	`#$nn`	2	2	`LDA #$10`
Zero page	`$nn`	2	3	`LDA $10`
Zero page,X	`$nn,X`	2	4	`LDA $10,X`
Zero page,Y	`$nn,Y`	2	4	`LDX $10,Y`
Absolute	`$nnnn`	3	4	`LDA $0400`

(c) 2026 masanorisakai / MIT License

Mode	Notation	Bytes	Cycles	Example
Absolute,X	`$nnnn,X`	3	4 +	`LDA $0410,X`
Absolute,Y	`$nnnn,Y`	3	4 +	`LDA $0420,Y`
Indirect	`($nnnn)`	3	5	`JMP ($0430)`
Indexed indirect	`($nn,X)`	2	6	`LDA ($20,X)`
Indirect indexed	`($nn),Y`	2	5 +	`LDA ($30),Y`
Relative	`$nnnn`	2	2 (3 if taken, 4 if taken to another page)	`BNE $0306`

For instructions other than `LDA`, the cycle count can differ. For example, `STA $0410,X` takes 5 cycles, whether or not it crosses a page. The cycle count of each instruction is collected in the instruction reference in Chapter 5.

4.11 Modes Added by the 65C02

The 65C02 adds the following modes and instructions. This includes cases where an instruction that already existed on the 6502, such as `BIT` or `INC`, gets a new mode.

Mode	Notation	Bytes	Example
Zero page indirect	`($nn)`	2	`LDA ($12)`
Absolute,X indirect	`($nnnn,X)`	3	`JMP ($1234,X)`
Zero page,X (new instruction)	`$nn,X`	2	`BIT $10,X`
Immediate (new instruction)	`#$nn`	2	`BIT #$10`
Accumulator (new instruction)	`A`	1	`INC A`
Relative (new instruction)	`$nnnn`	2	`BRA $0309`
Zero page relative	`$nn,$nnnn`	3	`BBR0 $10,$030C`

`BRA` is an instruction that always branches. `BBR0` branches when bit 0 of the zero page value is 0. In Glow65 you can switch to the 65C02 with the CPU switch on the panel (or the startup option `--cpu 65c02`). On the 65C02, the monitor's `L` displays these instructions.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

```
glow65-cli --cpu 65c02
*E 0300 B2 12 7C 34 12 34 10 89 10 80 FE 0F 10 FE 1A
*L 0300 030E
0300  B2 12     LDA   ($12)
0302  7C 34 12  JMP   ($1234,X)
0305  34 10     BIT   $10,X
0307  89 10     BIT   #$10
0309  80 FE     BRA   $0309
030B  0F 10 FE  BBR0  $10,$030C
030E  1A        INC   A
```

4.12 Practice Problems

The answers and explanations are in the next section.

Problem 1 Give the addressing mode of each instruction. (1) `STA $0300,X` (2) `LDX $20,Y` (3) `JMP ($0430)` (4) `LDA ($20,X)` (5) `LDA #$05`

Problem 2 Give the length (in bytes) of each instruction. (1) `LDA $0410,X` (2) `LDA ($20),Y` (3) `INX` (4) `BNE $0306` (5) `LDA #$05`

Problem 3 When X is `$05`, which address does `LDA $10,X` read?

Problem 4 When X is `$20`, which address does `LDA $F0,X` (zero page,X) read? Which address does `LDA $00F0,X` (absolute,X) read?

Problem 5 When Y is `$FF`, which address does `LDA $0420,Y` read? Does it cross a page?

Problem 6 Zero page `$0042` contains `00` and `$0043` contains `07`. When X is `$02`, which address does `LDA ($40,X)` read?

Problem 7 Zero page `$0050` contains `FF` and `$0051` contains `05`. When Y is `$01`, which address does `LDA ($50),Y` read? Does it cross a page?

Problem 8 Where does the `BNE` (`D0 F0`) at `$0310` jump to?

Problem 9 `$06FF` contains `50`, `$0700` contains `05`, and `$0600` contains `04`. Where does `JMP ($06FF)` jump to on the NMOS 6502 and on the 65C02?

4.13 Answers and Explanations

Problem 1 (1) Absolute,X (2) Zero page,Y (3) Indirect (4) Indexed indirect (5) Immediate

(c) 2026 masanorisakai / MIT License

- (1) A 2-byte address followed by `,X`. (2) A 1-byte address followed by `,Y`. This is a mode that `LDX` has. (3) A mode only `JMP` has, and the value in parentheses is an address. (4) `,X` inside the parentheses is `(zp,X)`. (5) `#` means the value itself.

Problem 2 (1) 3 bytes (2) 2 bytes (3) 1 byte (4) 2 bytes (5) 2 bytes

- (1) The opcode and a 2-byte address. (2) The opcode and 1 zero page byte. (3) No operand. (4) A branch has 1 byte for the distance. (5) The opcode and 1 byte for the value.

Problem 3 `$0015`

- `$10 + $05 = $15`. The high byte is `$00`.

Problem 4 Zero page,X reads `$0010`. Absolute,X reads `$0110`.

- Zero page,X wraps `$F0 + $20 = $110` at 256 and gets `$10`. Absolute,X does not wrap: `$00F0 + $20 = $0110`.

Problem 5 `$051F`. It crosses a page.

- `$0420 + $FF = $051F`. The high byte changes from `$04` to `$05`.

Problem 6 `$0700`

- `$40 + $02 = $42`. `00` at `$0042` is the low byte and `07` at `$0043` is the high byte, giving `$0700`.

Problem 7 `$0600`. It crosses a page.

- The base address is `FF` at `$0050` (low byte) and `05` at `$0051` (high byte), which is `$05FF`. Adding Y gives `$0600`. The high byte changes from `$05` to `$06`.

Problem 8 `$0302`

- `BNE` is a 2-byte instruction, so the PC after reading it is `$0312`. `$F0` is -16. `$0312 − 16 = $0302`.

Problem 9 `$0450` on the NMOS 6502, `$0550` on the 65C02.

- The NMOS reads the low byte from `$06FF` (`50`) and the high byte from the start of the same page, `$0600` (`04`), giving `$0450`. The 65C02 reads the high byte from `$0700` (`05`), giving `$0550`.

Sources

1. MOS Technology, "MCS6500 Microcomputer Family Hardware Manual", January 1976 edition: https://www.commodore.ca/wp-content/uploads/2018/11/kim1_hardware.txt

(c) 2026 masanorisakai / MIT License

Chapter 5 Instruction Reference

This chapter is a reference to the 151 official opcodes (56 instructions) of the 6502, grouped by function. For each instruction it gives the operation, the flags it affects, an opcode table, and usage examples. The instructions are split into one file per group.

The table of 151 opcodes in this chapter covers the NMOS 6502 only. The instructions and modes added by the 65C02 are in 5.7 Instructions Added by the 65C02.

How to Read This Chapter

Instruction Syntax

Syntax	Meaning
`#$nn`	The value itself (immediate). `nn` is 1 byte
`$nn`	A zero page address. `nn` is 1 byte
`$nnnn`	A 2-byte address. In machine code, the low byte comes first
`($nn,X)`, `($nn),Y`, `($nnnn)`	The three forms of indirect
`A`	The accumulator (the A register itself)
`label`	The branch target. The assembler converts it to a distance

Chapter 4 explains the addressing modes.

Symbols in the Flag Tables

The "Flags affected" table of each instruction shows the six flags N, V, D, I, Z, and C with the symbols below. B (break) and bit 5 have no storage in P, so they are not in the table.

Symbol	Meaning
○	Becomes 0 or 1 depending on the result
-	Does not change
0, 1	Always becomes 0, or always becomes 1
M7, M6	Takes bit 7 or bit 6 of the memory value as it is
Restored	Replaced by the value pulled from the stack

Reading the Opcode Tables

The columns of the "Opcodes" table are as follows.

Column	Meaning
Addressing mode	A mode the instruction has

(c) 2026 masanorisakai / MIT License

Column	Meaning
Syntax	How to write it in assembly language
Opcode	The first byte of the machine code (hexadecimal)
Bytes	The length of the whole instruction
Cycles	The number of cycles the instruction takes

The notation for cycles is as follows.

Notation	Meaning
`4`	Always 4 cycles
`4+`	A read instruction. If adding the index crosses into another page, it takes 1 more cycle (5 cycles)
`2 (+1, +2)`	A branch. 2 cycles when the condition is false, 3 when it is true, and 4 when it is true and the target is on another page

The extra cycle for a page crossing and the cycle counts of branches are in the table of the MOS manual [1].

Cycles and Bus Accesses in Glow65

When it calculates an address, the real 6502 performs extra reads (dummy reads) and writes (dummy writes) whose results are not used. Glow65 does not reproduce them. Glow65 counts the same number of cycles as the real chip, but the number of bus accesses shown on the panel and in the diagrams can be smaller than on the real chip. For example, `CLC` takes 2 cycles and makes 1 bus access.

Groups

Group	Instructions
5.1 Load and Store	LDA, LDX, LDY, STA, STX, STY
5.2 Register Transfer	TAX, TAY, TXA, TYA, TSX, TXS
5.3 Arithmetic and Comparison	ADC, SBC, CMP, CPX, CPY, INC, DEC, INX, INY, DEX, DEY
5.4 Logic, Shift, and Bit Test	AND, ORA, EOR, BIT, ASL, LSR, ROL, ROR
5.5 Branch and Jump	BCC, BCS, BEQ, BNE, BMI, BPL, BVC, BVS, JMP, JSR, RTS, BRK, RTI
5.6 Stack and Flag Operations	PHA, PLA, PHP, PLP, CLC, SEC, CLI, SEI, CLD, SED, CLV, NOP
5.7 Instructions Added by the 65C02	BRA, PHX, PHY, PLX, PLY, STZ, TRB, TSB, WAI, STP, BBR, BBS, RMB, SMB, and the new modes

List of All 56 Instructions

Mnemonic	Name	Function	Flags affected
LDA	Load Accumulator	Loads a value from memory into A	N Z

(c) 2026 masanorisakai / MIT License

Mnemonic	Name	Function	Flags affected
LDX	Load X Register	Loads a value from memory into X	N Z
LDY	Load Y Register	Loads a value from memory into Y	N Z
STA	Store Accumulator	Writes the value of A to memory	None
STX	Store X Register	Writes the value of X to memory	None
STY	Store Y Register	Writes the value of Y to memory	None
TAX	Transfer A to X	Copies the value of A to X	N Z
TAY	Transfer A to Y	Copies the value of A to Y	N Z
TXA	Transfer X to A	Copies the value of X to A	N Z
TYA	Transfer Y to A	Copies the value of Y to A	N Z
TSX	Transfer Stack Pointer to X	Copies the value of SP to X	N Z
TXS	Transfer X to Stack Pointer	Copies the value of X to SP	None
ADC	Add with Carry	Adds the memory value and the carry to A	N V Z C
SBC	Subtract with Carry	Subtracts the memory value and the borrow from A	N V Z C
CMP	Compare Accumulator	Compares A with a memory value	N Z C
CPX	Compare X Register	Compares X with a memory value	N Z C
CPY	Compare Y Register	Compares Y with a memory value	N Z C
INC	Increment Memory	Adds 1 to a memory value	N Z
DEC	Decrement Memory	Subtracts 1 from a memory value	N Z
INX	Increment X Register	Adds 1 to X	N Z
INY	Increment Y Register	Adds 1 to Y	N Z
DEX	Decrement X Register	Subtracts 1 from X	N Z
DEY	Decrement Y Register	Subtracts 1 from Y	N Z
AND	Logical AND	Bitwise AND of A and a memory value	N Z
ORA	Logical OR	Bitwise OR of A and a memory value	N Z
EOR	Exclusive OR	Bitwise XOR of A and a memory value	N Z
BIT	Bit Test	ANDs A with a memory value to set Z, and copies the top 2 bits of the memory value to N and V	N V Z
ASL	Arithmetic Shift Left	Shifts left by 1 bit; the most significant bit goes out to C	N Z C
LSR	Logical Shift Right	Shifts right by 1 bit; the least significant bit goes out to C	N Z C
ROL	Rotate Left	Rotates left by 1 bit through C	N Z C
ROR	Rotate Right	Rotates right by 1 bit through C	N Z C

(c) 2026 masanorisakai / MIT License

Mnemonic	Name	Function	Flags affected
BCC	Branch if Carry Clear	Branches if C is 0	None
BCS	Branch if Carry Set	Branches if C is 1	None
BEQ	Branch if Equal	Branches if Z is 1	None
BNE	Branch if Not Equal	Branches if Z is 0	None
BMI	Branch if Minus	Branches if N is 1	None
BPL	Branch if Plus	Branches if N is 0	None
BVC	Branch if Overflow Clear	Branches if V is 0	None
BVS	Branch if Overflow Set	Branches if V is 1	None
JMP	Jump	Jumps to the given address	None
JSR	Jump to Subroutine	Pushes the return address and jumps to the given address	None
RTS	Return from Subroutine	Pulls the return address and returns	None
BRK	Break	Causes a software interrupt	I
RTI	Return from Interrupt	Pulls P and the return address and returns	N V D I Z C
PHA	Push Accumulator	Pushes A onto the stack	None
PLA	Pull Accumulator	Pulls A from the stack	N Z
PHP	Push Processor Status	Pushes P onto the stack	None
PLP	Pull Processor Status	Pulls P from the stack	N V D I Z C
CLC	Clear Carry Flag	Sets C to 0	C
SEC	Set Carry Flag	Sets C to 1	C
CLI	Clear Interrupt Disable	Enables interrupts	I
SEI	Set Interrupt Disable	Disables interrupts	I
CLD	Clear Decimal Mode	Leaves decimal mode	D
SED	Set Decimal Mode	Enters decimal mode	D
CLV	Clear Overflow Flag	Sets V to 0	V
NOP	No Operation	Does nothing	None

Undefined Opcodes

The 105 opcodes other than the 151 official ones are not official instructions. Glow65 executes them as a 1-byte, 2-cycle NOP that has no operand. On a real NMOS 6502, each undefined opcode behaves differently,

(c) 2026 masanorisakai / MIT License

but Glow65 does not reproduce this.

On the 65C02, the opcodes that were undefined are all NOPs. The instructions added by the 65C02 (`BRA`, `STZ`, `PHX`, and others) are in 5.7 Instructions Added by the 65C02.

> **What you can try in Glow65**
>
> Place the undefined opcode `$02` and run it. `L` shows it as `.BYTE`.
>
> ```
> *E 0300 02 A2 05 8E F0 03 60
> *L 0300 0301
> 0300  02        .BYTE $02
> 0301  A2 05     LDX   #$05
> *G 0300
> *D 03F0 03F0
> 03F0: 05 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> `$02` runs as a 1-byte NOP. The `LDX #$05` after it runs, and `05` is written to `$03F0`.

Sources

1. MOS Technology, "MCS6500 Microcomputer Family Hardware Manual", January 1976 edition: https://www.commodore.ca/wp-content/uploads/2018/11/kim1_hardware.txt

(c) 2026 masanorisakai / MIT License

5.1 Load and Store

Load instructions put a value from memory into a register. Store instructions write the value of a register to memory. Load instructions change N and Z. Store instructions do not change any flag.

Every instruction in this group handles one 8-bit value. A 16-bit value is handled in two steps.

LDA (Load Accumulator)

Loads a value from memory into A. N and Z change according to the value.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Immediate	`LDA #$nn`	`$A9`	2	2
Zero page	`LDA $nn`	`$A5`	2	3
Zero page,X	`LDA $nn,X`	`$B5`	2	4
Absolute	`LDA $nnnn`	`$AD`	3	4
Absolute,X	`LDA $nnnn,X`	`$BD`	3	4+
Absolute,Y	`LDA $nnnn,Y`	`$B9`	3	4+
Indexed indirect `(zp,X)`	`LDA ($nn,X)`	`$A1`	2	6
Indirect indexed `(zp),Y`	`LDA ($nn),Y`	`$B1`	2	5+

Examples

```
        LDA #$80
```

A becomes `$80`. The most significant bit is 1, so N becomes 1.

```
        LDA #$00
```

A becomes `$00`. The value is 0, so Z becomes 1.

LDX (Load X Register)

Loads a value from memory into X. N and Z change according to the value. The indexed modes are zero page,Y and absolute,Y.

(c) 2026 masanorisakai / MIT License

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Immediate	`LDX #$nn`	`$A2`	2	2
Zero page	`LDX $nn`	`$A6`	2	3
Zero page,Y	`LDX $nn,Y`	`$B6`	2	4
Absolute	`LDX $nnnn`	`$AE`	3	4
Absolute,Y	`LDX $nnnn,Y`	`$BE`	3	4+

Examples

```
LDX #$05
```

X becomes `$05`.

Setup: memory `$0013` holds `$5A`.

```
LDY #$03
LDX $10,Y
```

Zero page,Y. Y (`$03`) is added to `$10`, giving `$0013`. The contents of `$0013`, `$5A`, go into X.

LDY (Load Y Register)

Loads a value from memory into Y. N and Z change according to the value.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Immediate	`LDY #$nn`	`$A0`	2	2
Zero page	`LDY $nn`	`$A4`	2	3
Zero page,X	`LDY $nn,X`	`$B4`	2	4
Absolute	`LDY $nnnn`	`$AC`	3	4

(c) 2026 masanorisakai / MIT License

Addressing mode	Syntax	Opcode	Bytes	Cycles
Absolute,X	`LDY $nnnn,X`	`$BC`	3	4+

Examples

```
        LDY #$FF
```

Y becomes `$FF`. The most significant bit is 1, so N becomes 1.

STA (Store Accumulator)

Writes the value of A to memory. Neither A nor the flags change.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page	`STA $nn`	`$85`	2	3
Zero page,X	`STA $nn,X`	`$95`	2	4
Absolute	`STA $nnnn`	`$8D`	3	4
Absolute,X	`STA $nnnn,X`	`$9D`	3	5
Absolute,Y	`STA $nnnn,Y`	`$99`	3	5
Indexed indirect `(zp,X)`	`STA ($nn,X)`	`$81`	2	6
Indirect indexed `(zp),Y`	`STA ($nn),Y`	`$91`	2	6

Examples

Setup: memory `$0400` holds `$00`.

```
        LDA #$55
        STA $0400
```

A (`$55`) is written to `$0400`.

Setup: memory `$0403` holds `$00`.

```
        LDA #$55
        LDX #$03
        STA $0400,X
```

(c) 2026 masanorisakai / MIT License

X (`$03`) is added to `$0400`, giving `$0403`, and `$55` is written there.

For STA, the absolute,X, absolute,Y, and indirect indexed modes take the same number of cycles whether or not the access crosses a page.

STX (Store X Register)

Writes the value of X to memory. It has a zero page,Y mode.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page	`STX $nn`	`$86`	2	3
Zero page,Y	`STX $nn,Y`	`$96`	2	4
Absolute	`STX $nnnn`	`$8E`	3	4

Examples

Setup: memory `$0010` holds `$00`.

```
LDX #$07
STX $10
```

X (`$07`) is written to `$0010`.

STY (Store Y Register)

Writes the value of Y to memory. It has a zero page,X mode.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page	`STY $nn`	`$84`	2	3
Zero page,X	`STY $nn,X`	`$94`	2	4
Absolute	`STY $nnnn`	`$8C`	3	4

(c) 2026 masanorisakai / MIT License

Examples

Setup: memory `$0400` holds `$00`.

```
        LDY #$09
        STY $0400
```

Y (`$09`) is written to `$0400`.

> **What you can try in Glow65**
>
> After `LDA #$80`, `PHP` and `PLA` bring P into A, and `STA` writes it to `$03F0`.
>
> ```
> *E 0300 A9 80 08 68 8D F0 03 60
> *L 0300 0304
> 0300  A9 80     LDA   #$80
> 0302  08        PHP
> 0303  68        PLA
> 0304  8D F0 03  STA   $03F0
> *G 0300
> *D 03F0 03F0
> 03F0: B4 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> `$B4` is `%10110100`. N (bit 7) is 1. Bit 5 and B (bit 4) are set to 1 by PHP when it pushes. I (bit 2) is also 1.

> **What you can try in Glow65**
>
> Three store instructions write the values of A, X, and Y to `$03F0` and the next two addresses.
>
> ```
> *E 0300 A9 55 A2 07 A0 09 8D F0 03 8E F1 03 8C F2 03 60
> *G 0300
> *D 03F0 03F2
> 03F0: 55 07 09 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```

(c) 2026 masanorisakai / MIT License

5.2 Register Transfer

These instructions copy the value of one register to another register. The source value does not change. Except for TXS, N and Z are set from the copied value.

Only A and X, A and Y, and X and SP can be copied directly. There is no transfer instruction between X and Y.

TAX (Transfer A to X)

Copies the value of A to X. N and Z are set from the copied value.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`TAX`	`$AA`	1	2

Examples

```
LDA #$80
TAX
```

A (`$80`) is copied to X. A stays the same, and N becomes 1.

TAY (Transfer A to Y)

Copies the value of A to Y. N and Z are set from the copied value.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`TAY`	`$A8`	1	2

Examples

(c) 2026 masanorisakai / MIT License

```
LDY #$55
LDA #$00
TAY
```

Y is `$55`. Copying A (`$00`) to Y makes Y `$00`, and Z becomes 1.

TXA (Transfer X to A)

Copies the value of X to A. N and Z are set from the copied value.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`TXA`	`$8A`	1	2

Examples

```
LDX #$33
LDA #$00
TXA
```

A is `$00`. X (`$33`) is copied to A.

TYA (Transfer Y to A)

Copies the value of Y to A. N and Z are set from the copied value.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`TYA`	`$98`	1	2

Examples

(c) 2026 masanorisakai / MIT License

```
LDY #$44
TYA
```

Y (`$44`) is copied to A.

TSX (Transfer Stack Pointer to X)

Copies the value of SP to X. N and Z are set from the copied value.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`TSX`	`$BA`	1	2

Examples

```
LDX #$F0
TXS
LDX #$00
TSX
```

When SP is `$F0`, `TSX` copies it to X, so X becomes `$F0` and N becomes 1.

Changing SP moves the stack. This example saves SP before it runs and restores it before it ends.

TXS (Transfer X to Stack Pointer)

Copies the value of X to SP. TXS does not change any flag.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`TXS`	`$9A`	1	2

Examples

(c) 2026 masanorisakai / MIT License

```
LDX #$F0
LDA #$00
TXS
```

Even though X is `$F0`, Z=1 and N=0 from the preceding `LDA #$00` stay as they are.

TXS does not change the flags. This is different from the other transfer instructions.

What you can try in Glow65

`LDA #$80` and `TAX`, then `LDA #$00` and `TXA`, copy values between A and X. `L` shows the instructions.

```
*E 0300 A9 80 AA A9 00 8A 8D F0 03 8E F1 03 60
*L 0300 0305
0300  A9 80     LDA   #$80
0302  AA        TAX
0303  A9 00     LDA   #$00
0305  8A        TXA
*G 0300
*D 03F0 03F1
03F0: 80 80 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

At the last `TXA`, A changes from `$00` to the value of X, `$80`. X stays `$80`.

What you can try in Glow65

Set SP to `$F0` and read it with `TSX`. The program writes the original SP to `$03E4`, the X read by `TSX` to `$03E5`, and P to `$03E6`.

```
*E 0300 BA 8E E4 03 A2 F0 9A A2 00 BA 08 68 8D E6 03 8E
*E 0310 E5 03 AE E4 03 9A 60
*G 0300
*D 03E4 03E6
03E4: FD F0 B4 00 00 00 00 00 00 00 00 00 00 00 00 00
```

`FD` at `$03E4` is the SP before the change. `F0` at `$03E5` is the result of `TSX`, and `B4` at `$03E6` shows that N is 1.

(c) 2026 masanorisakai / MIT License

5.3 Arithmetic and Comparison

This group covers addition, subtraction, comparison, and adding or subtracting 1. ADC and SBC use C (carry) in the calculation. Set C before them: CLC before ADC, and SEC before SBC.

CMP, CPX, and CPY discard the result of a subtraction and change only the flags. Read the result from C (greater than or equal), Z (equal), and N.

ADC (Add with Carry)

Adds the memory value and C to A, and puts the result in A. If the result exceeds `$FF`, C becomes 1. N, V, and Z also change.

Flags affected

N	V	D	I	Z	C
○	○	-	-	○	○

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Immediate	`ADC #$nn`	`$69`	2	2
Zero page	`ADC $nn`	`$65`	2	3
Zero page,X	`ADC $nn,X`	`$75`	2	4
Absolute	`ADC $nnnn`	`$6D`	3	4
Absolute,X	`ADC $nnnn,X`	`$7D`	3	4+
Absolute,Y	`ADC $nnnn,Y`	`$79`	3	4+
Indexed indirect `(zp,X)`	`ADC ($nn,X)`	`$61`	2	6
Indirect indexed `(zp),Y`	`ADC ($nn),Y`	`$71`	2	5+

Examples

```
        CLC
        LDA #$50
        ADC #$50
```

`$50 + $50 = $A0`. As signed numbers, 80 + 80 is outside the range (-128 to 127), so V becomes 1. N is also 1.

```
        CLC
        LDA #$FF
        ADC #$01
```

(c) 2026 masanorisakai / MIT License

`$FF + $01` does not fit in 8 bits. A becomes `$00`, C becomes 1, and Z becomes 1.

```
        SEC
        LDA #$01
        ADC #$01
```

With C at 1, `1 + 1 + 1` makes A `$03`.

Setup: memory from `$0400` holds `FF 12 01 00`.

```
        CLC
        LDA $0400
        ADC $0402
        STA $0404
        LDA $0401
        ADC $0403
        STA $0405
```

This is a 16-bit addition. It adds `$12FF + $0001`, starting from the low byte. The carry out of the low byte stays in C, and the ADC on the high byte picks it up. The result `$1300` is written to `$0404` (low byte) and `$0405` (high byte).

V flag: When the operands are treated as signed numbers, V becomes 1 if the result is outside -128 to 127. In other words, V becomes 1 when two numbers with the same sign are added and the sign of the result differs.

Decimal mode: When D is 1, ADC adds in BCD (a format that treats each 4 bits as a digit from 0 to 9). `$09 + $01` gives `$10`, not `$0A`. C shows the decimal carry. On the NMOS 6502, N, V, and Z in decimal mode may not match the value of the result. When you finish using decimal mode, switch back with CLD.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

The panel shows the V flag changing. Enter `E 0300 B8 A9 50 18 69 50 4C 00 03`, run `G 0300`, then set **RUN/STOP** to STOP and **STEP MODE** to INSTR. The program repeats `CLV`, `LDA #$50`, `CLC`, `ADC #$50`, and `JMP $0300`.

Right after `CLC`, A is `$50` (A6 and A4 are lit), and N and V are off (Figure 5-7). After `ADC #$50`, A is `$A0` (A7 and A5), and N and V are lit (Figure 5-8). As signed numbers, the sum of two positive numbers (80 + 80) became a negative number (-96), so V is 1. C stays 0.

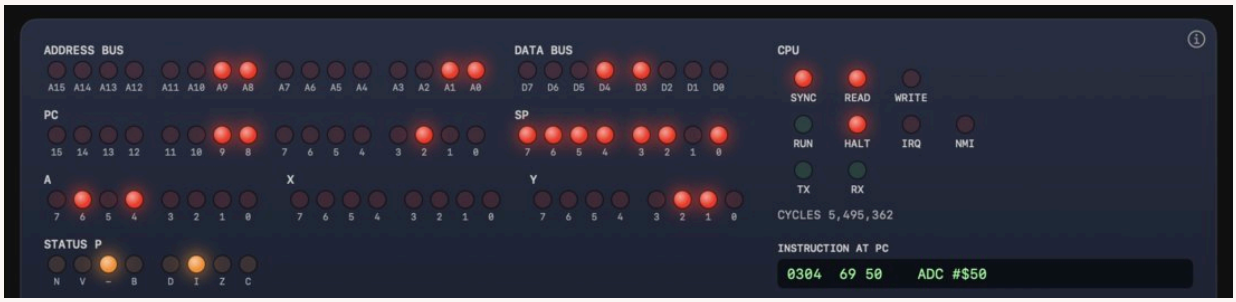


Figure 5-7 After `CLC`. A is `$50` and V is off



Figure 5-8 After `ADC #$50`. A is `$A0`, and N and V are lit

SBC (Subtract with Carry)

Subtracts the memory value and the borrow from A, and puts the result in A. The borrow is the inverse of C: if C is 1 there is no borrow, and if C is 0 there is a borrow. Set C to 1 with SEC before a subtraction.

Flags affected

N	V	D	I	Z	C
○	○	-	-	○	○

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Immediate	`SBC #$nn`	`$E9`	2	2
Zero page	`SBC $nn`	`$E5`	2	3
Zero page,X	`SBC $nn,X`	`$F5`	2	4

(c) 2026 masanorisakai / MIT License

Addressing mode	Syntax	Opcode	Bytes	Cycles
Absolute	`SBC $nnnn`	`$ED`	3	4
Absolute,X	`SBC $nnnn,X`	`$FD`	3	4+
Absolute,Y	`SBC $nnnn,Y`	`$F9`	3	4+
Indexed indirect `(zp,X)`	`SBC ($nn,X)`	`$E1`	2	6
Indirect indexed `(zp),Y`	`SBC ($nn),Y`	`$F1`	2	5+

Examples

```
        SEC
        LDA #$50
        SBC #$30
```

After `SEC`, `$50 − $30 = $20`. The result is not negative, so C stays 1.

```
        SEC
        LDA #$30
        SBC #$50
```

`$30 − $50` is negative. A becomes `$E0` (-32), C becomes 0 (a borrow occurred), and N becomes 1.

```
        SEC
        LDA #$50
        SBC #$B0
```

`$50 − $B0` is 80 - (-80) = 160 as signed numbers, which is out of range. V becomes 1.

Decimal mode: When D is 1, SBC also subtracts in BCD. After `SED`, `SEC`, and `LDA #$10`, running `SBC #$01` makes A `$09`. C stays 1 if there is no borrow.

CMP (Compare Accumulator)

Compares A with a memory value. It calculates A minus the value, discards the result, and changes only the flags. A does not change. C becomes 1 if A is greater than or equal to the value, Z becomes 1 if they are equal, and N takes the most significant bit of the result.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	○

Opcodes

(c) 2026 masanorisakai / MIT License

Addressing mode	Syntax	Opcode	Bytes	Cycles
Immediate	`CMP #$nn`	`$C9`	2	2
Zero page	`CMP $nn`	`$C5`	2	3
Zero page,X	`CMP $nn,X`	`$D5`	2	4
Absolute	`CMP $nnnn`	`$CD`	3	4
Absolute,X	`CMP $nnnn,X`	`$DD`	3	4+
Absolute,Y	`CMP $nnnn,Y`	`$D9`	3	4+
Indexed indirect `(zp,X)`	`CMP ($nn,X)`	`$C1`	2	6
Indirect indexed `(zp),Y`	`CMP ($nn),Y`	`$D1`	2	5+

Examples

```
        LDA #$30
        CMP #$30
```

When A equals the value, Z becomes 1 and C becomes 1.

```
        LDA #$30
        CMP #$40
```

When A is less than the value, C becomes 0. The result is negative, so N is 1.

```
        LDA #$30
        CMP #$20
```

When A is greater than the value, C becomes 1 and Z becomes 0.

After a comparison, branch with BEQ (equal), BNE (not equal), BCS (greater than or equal), or BCC (less than). The comparison treats the numbers as unsigned.

CPX (Compare X Register)

Compares X with a memory value. C, Z, and N change by the same rules as CMP. X does not change.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	○

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Immediate	`CPX #$nn`	`$E0`	2	2

(c) 2026 masanorisakai / MIT License

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page	`CPX $nn`	`$E4`	2	3
Absolute	`CPX $nnnn`	`$EC`	3	4

Examples

```
LDX #$10
CPX #$20
```

X is `$10` and the value is `$20`, so C becomes 0 and N becomes 1.

CPY (Compare Y Register)

Compares Y with a memory value. C, Z, and N change by the same rules as CMP. Y does not change.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	○

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Immediate	`CPY #$nn`	`$C0`	2	2
Zero page	`CPY $nn`	`$C4`	2	3
Absolute	`CPY $nnnn`	`$CC`	3	4

Examples

```
LDY #$20
CPY #$20
```

Y equals the value, so Z becomes 1 and C becomes 1.

INC (Increment Memory)

Adds 1 to a memory value. After `$FF` it wraps around to `$00`. N and Z change. C does not change.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

(c) 2026 masanorisakai / MIT License

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page	`INC $nn`	`$E6`	2	5
Zero page,X	`INC $nn,X`	`$F6`	2	6
Absolute	`INC $nnnn`	`$EE`	3	6
Absolute,X	`INC $nnnn,X`	`$FE`	3	7

Examples

Setup: memory `$0400` holds `$FF`.

```
        INC $0400
```

`$FF` at `$0400` becomes `$00`, and Z becomes 1.

DEC (Decrement Memory)

Subtracts 1 from a memory value. After `$00` it wraps around to `$FF`. N and Z change. C does not change.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page	`DEC $nn`	`$C6`	2	5
Zero page,X	`DEC $nn,X`	`$D6`	2	6
Absolute	`DEC $nnnn`	`$CE`	3	6
Absolute,X	`DEC $nnnn,X`	`$DE`	3	7

Examples

Setup: memory `$0400` holds `$00`.

```
        DEC $0400
```

`$00` at `$0400` becomes `$FF`, and N becomes 1.

INX (Increment X Register)

Adds 1 to X. N and Z change.

Flags affected

(c) 2026 masanorisakai / MIT License

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`INX`	`$E8`	1	2

Examples

```
LDX #$FF
INX
```

X (`$FF`) becomes `$00`, and Z becomes 1.

INY (Increment Y Register)

Adds 1 to Y. N and Z change.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`INY`	`$C8`	1	2

Examples

```
LDY #$7F
INY
```

Y (`$7F`) becomes `$80`, and N becomes 1.

DEX (Decrement X Register)

Subtracts 1 from X. N and Z change.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

(c) 2026 masanorisakai / MIT License

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`DEX`	`$CA`	1	2

Examples

```
        LDX #$01
        DEX
```

X (`$01`) becomes `$00`, and Z becomes 1.

DEY (Decrement Y Register)

Subtracts 1 from Y. N and Z change.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`DEY`	`$88`	1	2

Examples

```
        LDY #$00
        DEY
```

Y (`$00`) becomes `$FF`, and N becomes 1.

INC and DEC work on memory. The NMOS 6502 has no instruction that adds 1 to A. Use `CLC` followed by `ADC #$01`.

> **What you can try in Glow65**
>
> The program runs `CLC`, `LDA #$50`, and `ADC #$50`, then writes the resulting A to `$03F0` and P to `$03F1`.
>
> ```
> *E 0300 18 A9 50 69 50 8D F0 03 08 68 8D F1 03 60
> *G 0300
> *D 03F0 03F1
> 03F0: A0 F4 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> A is `$A0`. P (`$F4`) is `%11110100`: N (bit 7) and V (bit 6) are 1, and C (bit 0) is 0.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

This is a 16-bit addition. It adds `$12FF` (at `$0400` and `$0401`) and `$0001` (at `$0402` and `$0403`), and writes the result to `$0404` and `$0405`.

```
*E 0400 FF 12 01 00 00 00
*E 0300 18 AD 00 04 6D 02 04 8D 04 04 AD 01 04 6D 03 04
*E 0310 8D 05 04 60
*G 0300
*D 0400 0405
0400: FF 12 01 00 00 13 00 00 00 00 00 00 00 00 00 00
```

`$0404` holds `00` and `$0405` holds `13`, so the answer is `$1300`.

What you can try in Glow65

Add `$09 + $01` in decimal mode.

```
*E 0300 F8 18 A9 09 69 01 D8 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 10 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

The result is `$10`. In plain binary it would be `$0A`.

(c) 2026 masanorisakai / MIT License

5.4 Logic, Shift, and Bit Test

This group covers the bitwise logic operations (AND, OR, XOR), shifts that move bits sideways, and rotates that move bits around through the carry.

As Figure 5-1 shows, shifts and rotates use the carry (C) as their entrance and exit. They work on both the accumulator and memory.

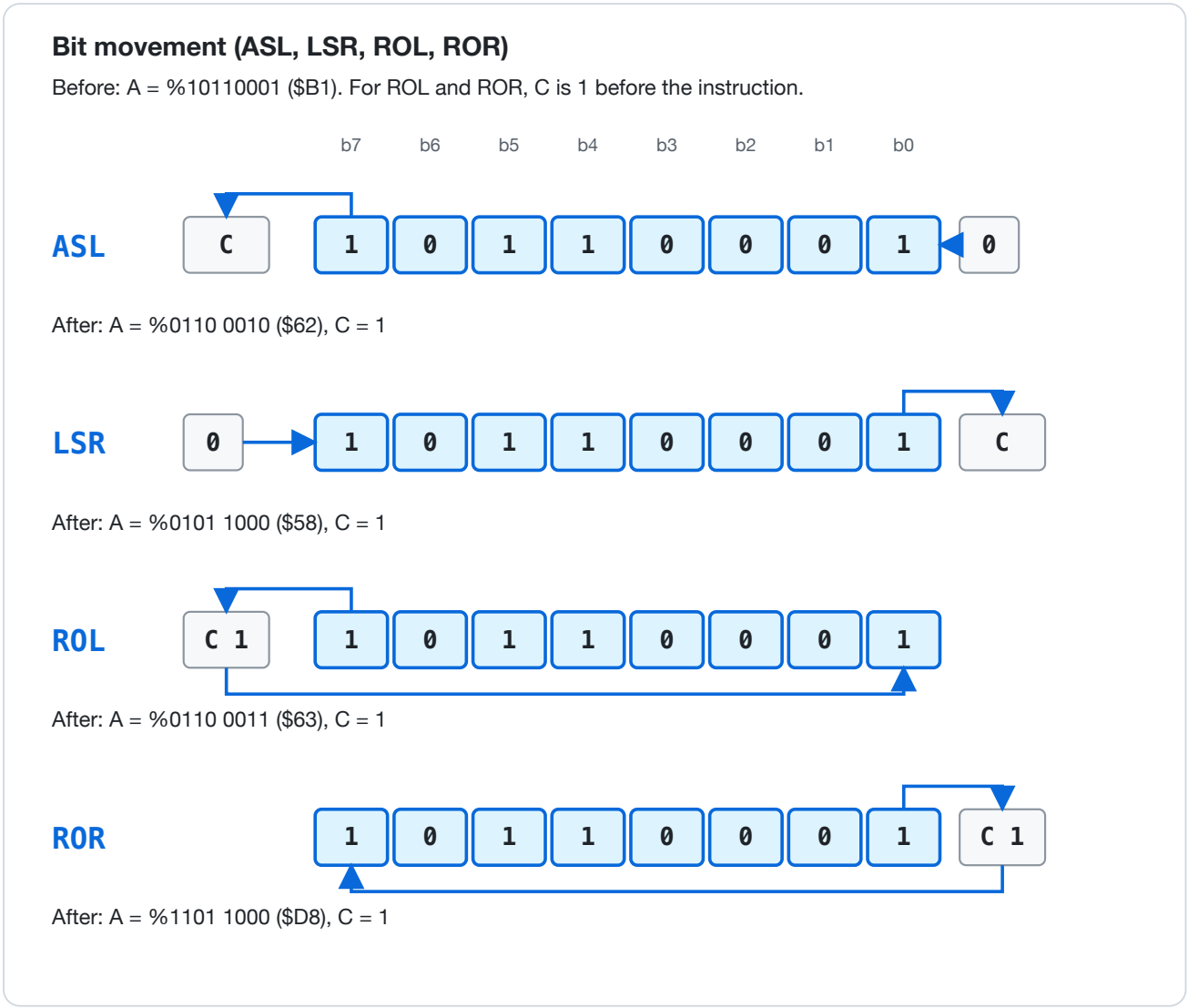


Figure 5-1 How the bits move in ASL, LSR, ROL, and ROR.

(c) 2026 masanorisakai / MIT License

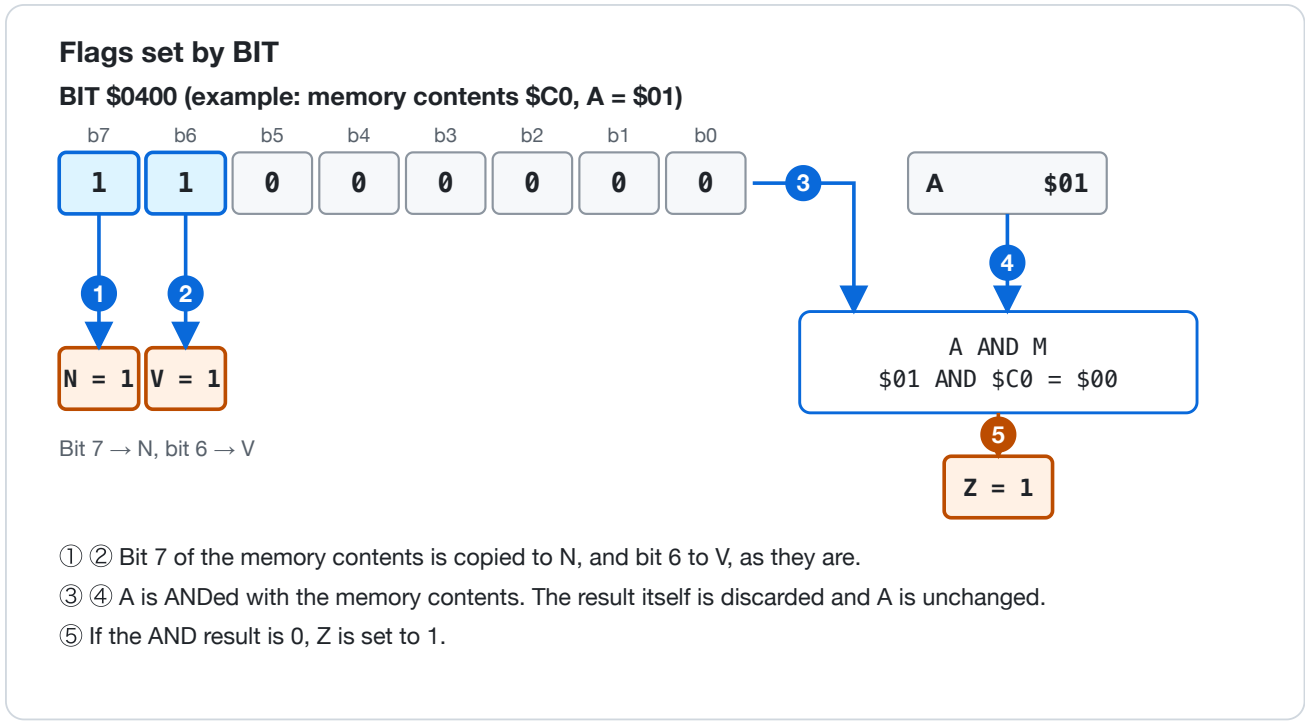


Figure 5-2 How BIT sets N, V, and Z.

AND (Logical AND)

ANDs A with a memory value, bit by bit. A bit is 1 only when both bits are 1. Use it to pick out specific bits (all other bits become 0). N and Z change.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Immediate	`AND #$nn`	`$29`	2	2
Zero page	`AND $nn`	`$25`	2	3
Zero page,X	`AND $nn,X`	`$35`	2	4
Absolute	`AND $nnnn`	`$2D`	3	4
Absolute,X	`AND $nnnn,X`	`$3D`	3	4+
Absolute,Y	`AND $nnnn,Y`	`$39`	3	4+
Indexed indirect `(zp,X)`	`AND ($nn,X)`	`$21`	2	6
Indirect indexed `(zp),Y`	`AND ($nn),Y`	`$31`	2	5+

Examples

(c) 2026 masanorisakai / MIT License

```
        LDA #$B5
        AND #$0F
```

`$B5 AND $0F` is `$05`. Only the low 4 bits remain.

```
        LDA #$F0
        AND #$3C
```

`$F0 AND $3C` is `$30`.

ORA (Logical OR)

ORs A with a memory value, bit by bit. A bit is 1 if either bit is 1. Use it to set specific bits to 1. N and Z change.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Immediate	`ORA #$nn`	`$09`	2	2
Zero page	`ORA $nn`	`$05`	2	3
Zero page,X	`ORA $nn,X`	`$15`	2	4
Absolute	`ORA $nnnn`	`$0D`	3	4
Absolute,X	`ORA $nnnn,X`	`$1D`	3	4+
Absolute,Y	`ORA $nnnn,Y`	`$19`	3	4+
Indexed indirect `(zp,X)`	`ORA ($nn,X)`	`$01`	2	6
Indirect indexed `(zp),Y`	`ORA ($nn),Y`	`$11`	2	5+

Examples

```
        LDA #$30
        ORA #$05
```

`$30 ORA $05` is `$35`.

```
        LDA #$00
        ORA #$80
```

(c) 2026 masanorisakai / MIT License

ORA with `$80` sets the most significant bit to 1. N is 1.

EOR (Exclusive OR)

XORs (exclusive OR) A with a memory value, bit by bit. A bit is 1 only when the two bits differ. Use it to flip specific bits. N and Z change.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Immediate	`EOR #$nn`	`$49`	2	2
Zero page	`EOR $nn`	`$45`	2	3
Zero page,X	`EOR $nn,X`	`$55`	2	4
Absolute	`EOR $nnnn`	`$4D`	3	4
Absolute,X	`EOR $nnnn,X`	`$5D`	3	4+
Absolute,Y	`EOR $nnnn,Y`	`$59`	3	4+
Indexed indirect `(zp,X)`	`EOR ($nn,X)`	`$41`	2	6
Indirect indexed `(zp),Y`	`EOR ($nn),Y`	`$51`	2	5+

Examples

```
LDA #$FF
EOR #$0F
```

`$FF EOR $0F` is `$F0`. The bits that are 1 in `$0F` are flipped.

```
LDA #$5A
EOR #$5A
```

The XOR of equal values is `$00`. Z becomes 1.

BIT (Bit Test)

ANDs A with a memory value and sets Z to 1 if the result is 0. The result itself is discarded, and A does not change. Bit 7 of the memory value is copied to N, and bit 6 is copied to V.

Flags affected

(c) 2026 masanorisakai / MIT License

N	V	D	I	Z	C
M7	M6	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page	`BIT $nn`	`$24`	2	3
Absolute	`BIT $nnnn`	`$2C`	3	4

Examples

Setup: memory `$0400` holds `$C0`.

```
LDA #$01
BIT $0400
```

The memory value `$C0` has bits 7 and 6 set to 1, so N and V become 1. `$01 AND $C0` is 0, so Z is also 1.

Setup: memory `$0400` holds `$3F`.

```
LDA #$01
BIT $0400
```

The memory value `$3F` has bits 7 and 6 at 0, so N and V become 0. `$01 AND $3F` is 1, so Z is 0.

Use BIT to examine bits without destroying A. N and V are decided by the memory value, not by A.

ASL (Arithmetic Shift Left)

Shifts left by 1 bit. The most significant bit goes out to C, and 0 enters the least significant bit. The value doubles (anything above 255 goes out to C). N, Z, and C change.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	○

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Accumulator	`ASL A`	`$0A`	1	2
Zero page	`ASL $nn`	`$06`	2	5
Zero page,X	`ASL $nn,X`	`$16`	2	6
Absolute	`ASL $nnnn`	`$0E`	3	6

(c) 2026 masanorisakai / MIT License

Addressing mode	Syntax	Opcode	Bytes	Cycles
Absolute,X	`ASL $nnnn,X`	`$1E`	3	7

Examples

```
        LDA #$81
        ASL A
```

`%10000001` becomes `%00000010`. The 1 that was the most significant bit goes out to C.

Setup: memory `$0400` holds `$40`.

```
        ASL $0400
```

`$40` at `$0400` becomes `$80`. N is 1 and C is 0.

LSR (Logical Shift Right)

Shifts right by 1 bit. The least significant bit goes out to C, and 0 enters the most significant bit. The value is halved (the remainder goes out to C). N is always 0. Z and C also change.

Flags affected

N	V	D	I	Z	C
0	-	-	-	○	○

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Accumulator	`LSR A`	`$4A`	1	2
Zero page	`LSR $nn`	`$46`	2	5
Zero page,X	`LSR $nn,X`	`$56`	2	6
Absolute	`LSR $nnnn`	`$4E`	3	6
Absolute,X	`LSR $nnnn,X`	`$5E`	3	7

Examples

```
        LDA #$03
        LSR A
```

`$03` becomes `$01`. The 1 that was the least significant bit goes out to C.

(c) 2026 masanorisakai / MIT License

```
        LDA #$80
        LSR A
```

`$80` becomes `$40`. C is 0.

ROL (Rotate Left)

Rotates left by 1 bit, through C. The most significant bit goes out to C, and the old C enters the least significant bit. N, Z, and C change.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	○

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Accumulator	`ROL A`	`$2A`	1	2
Zero page	`ROL $nn`	`$26`	2	5
Zero page,X	`ROL $nn,X`	`$36`	2	6
Absolute	`ROL $nnnn`	`$2E`	3	6
Absolute,X	`ROL $nnnn,X`	`$3E`	3	7

Examples

```
        SEC
        LDA #$80
        ROL A
```

With C at 1, `$80` becomes `$01`. The old C (1) enters the least significant bit, and the 1 that was the most significant bit goes out to C.

```
        CLC
        LDA #$40
        ROL A
```

With C at 0, `$40` becomes `$80`. N is 1.

Setup: memory from `$0400` holds `81 42`.

```
        ASL $0400
        ROL $0401
```

(c) 2026 masanorisakai / MIT License

This shifts a 16-bit value left. It doubles `$4281` with `ASL` and `ROL` and gets `$8502`. The C that the low-byte ASL shifts out enters the least significant bit of the high-byte ROL.

ROR (Rotate Right)

Rotates right by 1 bit, through C. The least significant bit goes out to C, and the old C enters the most significant bit. N, Z, and C change.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	○

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Accumulator	`ROR A`	`$6A`	1	2
Zero page	`ROR $nn`	`$66`	2	5
Zero page,X	`ROR $nn,X`	`$76`	2	6
Absolute	`ROR $nnnn`	`$6E`	3	6
Absolute,X	`ROR $nnnn,X`	`$7E`	3	7

Examples

```
SEC
LDA #$01
ROR A
```

With C at 1, `$01` becomes `$80`. N is 1 and C is also 1.

```
CLC
LDA #$01
ROR A
```

With C at 0, `$01` becomes `$00`. Z is 1 and C is 1.

The absolute,X mode of shifts and rotates (for example `ASL $0410,X`) reads memory and writes the result back. It takes the same number of cycles whether or not the access crosses a page.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

The results of `AND`, `ORA`, and `EOR` are written to `$03F0`, `$03F1`, and `$03F2`.

```
*E 0300 A9 B5 29 0F 8D F0 03 A9 30 09 05 8D F1 03 A9 FF
*E 0310 49 0F 8D F2 03 60
*G 0300
*D 03F0 03F2
03F0: 05 35 F0 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$B5 AND $0F` is `05`, `$30 ORA $05` is `35`, and `$FF EOR $0F` is `F0`.

What you can try in Glow65

Put `$4281` at `$0400` and `$0401` (`81 42`, low byte first), and double it with `ASL` and `ROL`.

```
*E 0400 81 42
*E 0300 0E 00 04 2E 01 04 60
*G 0300
*D 0400 0401
0400: 02 85 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

`$0400` holds `02` and `$0401` holds `85`, so the answer is `$8502`.

What you can try in Glow65

Put `$C0` at `$0400`, run `LDA #$01` and `BIT $0400`, and write P to `$03F0`.

```
*E 0400 C0
*E 0300 A9 01 2C 00 04 08 68 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: F6 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$F6` is `%11110110`. N (bit 7), V (bit 6), and Z (bit 1) are 1. Bit 5 and B (bit 4) are set to 1 by PHP, and I (bit 2) is also 1.

(c) 2026 masanorisakai / MIT License

5.5 Branch and Jump

These instructions change the flow of execution. The conditional branches (BCC, BCS, BEQ, BNE, BMI, BPL, BVC, BVS) look at a flag and jump by a distance measured from the address of the next instruction. JMP jumps to the given address without any condition.

JSR and RTS call and return from a subroutine (a small program that other code calls). BRK and RTI are the entrance and exit of an interrupt. These use the stack.

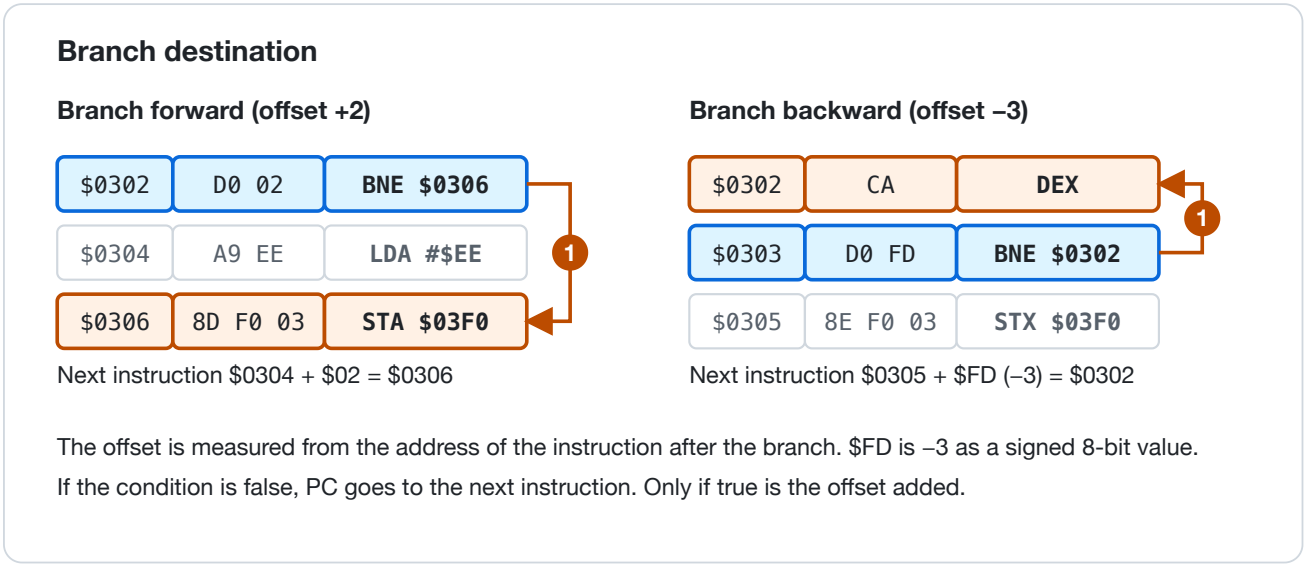


Figure 5-3 The target of a branch. It can go forward or backward.

(c) 2026 masanorisakai / MIT License

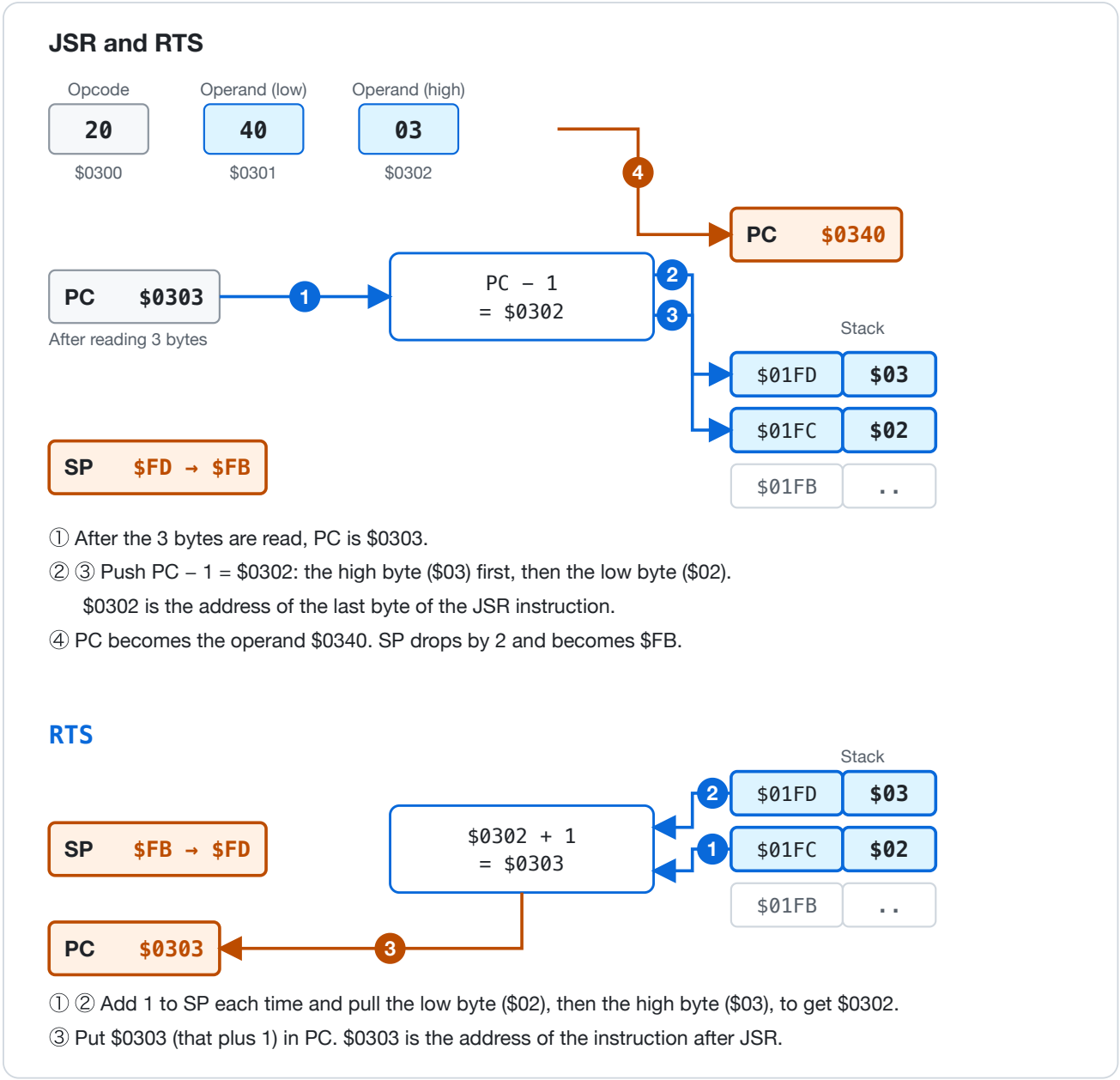


Figure 5-4 JSR and RTS. The pushed return address is the address of the last byte of the JSR.

(c) 2026 masanorisakai / MIT License

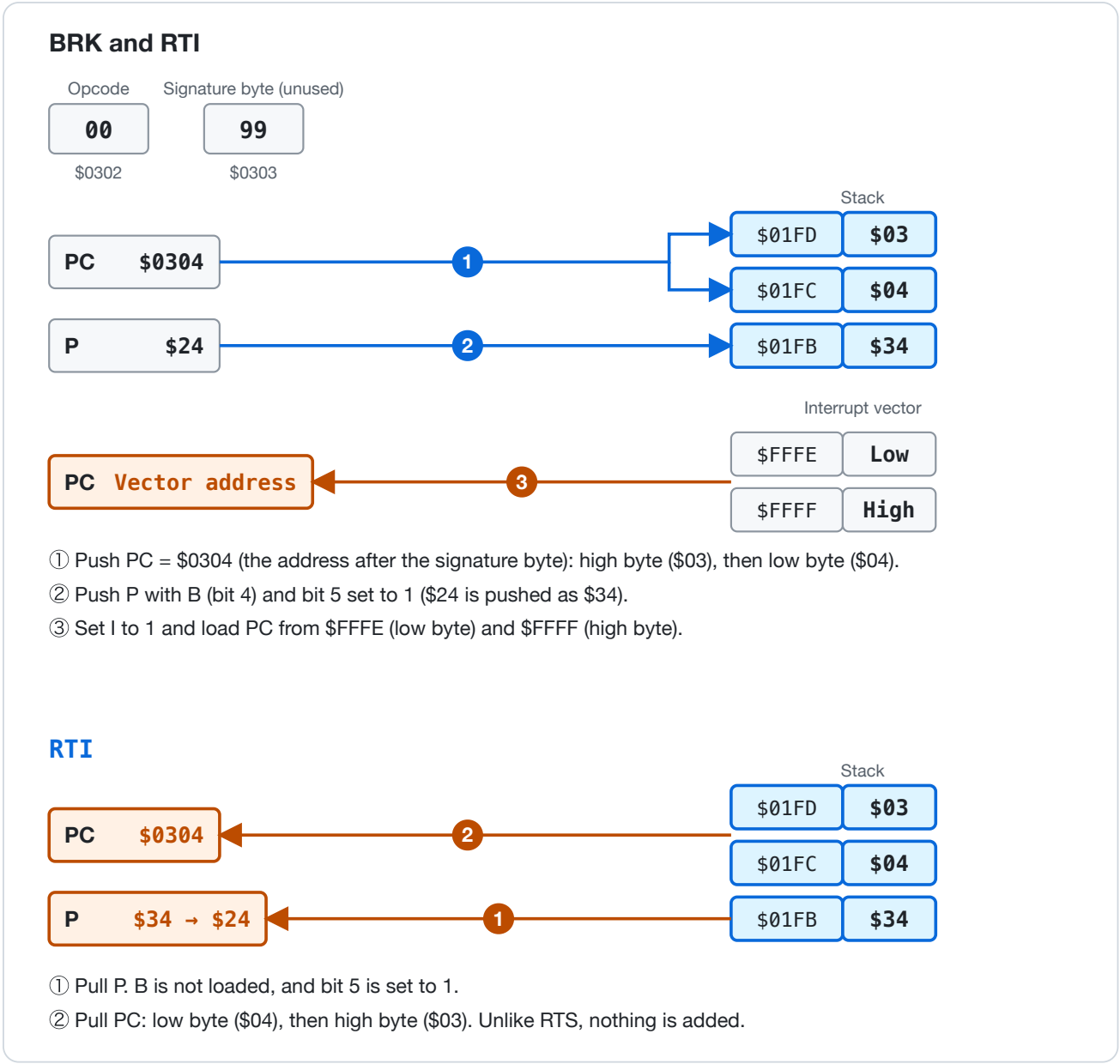


Figure 5-5 BRK and RTI.

Branch instructions do not change the flags. The distance is a signed 1-byte value, from -128 to +127. A branch takes 2 cycles when the condition is false, 3 when it is true, and 4 when it is true and the target is on another page[1].

BCC (Branch if Carry Clear)

Branches when C is 0. It is also used to test "less than" after CMP.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

(c) 2026 masanorisakai / MIT License

Addressing mode	Syntax	Opcode	Bytes	Cycles
Relative	`BCC label`	`$90`	2	2 (+1, +2)

Examples

```
        CLC
        LDX #$00
        BCC skip
        LDX #$EE
skip:   NOP
```

C is 0, so the branch is taken and `LDX #$EE` is skipped. X stays `$00`.

BCS (Branch if Carry Set)

Branches when C is 1. It is also used to test "greater than or equal" after CMP.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Relative	`BCS label`	`$B0`	2	2 (+1, +2)

Examples

```
        CLC
        LDX #$00
        BCS skip
        LDX #$EE
skip:   NOP
```

C is 0, so the branch is not taken. `LDX #$EE` runs and X becomes `$EE`.

BEQ (Branch if Equal)

Branches when Z is 1. This is the case when the previous result was 0, or when CMP found the values equal.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

(c) 2026 masanorisakai / MIT License

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Relative	`BEQ label`	`$F0`	2	2 (+1, +2)

Examples

```
        LDX #$00
        LDA #$00
        BEQ skip
        LDX #$EE
skip:   NOP
```

`LDA #$00` set Z to 1, so the branch is taken. X stays `$00`.

BNE (Branch if Not Equal)

Branches when Z is 0. It is often used to test for the end of a loop.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Relative	`BNE label`	`$D0`	2	2 (+1, +2)

Examples

```
        LDY #$00
        LDX #$03
loop:   INY
        DEX
        BNE loop
```

`INY` repeats until `DEX` makes X 0. The loop runs 3 times, and Y becomes `$03`.

BMI (Branch if Minus)

Branches when N is 1. This is the case when the most significant bit of the previous result is 1.

Flags affected

(c) 2026 masanorisakai / MIT License

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Relative	`BMI label`	`$30`	2	2 (+1, +2)

Examples

```
        LDX #$00
        LDA #$80
        BMI skip
        LDX #$EE
skip:   NOP
```

`LDA #$80` set N to 1, so the branch is taken. X stays `$00`.

BPL (Branch if Plus)

Branches when N is 0.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Relative	`BPL label`	`$10`	2	2 (+1, +2)

Examples

```
        LDX #$00
        LDA #$7F
        BPL skip
        LDX #$EE
skip:   NOP
```

`LDA #$7F` set N to 0, so the branch is taken. X stays `$00`.

BVC (Branch if Overflow Clear)

Branches when V is 0.

(c) 2026 masanorisakai / MIT License

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Relative	`BVC label`	`$50`	2	2 (+1, +2)

Examples

```
        CLV
        LDX #$00
        BVC skip
        LDX #$EE
skip:   NOP
```

`CLV` set V to 0, so the branch is taken. X stays `$00`.

BVS (Branch if Overflow Set)

Branches when V is 1.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Relative	`BVS label`	`$70`	2	2 (+1, +2)

Examples

```
        CLV
        LDX #$00
        BVS skip
        LDX #$EE
skip:   NOP
```

`CLV` set V to 0, so the branch is not taken. `LDX #$EE` runs and X becomes `$EE`.

(c) 2026 masanorisakai / MIT License

JMP (Jump)

Jumps to the given address. The flags do not change. It has absolute and indirect modes.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Absolute	`JMP $nnnn`	`$4C`	3	3
Indirect	`JMP ($nnnn)`	`$6C`	3	5

Examples

```
        LDA #$00
        JMP skip
        LDA #$EE
skip:   LDX #$11
```

`LDA #$EE` is skipped, and execution continues from `LDX #$11`. A stays `$00`.

The indirect mode (`JMP ($nnnn)`) has a quirk on the NMOS 6502 when the pointer is at the last byte of a page (`$xxFF`). Section 4.6 in Chapter 4 explains it.

JSR (Jump to Subroutine)

Pushes the address of the next instruction minus 1 (the address of the last byte of the JSR) onto the stack, and jumps to the given address. The high byte is pushed first, then the low byte[1]. The flags do not change.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Absolute	`JSR $nnnn`	`$20`	3	6

Examples

(c) 2026 masanorisakai / MIT License

```
        LDA #$00
        JSR $0340
        TAX
```

The subroutine at `$0340`:

```
        LDA #$AB
        RTS
```

The subroutine at `$0340` sets A to `$AB` and returns. After the return, `TAX` makes X `$AB` as well.

```
        JSR $0340
```

The subroutine at `$0340`:

```
        TSX
        LDA $0101,X
        STA $03E4
        LDA $0102,X
        STA $03E5
        RTS
```

The subroutine reads the return address that was pushed onto the stack. The JSR at `$0300` pushed the address of its last byte, `$0302`, so the low byte is `$02` and the high byte is `$03`.

RTS (Return from Subroutine)

Pulls the return address from the stack, adds 1, and puts it in PC[1]. Execution returns to the instruction after the JSR. The flags do not change.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`RTS`	`$60`	1	6

Examples

```
        LDX #$00
        LDA #$03
        PHA
        LDA #$0A
        PHA
        RTS
        LDX #$EE
        LDY #$01
```

Push `$030A` yourself and run RTS, and execution continues from `$030B` (`LDY #$01`), one past `$030A`. `LDX #$EE` is skipped.

The value that JSR pushes is 1 less than the address of the next instruction, and RTS adds 1. If you push a return address onto the stack yourself and use RTS, execution continues from the address after it.

BRK (Break)

Causes a software interrupt. It pushes PC (the address of the BRK + 2) and P with B set to 1 onto the stack, sets I to 1, and jumps to the address stored at `$FFFE` and `$FFFF`.

Flags affected

N	V	D	I	Z	C
-	-	-	1	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`BRK`	`$00`	1	7

Examples

```
        LDX #$00
        LDY #$00
        BRK
        .byte $99
        LDY #$55
```

The interrupt handler at `$0340`:

(c) 2026 masanorisakai / MIT License

```
        TSX
        LDA $0101,X
        STA $03E4
        LDA $0102,X
        STA $03E5
        LDA $0103,X
        STA $03E6
        LDX #$77
        RTI
```

The byte after `BRK` is a signature byte and is not used. The return address is `$0306`, the address after the signature byte. The handler sets X to `$77` and returns with RTI, so `LDY #$55` runs next.

BRK is a 1-byte instruction, but PC advances by 2 bytes. The following byte can serve as a signature that shows which kind of BRK it is.

The Glow65 monitor jumps from the vector at `$FFFE` and `$FFFF` to the handler address stored in RAM at `$03FA` and `$03FB`. In this example, `E 03FA 40 03` sets the handler address to `$0340`. Interrupts are explained in Chapter 6.

RTI (Return from Interrupt)

Pulls P, the low byte of PC, and the high byte of PC from the stack, in this order, and returns from the interrupt. Unlike RTS, it does not add 1 to PC. RTI does not load B (bit 4) of P.

Flags affected

N	V	D	I	Z	C
Restored	Restored	Restored	Restored	Restored	Restored

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`RTI`	`$40`	1	6

Examples

In the `BRK` example, the handler's last `RTI` returns to `$0306`, the address after the signature byte.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

`BNE` repeats `INY` and `DEX` 3 times. `L` shows where `BNE $0304` jumps.

```
*E 0300 A0 00 A2 03 C8 CA D0 FC 8C F0 03 60
*L 0300 030A
0300  A0 00     LDY   #$00
0302  A2 03     LDX   #$03
0304  C8        INY
0305  CA        DEX
0306  D0 FC     BNE   $0304
0308  8C F0 03  STY   $03F0
*G 0300
*D 03F0 03F0
03F0: 03 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

The distance `$FC` in `D0 FC` is -4. Counting back 4 from the next instruction, `$0308`, gives `$0304`.

What you can try in Glow65

The subroutine at `$0340` writes the return address that was pushed onto the stack to `$03F2` and `$03F3`.

```
*E 0340 BA BD 01 01 8D F2 03 BD 02 01 8D F3 03 60
*E 0300 20 40 03 60
*L 0300 0300
0300  20 40 03  JSR   $0340
*G 0300
*D 03F2 03F3
03F2: 02 03 00 00 00 00 24 F4 24 F4 00 00 00 00 00 00
```

The low byte `02` and the high byte `03` make the pushed value `$0302`. This is the address of the last byte of the `JSR`.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

`BRK` jumps to the handler at `$0340`, which writes the pushed return address to `$03F2` (high byte) and `$03F3` (low byte), and returns with RTI.

```
*E 03FA 40 03
*E 0340 BA BD 03 01 8D F2 03 BD 02 01 8D F3 03 A2 77 40
*E 0300 A2 00 A0 00 00 99 A0 55 8E F0 03 8C F1 03 60
*L 0300 0304
0300  A2 00     LDX   #$00
0302  A0 00     LDY   #$00
0304  00        BRK
*G 0300
*D 03F0 03F3
03F0: 77 55 03 06 00 00 00 00 24 F4 40 03 00 00 00 00
```

`$03F0` and `$03F1` hold X and Y after the return, `77` and `55`. `$03F2` and `$03F3` hold the return address, `$0306`. This is the address after the signature byte at `$0305`. The `40 03` at `$03FA` and `$03FB` is the handler address written in this example.

Sources

1. MOS Technology, "MCS6500 Microcomputer Family Hardware Manual", January 1976 edition: https://www.commodore.ca/wp-content/uploads/2018/11/kim1_hardware.txt

(c) 2026 masanorisakai / MIT License

5.6 Stack and Flag Operations

The stack is an area of memory from `$0100` to `$01FF`, used in last-in, first-out order: the value put in last is taken out first. SP (the stack pointer) points to the low byte of the address to write next. To push, the CPU writes the value and then subtracts 1 from SP. To pull, it adds 1 to SP and then reads.

This group contains the instructions that push (PHA, PHP) and pull (PLA, PLP), the instructions that change a flag directly, and NOP, which does nothing.

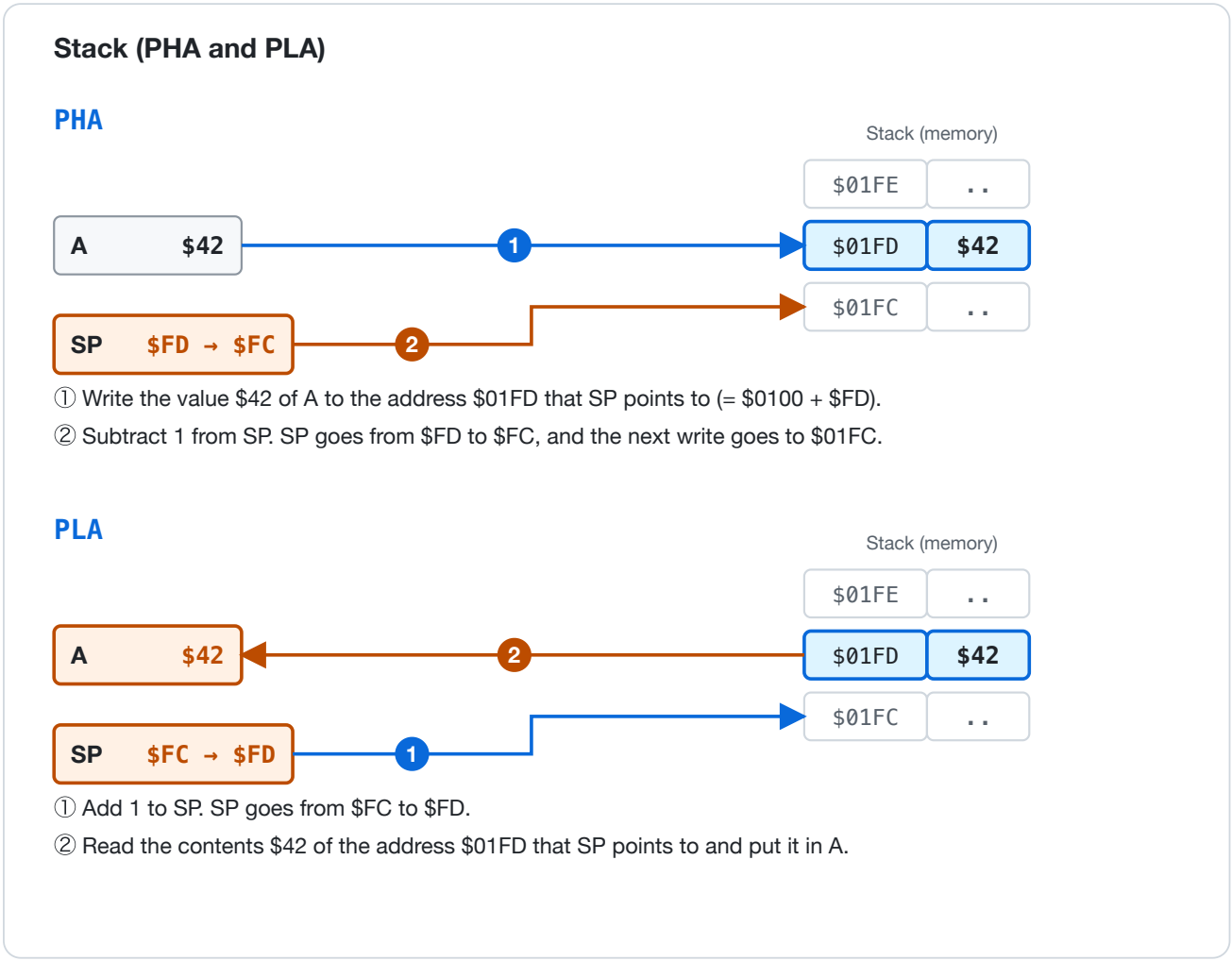


Figure 5-6 PHA and PLA. The stack grows from higher addresses to lower addresses.

PHA (Push Accumulator)

Writes the value of A to `$0100 + SP`, then subtracts 1 from SP. Neither A nor the flags change.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

(c) 2026 masanorisakai / MIT License

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`PHA`	`$48`	1	3

Examples

```
LDA #$42
PHA
TSX
LDA $0101,X
STA $03E4
LDA #$00
PLA
```

After A (`$42`) is pushed, `TSX` reads SP into X, and `LDA $0101,X` reads the address one above SP, where the pushed `$42` is found. Then A is set to `$00`, and `PLA` brings `$42` back.

PLA (Pull Accumulator)

Adds 1 to SP and loads the value at `$0100 + SP` into A. N and Z are set from the pulled value.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`PLA`	`$68`	1	4

Examples

```
LDA #$42
PHA
LDA #$00
PLA
```

The `$42` that was pushed returns to A with `PLA`. Both N and Z are 0.

PHP (Push Processor Status)

Pushes P onto the stack. The pushed value is P with B (bit 4) and bit 5 set to 1. P itself does not change.

Flags affected

(c) 2026 masanorisakai / MIT License

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`PHP`	`$08`	1	3

Examples

```
SEI
CLD
CLV
LDA #$01
SEC
PHP
PLA
```

`SEI`, `CLD`, `CLV`, `LDA #$01`, and `SEC` leave only C and I at 1, and then P is pushed. The pulled value is `$35` (`%00110101`). Bit 5 and B are 1.

PLP (Pull Processor Status)

Replaces P with the value pulled from the stack. PLP ignores B (bit 4) of the pulled value, and bit 5 stays 1. N, V, D, I, Z, and C all take the pulled value.

Flags affected

N	V	D	I	Z	C
Restored	Restored	Restored	Restored	Restored	Restored

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`PLP`	`$28`	1	4

Examples

```
SEC
PHP
CLC
PLP
```

After `SEC`, P is pushed, `CLC` sets C to 0, and `PLP` restores P. C returns to 1.

(c) 2026 masanorisakai / MIT License

```
        LDA #$C3
        PHA
        PLP
```

Push `$C3` (`%11000011`) and pull it with `PLP`, and N, V, Z, and C become 1 and D becomes 0.

PHP and PLP are used to save and restore the flags around an interrupt or a subroutine.

CLC (Clear Carry Flag)

Sets C to 0. Use it before ADC.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	0

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`CLC`	`$18`	1	2

Examples

```
        SEC
        CLC
```

C becomes 0.

SEC (Set Carry Flag)

Sets C to 1. Use it before SBC.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	1

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`SEC`	`$38`	1	2

Examples

(c) 2026 masanorisakai / MIT License

```
        CLC
        SEC
```

C becomes 1.

CLI (Clear Interrupt Disable)

Sets I to 0. The CPU now accepts interrupts (IRQ).

Flags affected

N	V	D	I	Z	C
-	-	-	0	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`CLI`	`$58`	1	2

Examples

```
        SEI
        CLI
```

I becomes 0.

SEI (Set Interrupt Disable)

Sets I to 1. The CPU no longer accepts interrupts (IRQ).

Flags affected

N	V	D	I	Z	C
-	-	-	1	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`SEI`	`$78`	1	2

Examples

```
        CLI
        SEI
```

(c) 2026 masanorisakai / MIT License

I becomes 1.

CLD (Clear Decimal Mode)

Sets D to 0. ADC and SBC calculate in binary again.

Flags affected

N	V	D	I	Z	C
-	-	0	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`CLD`	`$D8`	1	2

Examples

```
SED
CLD
```

D becomes 0.

SED (Set Decimal Mode)

Sets D to 1. ADC and SBC calculate in BCD.

Flags affected

N	V	D	I	Z	C
-	-	1	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`SED`	`$F8`	1	2

Examples

```
CLD
SED
```

D becomes 1.

After you use decimal mode, switch back with CLD. If D stays 1, the results of later additions and subtractions change.

(c) 2026 masanorisakai / MIT License

CLV (Clear Overflow Flag)

Sets V to 0. The 6502 has no instruction that sets V to 1. ADC, SBC, and BIT set it to 1 depending on their results.

Flags affected

N	V	D	I	Z	C
-	0	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`CLV`	`$B8`	1	2

Examples

```
CLC
LDA #$7F
ADC #$01
CLV
```

`ADC` gives `$7F + $01 = $80` and sets V to 1, and then `CLV` returns it to 0. N stays 1.

NOP (No Operation)

Does nothing. It is a 1-byte instruction and takes 2 cycles. No register or flag changes.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`NOP`	`$EA`	1	2

Examples

```
LDA #$80
NOP
```

After `NOP`, A (`$80`) and N (1) are unchanged.

NOP is used to leave a temporary gap in the middle of a program, to adjust execution time, and so on.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

After `SEI`, `CLD`, `CLV`, `LDA #$01`, and `SEC`, `PHP` pushes P, `PLA` pulls it into A, and the program writes it to `$03F0`.

```
*E 0300 78 D8 B8 A9 01 38 08 68 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 35 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$35` is `%00110101`. C (bit 0) and I (bit 2) are 1. Bit 5 and B (bit 4) are set to 1 by PHP.

What you can try in Glow65

`PHA` and `PLA` save and restore the value of A. Even after `LDA #$00` clears A, `PLA` brings `$42` back.

```
*E 0300 A9 42 48 A9 00 68 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 42 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

What you can try in Glow65

`SEC`, `PHP`, `CLC`, and `PLP` save C and restore it.

```
*E 0300 38 08 18 28 08 68 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 35 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$35` shows that C is back to 1.

(c) 2026 masanorisakai / MIT License

5.7 Instructions Added by the 65C02

The 65C02 is an improved version of the 6502. The 6502 is built with a process called NMOS, and the 65C02 is built with a process called CMOS. CMOS uses less power. All 151 official opcodes of the 6502 also exist on the 65C02. The 65C02 adds new instructions and modes to them, and fixes some behavior.

The 65C02 in Glow65 has the same instructions as the WDC W65C02S. These are the base 65C02 instructions, plus BBR, BBS, RMB, and SMB (added by Rockwell), and WAI and STP (added by WDC).

Differences from the 6502

There are 14 new instructions. They are listed in "List of New Instructions".

There are 3 new modes: zero page indirect `($nn)`, absolute,X indirect `($nnnn,X)`, and zero page relative `$nn,label`. Some 6502 instructions also get new forms, such as `INC A`, `DEC A`, `BIT #$nn`, `BIT $nn,X`, and `BIT $nnnn,X`.

The following behavior changes.

Item	NMOS 6502	65C02
`JMP ($xxFF)`	Reads the high byte from the start of the same page. 5 cycles	Reads the high byte from the address after the pointer. 6 cycles
When an interrupt (IRQ, NMI, BRK) is accepted	D does not change	D is set to 0
ADC in decimal mode	Z comes from the binary sum, and N from an intermediate value	N and Z come from the adjusted A. 1 cycle more
SBC in decimal mode	N and Z come from the binary difference	N and Z come from the adjusted A. 1 cycle more
`$nnnn,X` of ASL, LSR, ROL, ROR	7 cycles	6 cycles. 7 when it crosses into another page
Undefined opcodes	In Glow65, a 1-byte, 2-cycle NOP	All are NOPs. The length and cycles are fixed for each opcode

D is set to 0 so that an interrupt handler is not affected by decimal mode. The behavior of `JMP ($xxFF)` is explained under "Indirect" in chapter 4.

Running the 65C02 in Glow65

The **CPU** switch on the panel selects the 6502 or the 65C02. The switch works only while POWER is OFF. On the command line, give `glow65-cli --cpu 65c02`. Without the option, the CPU is a 6502.

Only when the 65C02 is selected does the monitor's `L` command disassemble the new instructions. On a 6502, the same bytes are shown as `.BYTE`.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

Disassemble the same 5 bytes on the 65C02 and on the 6502. `$CB` is WAI and `$DB` is STP.

65C02:

```
*E 0300 1A DA FA CB DB
*L 0300 0304
0300  1A        INC   A
0301  DA        PHX
0302  FA        PLX
0303  CB        WAI
0304  DB        STP
```

6502:

```
*E 0300 1A DA FA CB DB
*L 0300 0304
0300  1A        .BYTE $1A
0301  DA        .BYTE $DA
0302  FA        .BYTE $FA
0303  CB        .BYTE $CB
0304  DB        .BYTE $DB
```

The results of running them differ too. The following program adds 1 to A with `INC A`, pushes X and pulls it back with `PHX` and `PLX`, and writes 0 to `$03F2` with `STZ`. `$FF` is put in `$03F2` beforehand.

```
      LDA #$05
      INC A
      STA $03F0
      LDX #$07
      PHX
      LDX #$00
      PLX
      STX $03F1
      STZ $03F2
      RTS
```

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

65C02:

```
*E 03F2 FF
*E 0300 A9 05 1A 8D F0 03 A2 07 DA A2 00 FA 8E F1 03 9C F2 03 60
*G 0300
*D 03F0 03F2
03F0: 06 07 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

On a 6502, the same bytes give a different result. `1A`, `DA`, `FA`, and `9C` run as instructions that do nothing, and the `$FF` in `$03F2` is not overwritten.

```
*E 03F2 FF
*E 0300 A9 05 1A 8D F0 03 A2 07 DA A2 00 FA 8E F1 03 9C F2 03 60
*G 0300
*D 03F0 03F2
03F0: 05 00 FF 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

To assemble with ca65, write `.setcpu "65C02"` at the top of the source. `Tools/assemble.sh` turns it into a binary to be placed at `$0300`.

```
        .setcpu "65C02"
        LDA #$05
        INC A
```

List of New Instructions

Mnemonic	Name	Function	Flags affected
BRA	Branch Always	Always branches	None
PHX	Push X Register	Pushes X onto the stack	None
PHY	Push Y Register	Pushes Y onto the stack	None
PLX	Pull X Register	Pulls X from the stack	N Z
PLY	Pull Y Register	Pulls Y from the stack	N Z
STZ	Store Zero	Writes 0 to memory	None
TRB	Test and Reset Bits	ANDs A with memory to set Z, then clears in memory the bits that are 1 in A	Z
TSB	Test and Set Bits	ANDs A with memory to set Z, then sets in memory the bits that are 1 in A	Z

(c) 2026 masanorisakai / MIT License

Mnemonic	Name	Function	Flags affected
WAI	Wait for Interrupt	Waits until an interrupt arrives	None
STP	Stop the Clock	Stops until RESET	None
BBR	Branch on Bit Reset	Branches if bit n of a zero page value is 0	None
BBS	Branch on Bit Set	Branches if bit n of a zero page value is 1	None
RMB	Reset Memory Bit	Clears bit n of a zero page value	None
SMB	Set Memory Bit	Sets bit n of a zero page value	None

`BBR`, `BBS`, `RMB`, and `SMB` have a separate opcode for each bit number. They are written with the number, as in `BBR0` to `BBR7`.

New Instructions

BRA (Branch Always)

Always branches, with no condition. The offset is a signed 1-byte value, from -128 to +127, as for the other branch instructions. No flags change.

It takes 3 cycles. If the target is on another page, it takes 4.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Relative	`BRA label`	`$80`	2	3 (+1)

Examples

```
        LDX #$00
        BRA skip
        LDX #$EE
skip:   STX $03F0
```

`BRA` always branches, so `LDX #$EE` is skipped. X stays `$00`. On a 6502, to branch every time, you used `JMP` or a branch on a flag whose value you knew. `BRA` takes only 2 bytes, and it needs no rewriting if the code is moved.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

```
*E 0300 A2 00 80 02 A2 EE 8E F0 03 60
*G 0300
*D 03F0 03F0
03F0: 00 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

PHX (Push X Register)

Writes the value of X to `$0100 + SP`, then subtracts 1 from SP. Neither X nor the flags change. On a 6502, you pushed X with `TXA` and `PHA`, which destroyed A.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`PHX`	`$DA`	1	3

Examples

```
LDX #$07
PHX
LDX #$00
PLX
```

This pushes the `$07` in X, sets X to `$00`, and brings it back to `$07` with `PLX`.

PHY (Push Y Register)

Pushes the value of Y onto the stack. It works like PHX, but on Y.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`PHY`	`$5A`	1	3

(c) 2026 masanorisakai / MIT License

Examples

```
        LDY #$33
        PHY
        LDY #$00
        PLY
        STY $03F0
```

This pushes the `$33` in Y, sets Y to `$00`, and brings it back to `$33` with `PLY`. `$33` is written to `$03F0`.

> **What you can try in Glow65**
>
> ```
> *E 0300 A0 33 5A A0 00 7A 8C F0 03 60
> *G 0300
> *D 03F0 03F0
> 03F0: 33 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```

PLX (Pull X Register)

Adds 1 to SP and puts the value at `$0100 + SP` into X. N and Z are set from the pulled value.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`PLX`	`$FA`	1	4

Examples

See the PHX example. `PLX` brings X back to `$07`, and N and Z are 0.

PLY (Pull Y Register)

Pulls a value from the stack into Y. N and Z are set from the pulled value.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

(c) 2026 masanorisakai / MIT License

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`PLY`	`$7A`	1	4

Examples

See the PHY example. `PLY` brings Y back to `$33`.

STZ (Store Zero)

Writes 0 to memory. No register or flag changes. On a 6502, to clear memory you set A to `$00` and used `STA`, which destroyed A.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page	`STZ $nn`	`$64`	2	3
Zero page,X	`STZ $nn,X`	`$74`	2	4
Absolute	`STZ $nnnn`	`$9C`	3	4
Absolute,X	`STZ $nnnn,X`	`$9E`	3	5

Examples

```
        LDA #$55
        STZ $03F2
```

`$03F2` becomes `$00`. A stays `$55`. A run is shown in "Running the 65C02 in Glow65".

TRB (Test and Reset Bits)

ANDs A with the memory value and sets Z to 1 only if the result is 0. The result itself is discarded. Then, in the memory value, it clears the bits that are 1 in A and writes the value back. A does not change.

(c) 2026 masanorisakai / MIT License

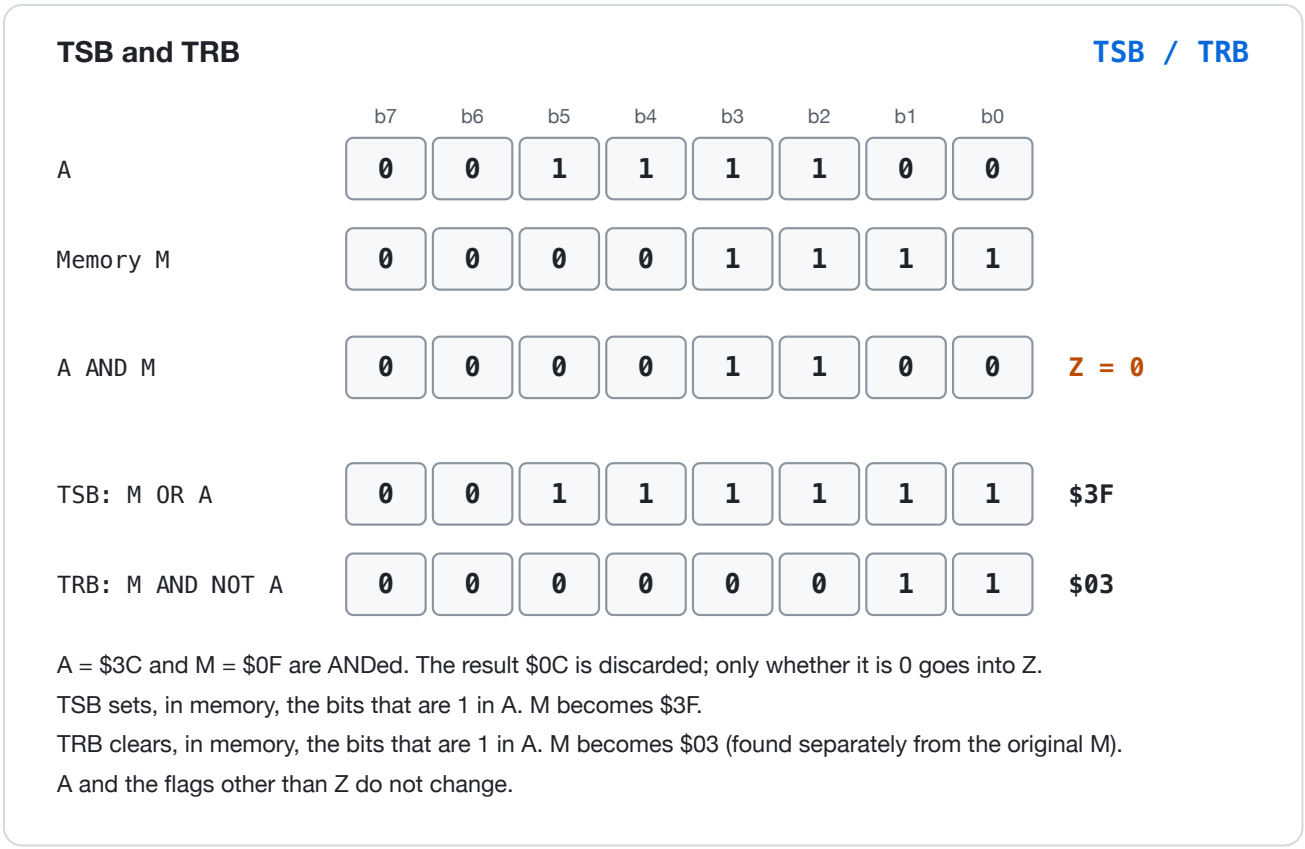


Figure 5-9 TSB and TRB. The 1 bits of A decide which bits of memory are rewritten.

Flags affected

N	V	D	I	Z	C
-	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page	`TRB $nn`	`$14`	2	5
Absolute	`TRB $nnnn`	`$1C`	3	6

Examples

```
LDA #$0F
STA $20
LDA #$3C
TRB $20
```

`$0F` at `$20` AND `$3C` in A is `$0C`, which is not 0, so Z is 0. The 1 bits of A are cleared in `$20`, so `$20` becomes `$03`.

(c) 2026 masanorisakai / MIT License

TSB (Test and Set Bits)

ANDs A with the memory value and sets Z to 1 only if the result is 0. Then, in the memory value, it sets the bits that are 1 in A and writes the value back. A does not change.

Flags affected

N	V	D	I	Z	C
-	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page	`TSB $nn`	`$04`	2	5
Absolute	`TSB $nnnn`	`$0C`	3	6

Examples

```
        LDA #$0F
        STA $20
        LDA #$3C
        TSB $20
```

`$20` becomes `$0F | $3C = $3F`. Z is 0.

TSB and TRB are used to set or clear particular bits of memory with a single instruction. You do not need to write a test and a rewrite one after the other.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

Put `$0F` in `$20` and `$3C` in A. Write `$20` after `TSB` to `$03F0`, and `$20` after `TRB` to `$03F1`.

```
*E 0300 A9 0F 85 20 A9 3C 04 20 A6 20 8E F0 03 14 20 A6 20 8E F1 03 60
*L 0300 0310
0300  A9 0F     LDA   #$0F
0302  85 20     STA   $20
0304  A9 3C     LDA   #$3C
0306  04 20     TSB   $20
0308  A6 20     LDX   $20
030A  8E F0 03  STX   $03F0
030D  14 20     TRB   $20
030F  A6 20     LDX   $20
*G 0300
*D 03F0 03F1
03F0: 3F 03 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

After `TSB` the value is `$3F`, and after the following `TRB` it is `$03`. `TRB` clears the 1 bits of A from the bits that `TSB` set.

WAI (Wait for Interrupt)

Stops execution and waits for an interrupt. It resumes when an NMI or IRQ signal arrives. No flags change.

If I is 0, the interrupt is accepted and the handler runs. When the handler returns with RTI, execution continues from the instruction after WAI. If I is 1, the interrupt is not accepted, and execution goes on to the instruction after WAI.

While waiting, the CPU does not touch the bus. In Glow65, the HALT LED on the panel lights up. During the wait, one step is one cycle.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`WAI`	`$CB`	1	3

Examples

(c) 2026 masanorisakai / MIT License

```
        CLI
        WAI
        INX
```

This sets I to 0 and waits for an interrupt. `INX` runs after an interrupt arrives and its handler returns.

STP (Stop the Clock)

Stops the CPU. It does not resume until RESET is pressed. An NMI or IRQ does not resume it. No flags change.

While stopped, the CPU does not touch the bus. In Glow65, as with WAI, the HALT LED on the panel lights up.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Implied	`STP`	`$DB`	1	3

Examples

```
        LDA #$01
        STA $03F0
        STP
```

After writing to `$03F0`, the CPU stops.

BBR (Branch on Bit Reset)

Tests bit n of a zero page value and branches if it is 0. n is 0 to 7, and there are 8 opcodes, `BBR0` to `BBR7`. The instruction is 3 bytes: the opcode, the zero page address, and the branch offset, in this order.

The offset is measured from the address of the next instruction, as for the other branch instructions. No flags change.

The bit number n is in bits 4 to 6 of the opcode. `$3F` is `%0011 1111`, and bits 6 to 4 are `011`, so it is `BBR3`.

(c) 2026 masanorisakai / MIT License

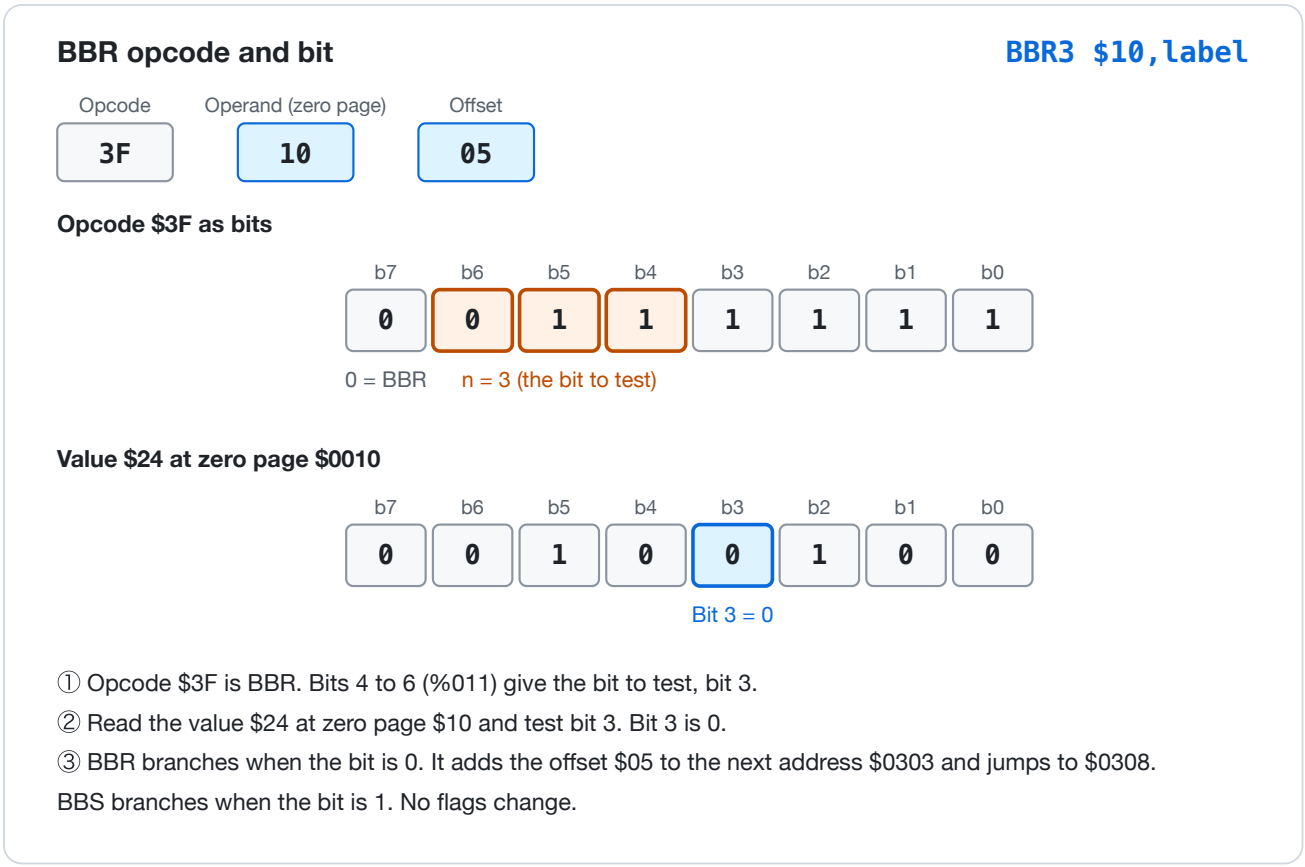


Figure 5-10 BBR3. Bit 3 of the value `$24` at zero page `$10` is 0, so the branch is taken.

It takes 5 cycles. It takes 1 more when the branch is taken, and 1 more again when it goes to another page.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page relative	`BBR0 $nn,label`	`$0F`	3	5 (+1, +2)
Zero page relative	`BBR1 $nn,label`	`$1F`	3	5 (+1, +2)
Zero page relative	`BBR2 $nn,label`	`$2F`	3	5 (+1, +2)
Zero page relative	`BBR3 $nn,label`	`$3F`	3	5 (+1, +2)
Zero page relative	`BBR4 $nn,label`	`$4F`	3	5 (+1, +2)
Zero page relative	`BBR5 $nn,label`	`$5F`	3	5 (+1, +2)
Zero page relative	`BBR6 $nn,label`	`$6F`	3	5 (+1, +2)
Zero page relative	`BBR7 $nn,label`	`$7F`	3	5 (+1, +2)

Examples

(c) 2026 masanorisakai / MIT License

```
        LDA #$00
        STA $10
        SMB3 $10
        LDX #$00
        BBS3 $10,yes
        BRA done
yes:    LDX #$AA
done:   STX $03F0
```

`SMB3` sets bit 3 of zero page `$10` to 1, and `BBS3` tests it and branches to `yes`. X becomes `$AA`.

BBR and BBS can test a bit in zero page without using A or X.

BBS (Branch on Bit Set)

Tests bit n of a zero page value and branches if it is 1. There are 8 opcodes, `BBS0` to `BBS7`. The length, the way the offset is measured, the cycles, and the way the bit number is stored are the same as for BBR. The difference from BBR is that bit 7 of the opcode is 1.

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page relative	`BBS0 $nn,label`	`$8F`	3	5 (+1, +2)
Zero page relative	`BBS1 $nn,label`	`$9F`	3	5 (+1, +2)
Zero page relative	`BBS2 $nn,label`	`$AF`	3	5 (+1, +2)
Zero page relative	`BBS3 $nn,label`	`$BF`	3	5 (+1, +2)
Zero page relative	`BBS4 $nn,label`	`$CF`	3	5 (+1, +2)
Zero page relative	`BBS5 $nn,label`	`$DF`	3	5 (+1, +2)
Zero page relative	`BBS6 $nn,label`	`$EF`	3	5 (+1, +2)
Zero page relative	`BBS7 $nn,label`	`$FF`	3	5 (+1, +2)

Examples

See the BBR example. `BBS3 $10,yes` branches to `yes` if bit 3 of `$10` is 1.

RMB (Reset Memory Bit)

Sets bit n of a zero page value to 0. n is 0 to 7, and there are 8 opcodes, `RMB0` to `RMB7`. The other bits do not change. No flags change.

(c) 2026 masanorisakai / MIT License

Flags affected

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page	`RMB0 $nn`	`$07`	2	5
Zero page	`RMB1 $nn`	`$17`	2	5
Zero page	`RMB2 $nn`	`$27`	2	5
Zero page	`RMB3 $nn`	`$37`	2	5
Zero page	`RMB4 $nn`	`$47`	2	5
Zero page	`RMB5 $nn`	`$57`	2	5
Zero page	`RMB6 $nn`	`$67`	2	5
Zero page	`RMB7 $nn`	`$77`	2	5

Examples

```
        LDA #$FF
        STA $10
        RMB0 $10
        RMB7 $10
```

Bits 0 and 7 of `$FF` are cleared, so `$10` becomes `$7E`.

> **What you can try in Glow65**
>
> ```
> *E 0300 A9 FF 85 10 07 10 77 10 60
> *L 0304 0306
> 0304  07 10     RMB0  $10
> 0306  77 10     RMB7  $10
> *G 0300
> *D 0010 0010
> 0010: 7E 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
> ```

SMB (Set Memory Bit)

Sets bit n of a zero page value to 1. There are 8 opcodes, `SMB0` to `SMB7`. The other bits do not change. No flags change.

Flags affected

(c) 2026 masanorisakai / MIT License

N	V	D	I	Z	C
-	-	-	-	-	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page	`SMB0 $nn`	`$87`	2	5
Zero page	`SMB1 $nn`	`$97`	2	5
Zero page	`SMB2 $nn`	`$A7`	2	5
Zero page	`SMB3 $nn`	`$B7`	2	5
Zero page	`SMB4 $nn`	`$C7`	2	5
Zero page	`SMB5 $nn`	`$D7`	2	5
Zero page	`SMB6 $nn`	`$E7`	2	5
Zero page	`SMB7 $nn`	`$F7`	2	5

Examples

`SMB3 $10` in the BBR example sets bit 3 of `$10` to 1. `$10` was `$00`, so it becomes `$08`.

> **What you can try in Glow65**
>
> `SMB3` sets bit 3 of `$10`, `BBS3` branches, and X is set to `$AA` and written to `$03F0`.
>
> ```
> *E 0300 A9 00 85 10 B7 10 A2 00 BF 10 02 80 02 A2 AA 8E F0 03 60
> *L 0304 030C
> 0304  B7 10     SMB3  $10
> 0306  A2 00     LDX   #$00
> 0308  BF 10 02  BBS3  $10,$030D
> 030B  80 02     BRA   $030F
> *G 0300
> *D 03F0 03F0
> 03F0: AA 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> *D 0010 0010
> 0010: 08 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
> ```
>
> The offset `$02` of `BBS3` is added to the address of the next instruction, `$030B`, giving `$030D`.

New Modes of 6502 Instructions

The following instructions also existed on the 6502. On the 65C02, they have new modes.

(c) 2026 masanorisakai / MIT License

INC A

Adds 1 to A. On the 6502, `INC` worked only on memory. On the 65C02, it works directly on A.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Accumulator	`INC A`	`$1A`	1	2

Examples

```
        LDA #$05
        INC A
```

A becomes `$06`. A run is shown in "Running the 65C02 in Glow65".

DEC A

Subtracts 1 from A.

Flags affected

N	V	D	I	Z	C
○	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Accumulator	`DEC A`	`$3A`	1	2

Examples

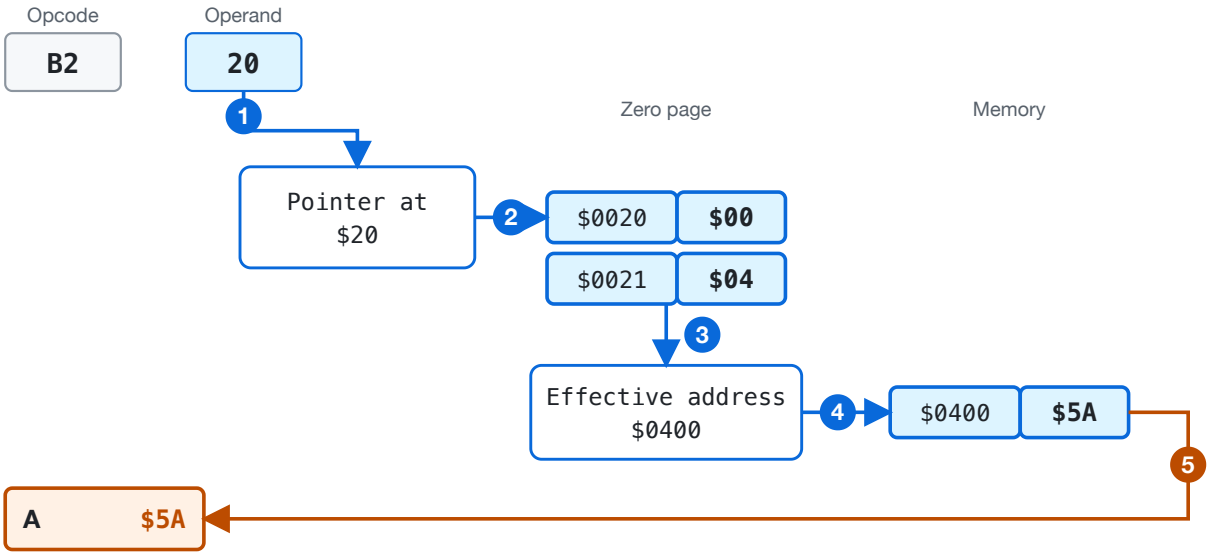
```
        LDA #$00
        DEC A
```

A becomes `$FF`, N becomes 1, and Z becomes 0.

Zero Page Indirect `($nn)`

ORA, AND, EOR, ADC, STA, LDA, CMP, and SBC have this mode. The 2 bytes in zero page are used as the effective address. Unlike `($nn),Y`, Y is not added.

(c) 2026 masanorisakai / MIT License



① Read opcode $B2 (LDA) and operand $20.
② Read $00 at $0020 as the low byte and $04 at $0021 as the high byte.
③ The two bytes together, $0400, are the effective address. Neither X nor Y is added.
④⑤ Read the contents $5A of $0400 and put it in A.

Figure 5-11 Zero page indirect (`LDA ($20)`). The effective address is `$0400`.

The flag changes and the operation are the same as for each instruction. Every instruction takes 5 cycles. ADC and SBC take 1 more in decimal mode.

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Zero page indirect `(zp)`	`ORA ($nn)`	`$12`	2	5
Zero page indirect `(zp)`	`AND ($nn)`	`$32`	2	5
Zero page indirect `(zp)`	`EOR ($nn)`	`$52`	2	5
Zero page indirect `(zp)`	`ADC ($nn)`	`$72`	2	5
Zero page indirect `(zp)`	`STA ($nn)`	`$92`	2	5
Zero page indirect `(zp)`	`LDA ($nn)`	`$B2`	2	5
Zero page indirect `(zp)`	`CMP ($nn)`	`$D2`	2	5
Zero page indirect `(zp)`	`SBC ($nn)`	`$F2`	2	5

Examples

```
LDA ($20)
STA $03F0
```

(c) 2026 masanorisakai / MIT License

When `$0020` holds `00` and `$0021` holds `04`, `LDA ($20)` reads `$0400`. If `$0400` is `$5A`, A becomes `$5A`.

On a 6502, to do the same you had to set Y to `$00` and write `LDA ($20),Y`.

> **What you can try in Glow65**
>
> ```
> *E 0020 00 04
> *E 0400 5A
> *E 0300 B2 20 8D F0 03 60
> *L 0300 0300
> 0300  B2 20     LDA   ($20)
> *G 0300
> *D 03F0 03F0
> 03F0: 5A 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```

New Modes of BIT

BIT gains immediate `#$nn`, zero page,X `$nn,X`, and absolute,X `$nnnn,X`.

Zero page,X and absolute,X change N, V, and Z, as BIT does. Absolute,X takes 1 more cycle when it crosses into another page.

Immediate BIT changes only Z. N and V do not change. The operand is the value itself, so there is no memory bit 7 or 6 to copy.

Flags affected (zero page,X and absolute,X)

N	V	D	I	Z	C
M7	M6	-	-	○	-

Flags affected (immediate)

N	V	D	I	Z	C
-	-	-	-	○	-

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Immediate	`BIT #$nn`	`$89`	2	2
Zero page,X	`BIT $nn,X`	`$34`	2	4
Absolute,X	`BIT $nnnn,X`	`$3C`	3	4+

Examples

(c) 2026 masanorisakai / MIT License

```
        LDA #$01
        BIT #$80
        PHP
        PLA
        STA $03F0
```

`$01` AND `$80` is 0, so Z becomes 1. N stays 0. With `BIT $10`, when the memory is `$80`, N becomes 1.

> **What you can try in Glow65**
>
> The first half is `BIT #$80`. The second half is `BIT $10` with `$80` placed in `$10`. In both, A is `$01` and the AND result is 0. P is pulled and written to `$03F0` and `$03F1`.
>
> ```
> *E 0300 A9 01 89 80 08 68 8D F0 03 A9 80 85 10 A9 01 24 10 08
> *E 0312 68 8D F1 03 60
> *G 0300
> *D 03F0 03F1
> 03F0: 36 B6 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> `$36` is `%00110110`: Z (bit 1) is 1 and N (bit 7) is 0. `$B6` is `%10110110`: N is 1 as well.

JMP Absolute,X Indirect `($nnnn,X)`

The 2 bytes at address `$nnnn + X` are used as the jump target. From a jump table (a table of target addresses), you can jump to the entry chosen by X. An address is 2 bytes, so X goes up in steps of 2. It takes 6 cycles.

One behavior of `JMP ($nnnn)` has changed. The high byte is read from the address after the pointer. On a 6502, when the pointer was at the end of a page (`$xxFF`), it was read from the start of the same page. The cycles go from 5 to 6.

Flags affected

No flags change.

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Indirect	`JMP ($nnnn)`	`$6C`	3	6
Absolute,X indirect `(abs,X)`	`JMP ($nnnn,X)`	`$7C`	3	6

Examples

```
        LDX #$02
        JMP ($0340,X)
```

(c) 2026 masanorisakai / MIT License

X is `$02`, so the 2 bytes at `$0342` and `$0343` are the target. If a table is placed from `$0340`, with `$0320` and then `$0330`, the jump goes to `$0330`.

> **What you can try in Glow65**
>
> Two addresses (`$0320` and `$0330`) are placed at `$0340`. The code at each writes `$11` or `$22` to `$03F0`. X is `$02`, so the jump goes to the second one, `$0330`.
>
> ```
> *E 0340 20 03 30 03
> *E 0330 A9 22 8D F0 03 60
> *E 0320 A9 11 8D F0 03 60
> *E 0300 A2 02 7C 40 03
> *L 0300 0302
> 0300  A2 02     LDX   #$02
> 0302  7C 40 03  JMP   ($0340,X)
> *G 0300
> *D 03F0 03F0
> 03F0: 22 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```

ASL, LSR, ROL, ROR Absolute,X

The operation does not change. The cycles go down from 7 to 6. When it crosses into another page, it takes 1 more, which is 7.

Opcodes

Addressing mode	Syntax	Opcode	Bytes	Cycles
Absolute,X	`ASL $nnnn,X`	`$1E`	3	6+
Absolute,X	`LSR $nnnn,X`	`$5E`	3	6+
Absolute,X	`ROL $nnnn,X`	`$3E`	3	6+
Absolute,X	`ROR $nnnn,X`	`$7E`	3	6+

Opcodes That Were Undefined

On the 65C02, every opcode that was undefined on the 6502 is a NOP. The length and cycles are fixed for each opcode. No register or flag changes. No memory is read.

Bytes	Cycles	Opcodes
1	1	`$03`, `$0B`, `$13`, `$1B`, `$23`, `$2B`, `$33`, `$3B`, `$43`, `$4B`, `$53`, `$5B`, `$63`, `$6B`, `$73`, `$7B`, `$83`, `$8B`, `$93`, `$9B`, `$A3`, `$AB`, `$B3`, `$BB`, `$C3`, `$D3`, `$E3`, `$EB`, `$F3`, `$FB`
2	2	`$02`, `$22`, `$42`, `$62`, `$82`, `$C2`, `$E2`
2	3	`$44`

(c) 2026 masanorisakai / MIT License

Bytes	Cycles	Opcodes
2	4	`$54`, `$D4`, `$F4`
3	4	`$DC`, `$FC`
3	8	`$5C`

A Difference in Decimal Mode

In decimal mode, `ADC` and `SBC` set N and Z differently. The following program adds `$99 + $01` in decimal mode.

```
        SED
        CLC
        LDA #$99
        ADC #$01
        PHP
        STA $03F0
        PLA
        STA $03F1
        CLD
```

The answer is `$00`, and C becomes 1. The result is 0, so it is natural for Z to be 1.

What you can try in Glow65

The value of A is written to `$03F0`, and P right after `ADC` is written to `$03F1`.

65C02:

```
*E 0300 F8 18 A9 99 69 01 08 8D F0 03 68 8D F1 03 D8 60
*G 0300
*D 03F0 03F1
03F0: 00 3F 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

6502:

```
*E 0300 F8 18 A9 99 69 01 08 8D F0 03 68 8D F1 03 D8 60
*G 0300
*D 03F0 03F1
03F0: 00 BD 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

A is `$00` on both. On the 65C02, P is `$3F` (`%00111111`): N is 0, Z is 1, and C is 1. On the 6502, P is `$BD` (`%10111101`): N is 1, Z is 0, and C is 1. On the 6502, Z is 0 even though A is `$00`.

(c) 2026 masanorisakai / MIT License

Practice Problems

The answers and explanations are in the next section.

Problem 1 Write the machine code (a sequence of hexadecimal bytes) for each instruction. (1) `STZ $0310` (2) `INC A` (3) `LDA ($30)` (4) `BBS5 $40,label` (`label` is 2 bytes after the address of the instruction that follows this one)

Problem 2 `80 F4` is at `$0310`. Where does it jump to on the 65C02?

Problem 3 Zero page `$10` holds `$81` (`%10000001`). Which of `BBR0`, `BBR1`, `BBS6`, and `BBS7` branch?

Problem 4 A is `$0F` and zero page `$21` is `$F0`. `TSB $21` and `TRB $21` are each executed. Give `$21` and Z after each. Run each instruction starting from `$F0`.

Problem 5 Zero page `$30` holds `00` and `$31` holds `05`. `$0500` is `$7E` and `$0501` is `$9C`. Y is `$01`. What values go into A for `LDA ($30)` and for `LDA ($30),Y`?

Problem 6 The byte sequence `A9 05 1A 8D F0 03 60` is placed at `$0300` and run. What value is written to `$03F0` on the 65C02 and on the 6502?

Answers and Explanations

Problem 1 (1) `9C 10 03` (2) `1A` (3) `B2 30` (4) `DF 40 02`

- (1) The absolute form of `STZ` is `$9C`. The address is written low byte first, `10 03`. (2) `INC A` is 1 byte, `$1A`. (3) `LDA` in zero page indirect mode is `$B2`. (4) `BBS5` is `$DF` (bits 4 to 6 are `101`, and bit 7 is 1). It is followed by the zero page address `40` and the offset `02`.

Problem 2 `$0306`

- `80 F4` is a 2-byte instruction, so PC is `$0312` after it is read. `$F4` is -12. `$0312 − 12 = $0306`.

Problem 3 `BBR1` and `BBS7`

- `$81` has bits 7 and 0 set to 1, and the others 0. `BBR0` does not branch, because bit 0 is 1. `BBR1` branches, because bit 1 is 0. `BBS6` does not branch, because bit 6 is 0. `BBS7` branches, because bit 7 is 1.

Problem 4 `TSB $21` gives `$FF`, and Z is 1. `TRB $21` leaves `$F0`, and Z is 1.

- In both, the AND result is `$0F AND $F0 = $00`, so Z is 1. TSB sets the 1 bits of A (the low 4 bits), so `$F0 OR $0F = $FF`. TRB clears the 1 bits of A, but the low 4 bits of `$F0` are already 0, so nothing changes.

Problem 5 `LDA ($30)` gives `$7E`. `LDA ($30),Y` gives `$9C`.

- `$30` and `$31` give the base address `$0500`. `($30)` does not add Y, so it reads `$0500`, which is `$7E`. `($30),Y` adds Y (`$01`) and reads `$0501`, which is `$9C`.

(c) 2026 masanorisakai / MIT License

Problem 6 `$06` on the 65C02. `$05` on the 6502.

- On the 65C02, `1A` is `INC A`, so A goes from `$05` to `$06`. On the 6502, `1A` is an undefined opcode that does nothing, so A stays `$05`.

(c) 2026 masanorisakai / MIT License

Chapter 6 Flags, the Stack, and Interrupts

Chapters 3 to 5 covered the registers and the instructions. This chapter explains three mechanisms that work behind them: **flags**, which remember the result of an operation; the **stack**, which holds return addresses and temporary values; and **interrupts**, which pause the running program and move to another piece of code. The chapter ends with the **reset**, which sets the starting conditions.

The hands-on interrupt exercises are in tutorial chapter 6. This chapter gives the background and the overall picture.

6.1 Flags (the P Register)

The P register has 8 bits, and each bit is a **flag**. Six flags are real: N, V, D, I, Z, and C. Bit 5 and bit 4 (B) do not exist inside P.

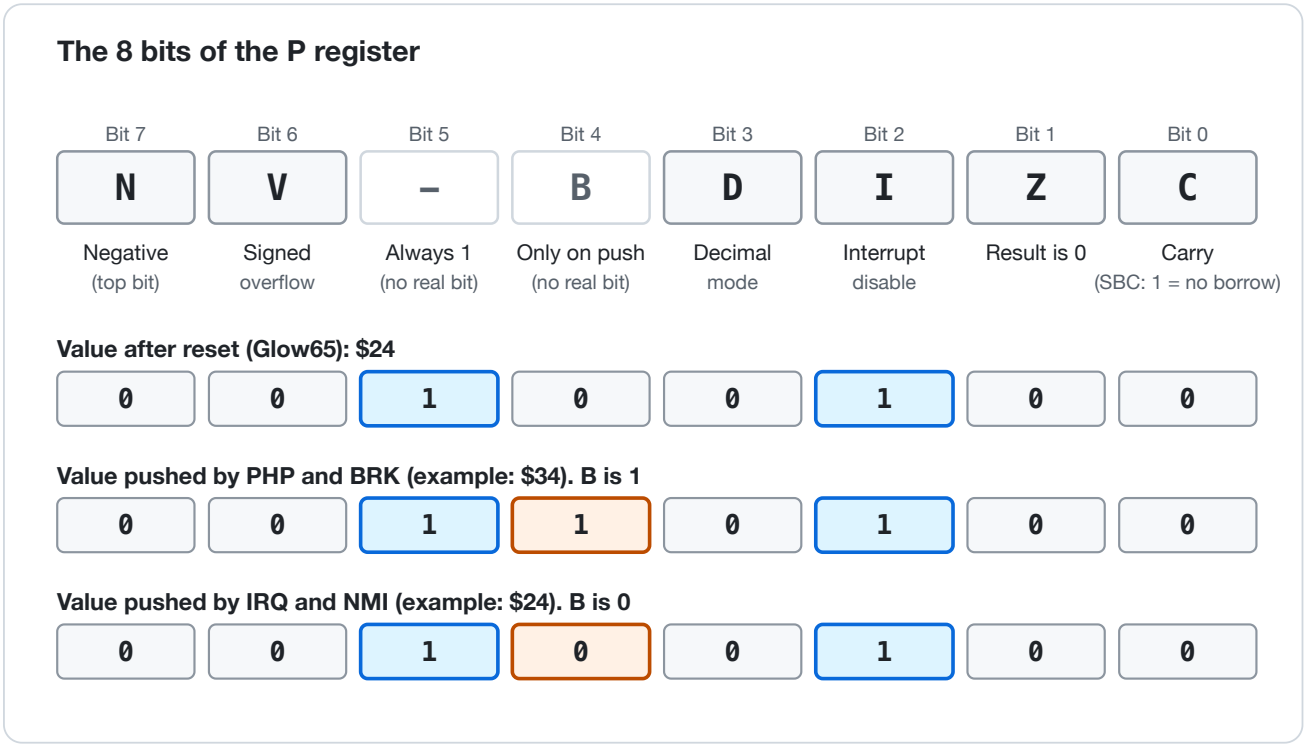


Figure 6-1 The 8 bits of the P register.

6.1.1 N and Z (Result Flags)

N (negative) is a copy of the most significant bit (bit 7) of the last result. Read as a signed number, 1 means negative. **Z** (zero) becomes 1 when the result is 0.

Load, transfer, add, logic, shift, and increment/decrement instructions set N and Z according to the result. Store, branch, and jump instructions do not change them. The example in section 3.6 continues here.

(c) 2026 masanorisakai / MIT License

6.1.2 C (Carry)

C holds the digit that did not fit in 8 bits. Each instruction uses it in its own way.

Instruction	What C does
ADC	Becomes 1 if the result exceeds `$FF`. In an addition, C is also added
SBC	Becomes 1 if the subtraction needs no borrow, 0 if it does. Set C to 1 with SEC before subtracting
ASL, ROL	The most significant bit moves out into C
LSR, ROR	The least significant bit moves out into C
ROL, ROR	The old C moves into the vacated bit
CMP, CPX, CPY	Becomes 1 if the register is greater than or equal to the value

In a 16-bit addition, the ADC on the low byte produces a C, and the ADC on the high byte takes it in. This hand-over is shown in the 16-bit example in section 5.3 and in tutorial chapter 2.

> **What you can try in Glow65**
>
> Write P after `$FF + $01`, `$30 - $50`, and `$50 - $30`, in order, starting at `$03F0`.
>
> ```
> *E 0300 18 A9 FF 69 01 08 68 8D F0 03 38 A9 30 E9 50 08
> *E 0310 68 8D F1 03 38 A9 50 E9 30 08 68 8D F2 03 60
> *G 0300
> *D 03F0 03F2
> 03F0: 37 B4 35 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> `$37` has C and Z set to 1 (`$FF + $01` becomes 0 and a carry comes out). `$B4` has C = 0 and N = 1 (`$30 - $50` needed a borrow). `$35` has C = 1 (`$50 - $30` needs no borrow).

6.1.3 V (Overflow)

V shows overflow when the calculation is read as signed numbers. An 8-bit signed number ranges from -128 to 127. If you add two numbers with the same sign and the sign of the result changes, V becomes 1.

Calculation	Result	Read as signed	V
`$50 + $50`	`$A0`	80 + 80 = 160 (out of range)	1
`$7F + $01`	`$80`	127 + 1 = 128 (out of range)	1
`$50 + $10`	`$60`	80 + 16 = 96 (in range)	0
`$80 + $80`	`$00`	-128 + -128 = -256 (out of range)	1

Look at C when you treat the numbers as unsigned, and at V when you treat them as signed. The same addition gives the two flags different meanings. For example, `$80 + $80` is 128 + 128 = 256 as unsigned numbers, so C is also 1.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

Write P after four additions, in order, starting at `$03F0`. Before each addition, `CLC` sets C to 0.

```
*E 0300 18 A9 50 69 50 08 68 8D F0 03 18 A9 7F 69 01 08
*E 0310 68 8D F1 03 18 A9 50 69 10 08 68 8D F2 03 18 A9
*E 0320 80 69 80 08 68 8D F3 03 60
*G 0300
*D 03F0 03F3
03F0: F4 F4 34 77 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$F4` has N and V set to 1. `$34` has N and V set to 0. `$77` has V, Z, and C set to 1 and N = 0 (`$80 + $80` gives `$00` and a carry out).

6.1.4 D (Decimal Mode)

When **D** is 1, ADC and SBC calculate each 4-bit half as a digit from 0 to 9 (BCD). `$09 + $01` gives `$10`, not `$0A`. An example is in the decimal mode explanation in section 5.3.

The Glow65 CPU behaves like an NMOS 6502 by default. On an NMOS 6502, N, V, and Z in decimal mode may not match the result. An NMOS 6502 also leaves D unchanged when it accepts an interrupt. If a handler uses addition or subtraction, run CLD at the start of the handler.

What you can try in Glow65

After `SED`, write P to `$03F0`. After `CLD`, write it again to `$03F1`.

```
*E 0300 F8 08 68 8D F0 03 D8 08 68 8D F1 03 60
*G 0300
*D 03F0 03F1
03F0: 3C 34 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

D is bit 3. `$3C` (`%00111100`) has D = 1, and `$34` (`%00110100`) has D = 0.

6.1.5 I (Interrupt Disable)

While **I** is 1, the CPU does not accept IRQ. It accepts NMI regardless of I. SEI sets I to 1, and CLI sets it to 0. I is 1 after a reset, and also right after the CPU accepts an interrupt.

6.1.6 B and Bit 5

B (bit 4) is not a flag inside P. B is 1 only in the P that PHP and BRK push onto the stack. In the P pushed by IRQ and NMI, B is 0. A handler can look at B in the pushed P to tell whether it was called by BRK or by IRQ.

(c) 2026 masanorisakai / MIT License

Bit 5 is always 1. PLP and RTI do not take B from the pulled value, and they leave bit 5 as 1. The bottom two rows of Figure 6-1 show the differences in the pushed values.

6.1.7 Flags Tested by Conditional Branches

A conditional branch instruction looks at one flag and decides whether to branch.

Instruction	Flag tested	Branches when
BCC	C	0
BCS	C	1
BEQ	Z	1
BNE	Z	0
BMI	N	1
BPL	N	0
BVC	V	0
BVS	V	1

6.1.8 After a Comparison (CMP)

CMP subtracts a value from A, throws the result away, and changes only the flags. C is 1 when A is greater than or equal to the value, and Z is 1 when they are equal. N is the most significant bit of the result of A minus the value.

CMP results and flags

Results of comparing with different values of M when A = $30

		C	Z	N	Branch to use
A < M	CMP #$40	0	0	1	BCC (C=0)
A = M	CMP #$30	1	1	0	BEQ (Z=1)
A > M	CMP #$20	1	0	0	BCS (C=1), BNE (Z=0)

C is 1 when A is greater than or equal to M. Z is 1 when A equals M. N is the top bit of A – M.
N does not directly show signed order. The comparison treats both values as unsigned.

Figure 6-2 Flags and branches after CMP.

When A is less than the value, C is 0. When they are equal, Z is 1. When A is greater, C is 1 and Z is 0. Use C and Z to compare unsigned numbers. N does not directly tell you the signed order.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

After `LDA #$30`, write P after `CMP #$30`, `CMP #$40`, and `CMP #$20`, in order, starting at `$03F0`.

```
*E 0300 A9 30 C9 30 08 68 8D F0 03 A9 30 C9 40 08 68 8D
*E 0310 F1 03 A9 30 C9 20 08 68 8D F2 03 60
*G 0300
*D 03F0 03F2
03F0: 37 B4 35 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$37` has C and Z set to 1 (equal). `$B4` has C = 0 (less). `$35` has C = 1 and Z = 0 (greater).

6.2 The Stack

6.2.1 Location and SP

The stack is the 256 bytes of memory from `$0100` to `$01FF`. The value put in last is taken out first (last in, first out). SP (the stack pointer) is 8 bits, and the address `$0100` plus SP is the place where the next value is written.

To push, the CPU writes to that place and then subtracts 1 from SP. To pull, it adds 1 to SP and then reads that address. The stack grows from high addresses toward low addresses. The diagram is Figure 5-6 in section 5.6.

The Glow65 CPU sets SP to `$FD` on reset. The monitor sets SP to `$FF` right after reset. A program called with the monitor's `G` starts with SP at `$FD`, after the return address (2 bytes) has been pushed.

6.2.2 What Goes on the Stack

Operation	What is pushed	Instruction that pulls it
PHA	A (1 byte)	PLA
PHP	P (1 byte, with B and bit 5 set to 1)	PLP
JSR	Return address (2 bytes: the address of the last byte of the JSR, high byte first)	RTS (adds 1 and puts it in PC)
BRK, IRQ, NMI	High byte of PC, low byte of PC, P (3 bytes)	RTI

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

Push three values with `PHA` and read SP with `TSX`. Copy the stack page to `$0500`, then pull the three values with `PLA`.

```
*E 0300 A9 11 48 A9 22 48 A9 33 48 BA 8E F0 03 A2 0F BD
*E 0310 F0 01 9D 00 05 CA 10 F7 68 8D F1 03 68 8D F2 03
*E 0320 68 8D F3 03 60
*G 0300
*D 03F0 03F3
03F0: FA 33 22 11 00 00 00 00 24 F4 24 F4 00 00 00 00
*D 0500 050F
0500: 00 00 00 00 00 00 00 00 00 00 00 33 22 11 37 F2
```

`$03F0` holds SP after the pushes: `$FA`. `$03F1` to `$03F3` hold `33`, `22`, `11` in the order they were pulled, the reverse of the order they were pushed. In the copy at `$0500`, `$050B` to `$050D` correspond to `$01FB` to `$01FD`, and `33 22 11` are laid out from the lower address. The following `37 F2` is the return address that the monitor's `G` pushed.

6.2.3 Nested JSR

A subroutine can call another subroutine. Each call pushes a return address, and each RTS pulls the newest one first.

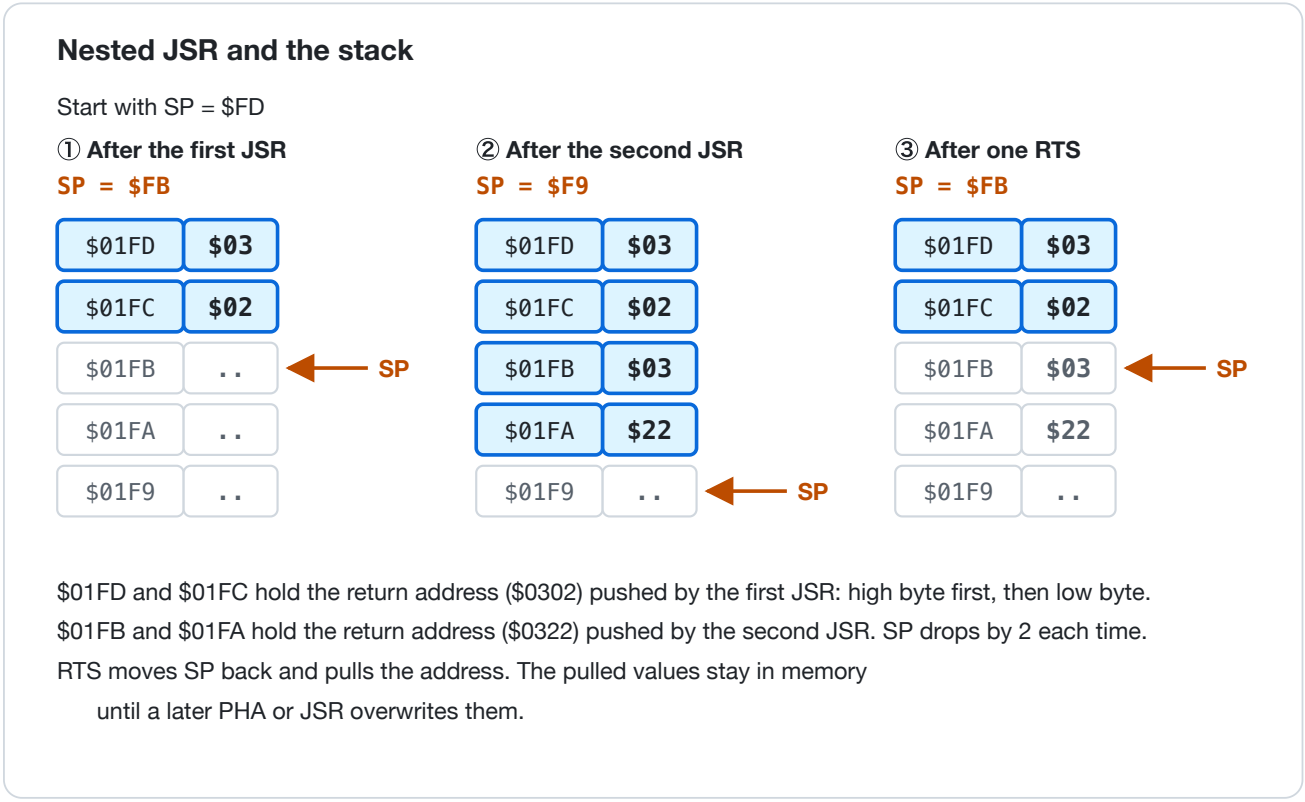


Figure 6-3 The stack with two nested JSR calls.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

The program at `$0300` calls the subroutine at `$0320`, and `$0320` calls `$0330`. `$0330` writes SP to `$03F0` and copies the stack page to `$0500`.

```
*E 0320 20 30 03 60
*E 0330 BA 8E F0 03 A2 0F BD F0 01 9D 00 05 CA 10 F7 60
*E 0300 20 20 03 60
*G 0300
*D 03F0 03F0
03F0: F9 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
*D 0500 050F
0500: 00 00 00 00 00 00 00 00 00 00 22 03 02 03 37 F2
```

`F9` at `$03F0` is SP after the second JSR. `$050A` to `$050D` of the copy hold `22 03 02 03`, which correspond to `$01FA` to `$01FD`. The high byte `03` is at `$01FD` and the low byte `02` at `$01FC`, which is the return address `$0302` of the first JSR. `$01FB` and `$01FA` hold the second one, `$0322`.

6.2.4 SP Wrap-Around

SP is 8 bits, so after `$00` it goes back to `$FF`. The stack goes around inside `$0100` to `$01FF` and never appears in other pages. If you push too much, old values are overwritten.

What you can try in Glow65

Set SP to `$00` and run `PHA`. Write the original SP to `$03F0` and the SP after the push to `$03F1`, then read `$0100`.

```
*E 0300 BA 8E F0 03 A2 00 9A A9 AA 48 BA 8E F1 03 AE F0
*E 0310 03 9A 60
*G 0300
*D 03F0 03F1
03F0: FD FF 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
*D 0100 0100
0100: AA 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

`FD` at `$03F0` is the original SP, and `FF` at `$03F1` is the SP after the push. The `$AA` from A was written to `$0100`.

6.2.5 TSX and TXS

TSX reads SP into X, and TXS writes X into SP. Changing SP moves the stack. If a program running under the monitor changes SP, it must restore the original value before returning to the monitor.

(c) 2026 masanorisakai / MIT License

Keep `$0100` to `$01FF` for the stack only. A variable placed here is destroyed by pushed values. In Glow65, `$0200` to `$027F` is the monitor's line input buffer, and `$0300` to `$03F7` is the area for your own programs.

6.3 Interrupts

6.3.1 The Four Interrupts

There are four causes that make the CPU pause the running program and move elsewhere.

Name	Source	Masked by I	Trigger	Vector	B in the pushed P
NMI	NMI button on the panel	No	Edge (once per change)	`$FFFA` `$FFFB`	0
IRQ	UART receive interrupt	Yes	Level (while the request continues)	`$FFFE` `$FFFF`	0
BRK	BRK instruction	No (always runs)	None	`$FFFE` `$FFFF`	1
Reset	RESET switch	No	None	`$FFFC` `$FFFD`	Nothing is pushed

A vector is a place that holds the destination address in 2 bytes (low byte first). BRK and IRQ share the same vector.

6.3.2 The Acceptance Sequence

An interrupt request is accepted after the running instruction finishes [1]. When the CPU accepts it, it does the following automatically.

1. Push the high byte of PC, the low byte of PC, and P, in that order, onto the stack.
2. Set I to 1.
3. Read the vector address and put it in PC.
4. The handler (the program that deals with the interrupt) starts.

The pushed PC is the address of the next instruction for IRQ and NMI. For BRK, it is the address of the BRK plus 2. The handler ends with RTI. RTI pulls P and PC and returns without adding 1 to PC. The flags are restored as well.

A real 6502 takes 7 cycles to accept an interrupt. Glow65 does not reproduce the extra dummy reads, so only 5 bus accesses appear.

(c) 2026 masanorisakai / MIT License

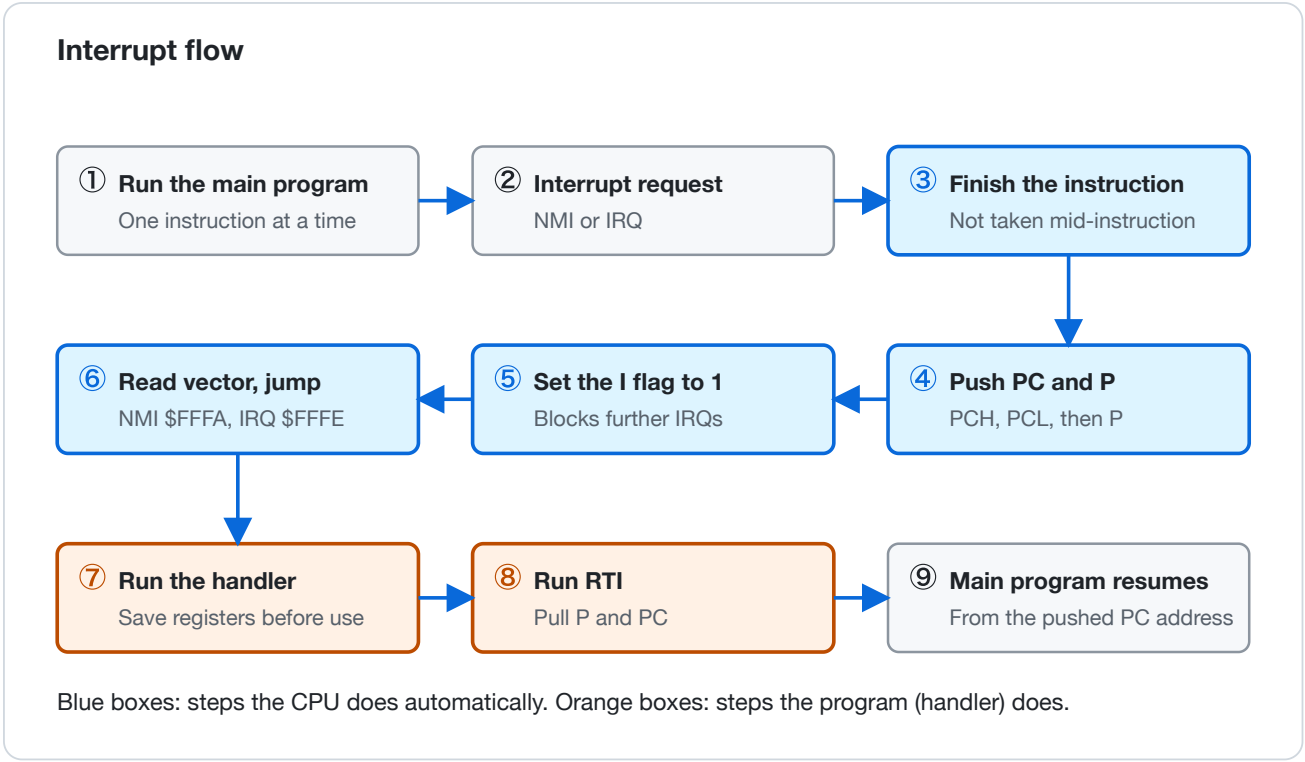


Figure 6-4 The flow of an interrupt.

6.3.3 The Difference between NMI and IRQ

NMI is accepted once each time the request changes from "none" to "present". It is accepted regardless of the I flag.

IRQ is accepted again and again while the request continues, as long as I is 0. If the handler returns with RTI without clearing the cause, the CPU accepts it again at once, because RTI restores P and I goes back to 0.

(c) 2026 masanorisakai / MIT License

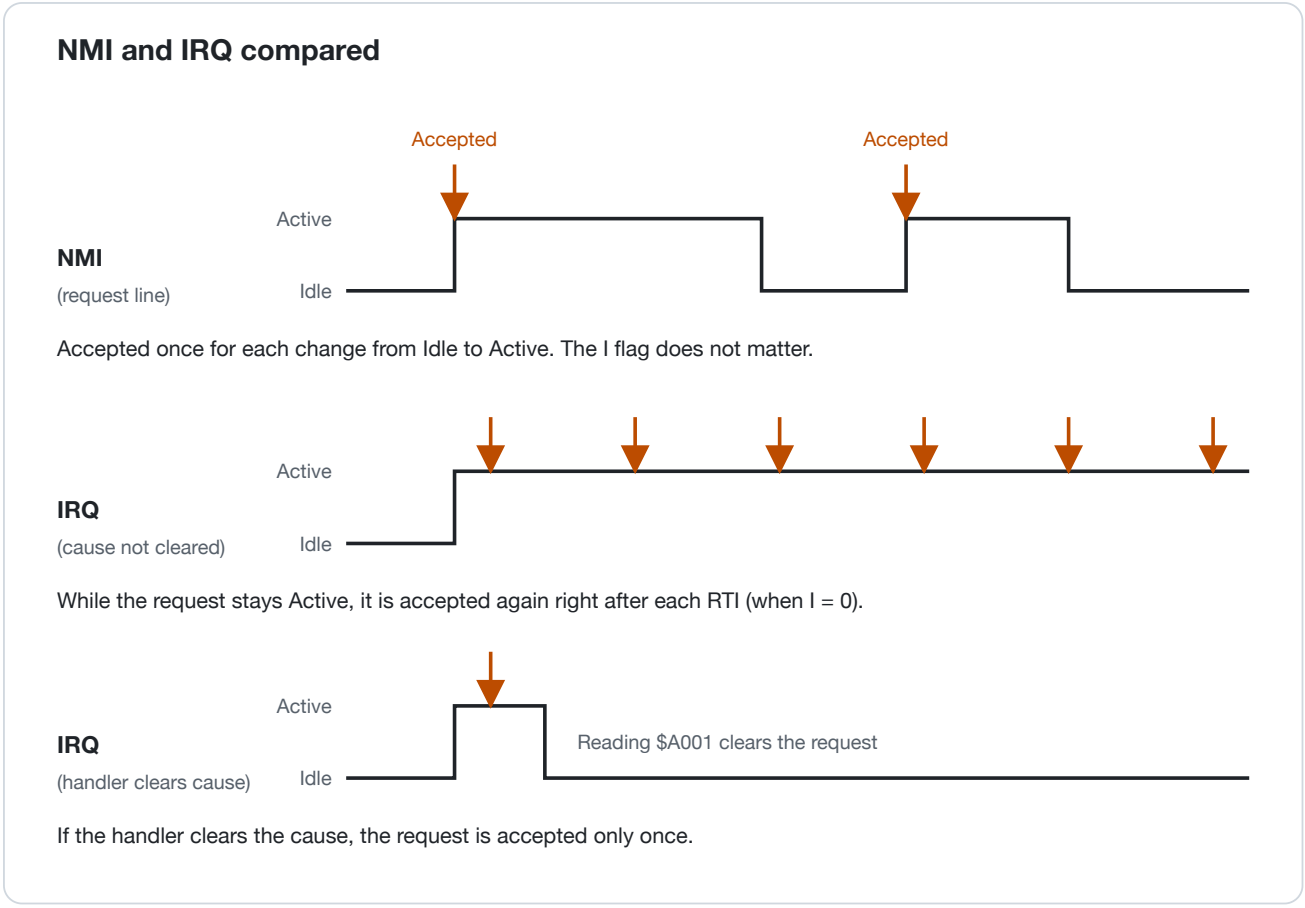


Figure 6-5 The difference between NMI and IRQ.

What you can try in Glow65

Enable the UART receive interrupt and send one character. The handler does not read `$A001`; it increments `$03F2` and returns with RTI. On the fifth call, it reads `$A001` and stops the interrupt.

```
*E 0340 EE F2 03 AD F2 03 C9 05 D0 08 AD 01 A0 A9 00 8D
*E 0350 00 A0 40
*E 0300 78 A9 40 8D FA 03 A9 03 8D FB 03 A9 00 8D F2 03
*E 0310 8D F1 03 A9 80 8D 00 A0 58 AD F2 03 C9 05 D0 F9
*E 0320 78 A9 00 8D 00 A0 60
*G 0300
*D 03F2 03F2
03F2: 05 00 00 00 00 00 24 F4 40 03 00 00 00 00 00 00
```

One received character called the handler 5 times, because the cause (unread received data) remained.

6.3.4 BRK

BRK is an instruction that causes an interrupt from a program. It uses the IRQ vector and is not masked by I. The byte after BRK is the signature byte. The CPU does not use this byte and skips it (the pushed PC is the

(c) 2026 masanorisakai / MIT License

address of the BRK + 2). If the handler reads the pushed PC, the address of the signature byte is one before that PC.

> **What you can try in Glow65**
>
> The BRK handler writes the P pushed on the stack to `$03F0`.
>
> ```
> *E 03FA 40 03
> *E 0340 BA BD 01 01 8D F0 03 40
> *E 0300 A2 00 A0 00 00 99 A0 55 60
> *G 0300
> *D 03F0 03F0
> 03F0: 36 00 00 00 00 00 00 00 24 F4 40 03 00 00 00 00
> ```
>
> `$36` is `%00110110`, and B (bit 4) is 1. Z and I are also 1.

6.3.5 Writing a Handler

The CPU saves only PC and P. If the handler uses A, X, or Y, it must save them at the start and restore them before RTI. Restore them by pulling in the reverse order of the pushes.

```
handler:
        pha             ; save A
        txa
        pha             ; save X
        ...             ; do the work
        pla
        tax             ; restore X
        pla             ; restore A
        rti
```

The next example is a handler called by the UART receive interrupt. It writes the pushed P to `$03F0`, and the I flag inside the handler to `$03F2`. The main program waits until `$03F1` becomes 1.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

When you send one character, the handler is called.

```
*E 0340 48 8A 48 BA BD 03 01 8D F0 03 08 68 29 04 8D F2
*E 0350 03 AD 01 A0 A9 01 8D F1 03 68 AA 68 40
*E 0300 78 A9 40 8D FA 03 A9 03 8D FB 03 A9 00 8D F1 03
*E 0310 A9 80 8D 00 A0 58 AD F1 03 F0 FB 78 A9 00 8D 00
*E 0320 A0 60
*G 0300
*D 03F0 03F2
03F0: 22 01 04 00 00 00 00 00 24 F4 40 03 00 00 00 00
```

`22` at `$03F0` is the pushed P: B (bit 4) is 0, I (bit 2) is 0, bit 5 is 1, and Z is 1. `01` at `$03F1` is the done flag the handler set. `04` at `$03F2` shows that I was 1 inside the handler.

6.3.6 Interrupts in Glow65

The vectors are in ROM at `$FFFA` to `$FFFF` and cannot be rewritten. The monitor sends NMI, IRQ (and BRK) to your handler through two addresses in RAM.

RAM address	Contents
`$03F8` `$03F9`	Address of the NMI handler (low byte first)
`$03FA` `$03FB`	Address of the IRQ and BRK handler (low byte first)

At reset, the monitor points both at a routine that contains only RTI. To use your own handler, write its address there.

The UART interrupt is enabled by bit 7 of the control register (written at `$A000`). While it is enabled and there is unread received data, an IRQ request is raised. Bit 7 of the status register (read at `$A000`) is 1 while an interrupt is being requested. Reading `$A001` lowers the request. NMI is raised with the NMI button on the panel.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

Load `tutorial/irq_echo.bin` at `$0300`, run it, and send `A`, `B`, and Enter.

```
*G 0300
AB
*D 03F0 03F2
03F0: CA E9 0D 00 00 00 00 00 24 F4 3A 03 00 00 00 00
```

The handler, called by the interrupt, echoes the characters. `$03F0` and `$03F1` hold a value the main program counted in the meantime, and it changes with each run. `0D` at `$03F2` is the sign that Enter was received.

(c) 2026 masanorisakai / MIT License

6.3.7 Watching Interrupts on the Panel

What you can try in Glow65

You can follow the acceptance of an interrupt and the return from the handler one step at a time on the panel. Watch NMI with `nmi_count.bin` from tutorial chapter 6.

1. Load `tutorial/nmi_count.bin` and run `G 0300` in the monitor. The program repeats `JMP $030F` at `$030F`.
2. Set **RUN/STOP** to STOP to stop the CPU. Set **STEP MODE** to CYCLE.
3. Press the **NMI** button.

The state right after the press is as follows (Figure 6-6). The NMI LED lights, the CPU stays stopped, PC is `$030F`, and SP is `$FD`. The NMI LED shows the request before it is accepted.

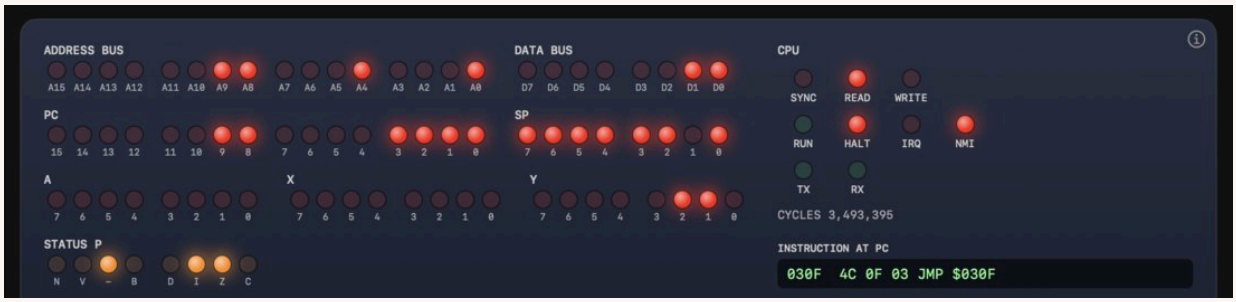


Figure 6-6 Right after the NMI button is pressed

Each press of **STEP** advances the acceptance by one step.

STEP	ADDRESS BUS	DATA BUS	What happened
1st	`$01FD`	`$03`	WRITE lights. The high byte of PC is pushed. The NMI LED goes out
2nd	`$01FC`	`$0F`	The low byte of PC is pushed
3rd	`$01FB`	`$26`	P is pushed
4th	`$FFFA`	`$1E`	READ lights. The low byte of the NMI vector is read
5th	`$FFFB`	`$F4`	The high byte of the NMI vector is read
6th	unchanged	unchanged	PC becomes `$F41E` and SP becomes `$FA`. The acceptance is complete

From the 1st to the 5th step, PC stays at `$030F` and SP stays at `$FD`. This is because, in the middle of an instruction, PC and the registers show their values from before the instruction (section 3.4 in Chapter 3). The 1st STEP raises CYCLES by 7.

(c) 2026 masanorisakai / MIT License



Figure 6-7 The 1st STEP. The high byte of PC has been pushed to `$01FD`

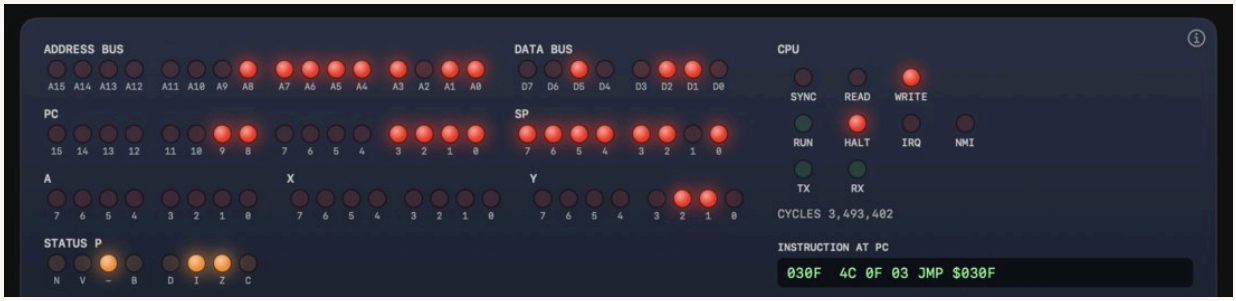


Figure 6-8 The 3rd STEP. P (`$26`) has been pushed to `$01FB`

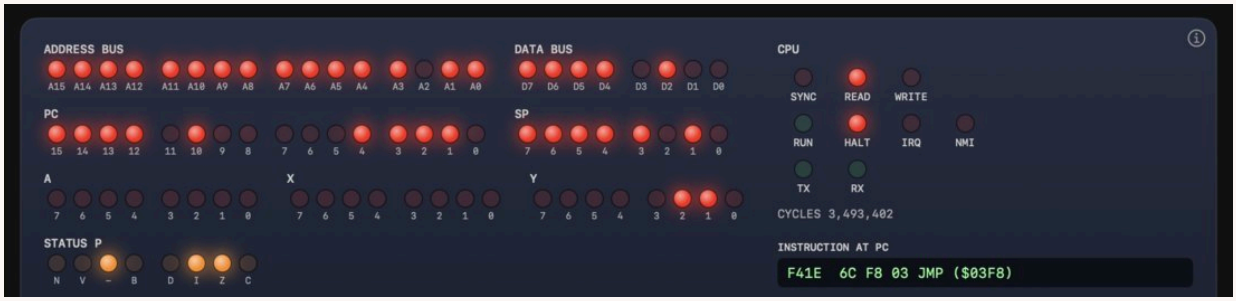


Figure 6-9 The acceptance is complete. PC is `$F41E` (the monitor's `JMP ($03F8)`) and SP is `$FA` (3 bytes pushed)

If you set STEP MODE to INSTR and keep pressing STEP, you can watch the handler run and return. The acceptance finishes in a single STEP.

STEP	What ran	PC	SP
1st	Acceptance of the interrupt	`$F41E`	`$FA`
2nd	`JMP ($03F8)`	`$0312` (the handler's `PHA`)	`$FA`
3rd	`PHA`	`$0313`	`$F9`
11th	`RTI`	`$030F`	`$FD`

`PHA` lowers SP to `$F9` (Figure 6-10), and `RTI` returns PC to `$030F` and SP to `$FD` (Figure 6-11). The 4th to 10th steps are the handler's `INC $03F0`, `LDA $A000`, `AND #$02`, `BEQ`, `LDA #$21`, `STA $A001`, and `PLA`.

(c) 2026 masanorisakai / MIT License



Figure 6-10 After `PHA`. SP becomes `$F9`



Figure 6-11 After `RTI`. PC and SP are back to their old values

Watch IRQ with `tutorial/irq_echo.bin`. Run it with `G 0300`, set RUN/STOP to STOP and STEP MODE to INSTR, and type `A` on the terminal. The UART now holds a received character and the I flag is 0, so the next STEP accepts the interrupt.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

STEP	What ran	PC	IRQ LED
1st	Acceptance of the interrupt	`$F421`	Lit
2nd	`JMP ($03FA)`	`$033A`	Lit
3rd	`PHA`	`$033B`	Lit
4th	`LDA $A001`	`$033E`	Lit. A is `$41 (A)`
5th	`CMP #$0D`	`$0340`	Off

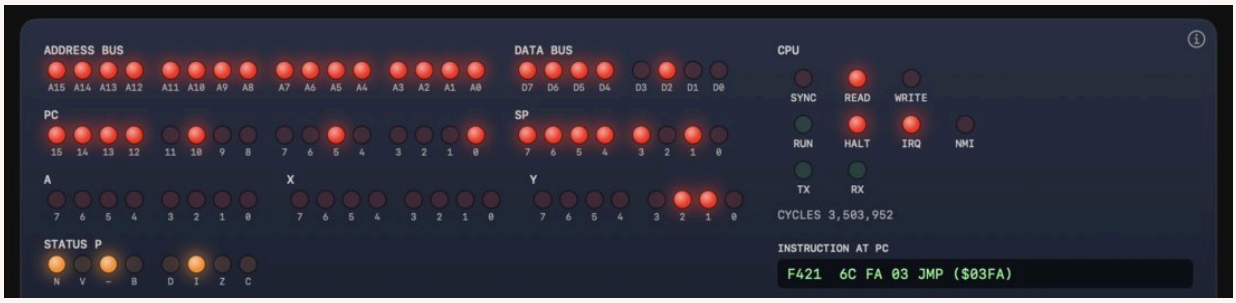


Figure 6-12 Right after IRQ is accepted. PC is `$F421` (the monitor's `JMP ($03FA)`)

Reading the received data with `LDA $A001` lowers the IRQ request. The IRQ LED shows the state of the line before the instruction runs, so it is still lit at the STEP that ran `LDA $A001` (Figure 6-13), and goes out at the next STEP (Figure 6-14).

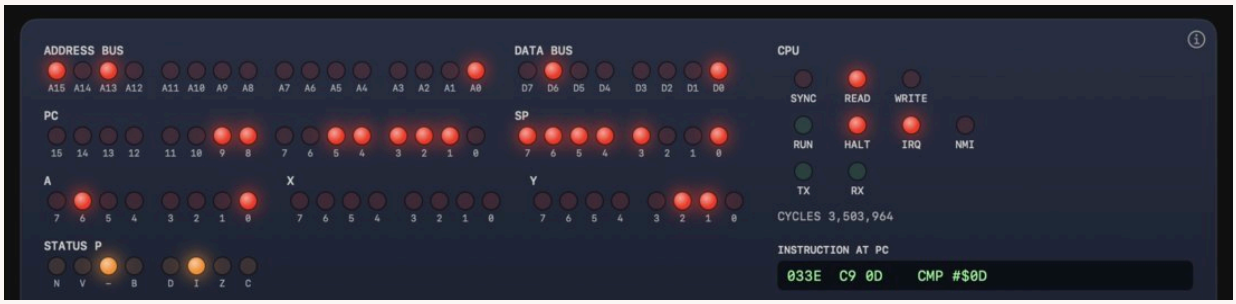


Figure 6-13 After `LDA $A001`. A holds `$41` and the IRQ LED is still lit

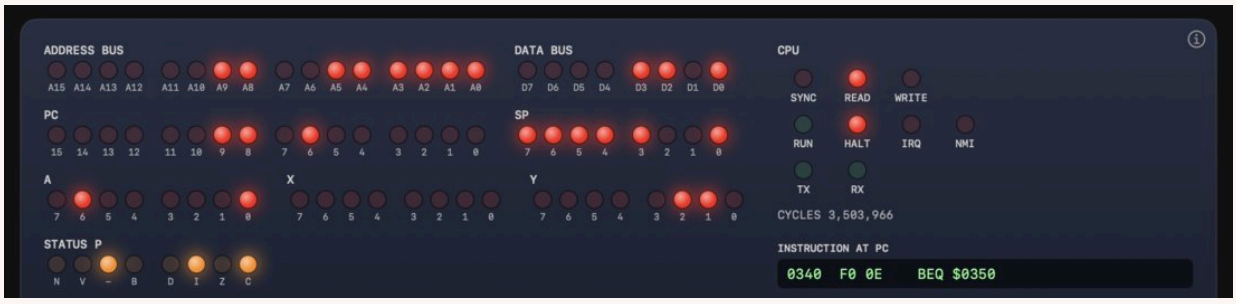


Figure 6-14 After the next STEP. The IRQ LED goes out

The details of the procedure and how to write the programs are in tutorial chapter 6.

(c) 2026 masanorisakai / MIT License

6.4 Reset

A reset throws away the running program and starts again from a fixed state. The Glow65 CPU is in the following state after a reset.

Item	Value after reset
A, X, Y	0
SP	`$FD`
P	`$24` (I is 1, D is 0)
PC	The 2 bytes at `$FFFC` and `$FFFD` (low byte first)

On a real 6502, the internal state of the registers after reset is not defined [1]. The program initializes them first. The Glow65 monitor also runs `SEI`, `CLD`, `LDX #$FF`, and `TXS` at its start, which sets I to 1, D to 0, and SP to `$FF`. It then master-resets the UART and points the NMI and IRQ vectors at a routine that contains only RTI.

The RESET switch keeps the contents of RAM. Turning POWER off and on again clears RAM.

6.5 Practice Problems

The answers and explanations are in the next section.

Problem 1 When P is `$B5`, which flags (N, V, D, I, Z, C) are 1?

Problem 2 After `CLC`, you perform the following additions. Give N, V, and C after each addition. (1) `$20 + $30` (2) `$60 + $60` (3) `$C0 + $C0`

Problem 3 When A is `$40`, give C, Z, and N after (1) `CMP #$41`, (2) `CMP #$40`, and (3) `CMP #$3F`.

Problem 4 With SP at `$FD`, you run `PHA` twice and then `JSR`. What is SP right after entering the subroutine?

Problem 5 You run the `JSR $0340` (`20 40 03`) at `$0300` with SP at `$FD`. Give the values pushed on the stack and the addresses where they are placed.

Problem 6 With SP at `$00`, you run `PHA`. At which address is the value of A written? What is SP after the instruction?

Problem 7 Right before an IRQ is accepted, PC is `$0312`, SP is `$FD`, and P is `$20`. Give SP, the contents of the stack, and P (the value of I) after the acceptance.

Problem 8 You enabled the UART receive interrupt. What happens if the handler returns with RTI without reading `$A001`?

Problem 9 In the BRK handler for a BRK at `$0302`, you check the PC pushed on the stack and find `$0304`. What is the address of the signature byte?

Problem 10 Explain how a handler can tell whether it was entered from BRK or from IRQ.

(c) 2026 masanorisakai / MIT License

6.6 Answers and Explanations

Problem 1 N, I, C (and also bit 5 and B).

- `$B5` is `%10110101`. Bits 7 (N), 5, 4 (B), 2 (I), and 0 (C) are 1.

Problem 2 (1) N = 0, V = 0, C = 0 (2) N = 1, V = 1, C = 0 (3) N = 1, V = 0, C = 1

- (1) The result is `$50`. (2) The result is `$C0`. 96 + 96 = 192 is out of the signed range, so V is 1. (3) The result is `$80` (-64 + -64 = -128 is in range, so V is 0). The sum exceeds `$FF`, so C is 1.

Problem 3 (1) C = 0, Z = 0, N = 1 (2) C = 1, Z = 1, N = 0 (3) C = 1, Z = 0, N = 0

- (1) `$40 - $41` is negative. A is smaller. (2) They are equal. (3) `$40 - $3F` is 1. A is larger.

Problem 4 `$F9`

- Two `PHA` take SP from `$FD` to `$FB`. JSR pushes 2 bytes, so SP becomes `$F9`.

Problem 5 `$03` (high byte) at `$01FD`, and `$02` (low byte) at `$01FC`.

- The value pushed is the address of the last byte of the JSR, `$0302`. The high byte goes first, to `$0100 + $FD = $01FD`, and then the low byte goes to `$01FC`.

Problem 6 It is written at `$0100`. SP becomes `$FF`.

- The CPU writes at `$0100 + $00 = $0100`, and subtracting 1 from SP wraps from `$00` to `$FF`.

Problem 7 SP is `$FA`. `$01FD` holds `$03`, `$01FC` holds `$12`, and `$01FB` holds `$20`. I is 1 (P is `$24`).

- The CPU pushes the high byte of PC `$03`, the low byte `$12`, and then P. In the pushed P, B is 0 and bit 5 is 1, so it stays `$20`. After the acceptance I becomes 1, so P is `$24`.

Problem 8 Even when the handler returns, the request continues. When RTI restores P and I becomes 0, the CPU accepts the IRQ again at once. This repeats.

- IRQ works as a level until the cause disappears. Reading `$A001` clears the cause.

Problem 9 `$0303`

- The BRK is at `$0302`, and the signature byte is the next address, `$0303`. The pushed PC is `$0304`, the address after the signature byte, so the address of the signature byte is one before PC.

Problem 10 Check B (bit 4) in the P pushed on the stack. If it is 1, the cause is BRK. If it is 0, the cause is IRQ.

- The P pushed by BRK has B = 1, and the P pushed by IRQ has B = 0. At the start of the handler, read the pushed P with `TSX` and `LDA $0101,X`, and test it with `AND #$10`.

(c) 2026 masanorisakai / MIT License

Sources

1. MOS Technology, "MCS6500 Microcomputer Family Hardware Manual", January 1976 edition: https://www.commodore.ca/wp-content/uploads/2018/11/kim1_hardware.txt

(c) 2026 masanorisakai / MIT License

Chapter 7 The Memory Map and I/O

The CPU can read and write only the byte that an address selects. RAM, ROM, and the devices connected to the keyboard and screen all sit behind addresses. This chapter explains the memory map, which assigns addresses to parts, and memory-mapped I/O, which operates devices through addresses. The examples are the Glow65 UART and the monitor routines.

7.1 The Memory Map

7.1.1 Assigning Addresses

For every instruction, the CPU puts a 16-bit address on the address bus (Chapter 3). RAM, ROM, and I/O parts are connected behind that address. The **memory map** is the plan that decides which part gets which addresses in the 64KB address space. The circuit that selects a part from an address is called **address decoding**.

The number of addresses given to a part is the size of that part. Some addresses are given to nothing. The differences between RAM, ROM, and I/O are explained in section 2.8 of Chapter 2.

7.1.2 The Glow65 Memory Map

Address	Part	Contents
`$0000–$7FFF`	RAM	32KB
`$8000–$9FFF`	ROM	GAME65G (about 2.7KB used)
`$A000–$A0FF`	UART	Even addresses are status and control, odd addresses are data
`$A100–$BFFF`	None	Nothing is connected
`$C000–$EFFF`	ROM	Microsoft BASIC (about 8.6KB used)
`$F000–$FFFF`	ROM	Monitor, jump table, vectors

(c) 2026 masanorisakai / MIT License

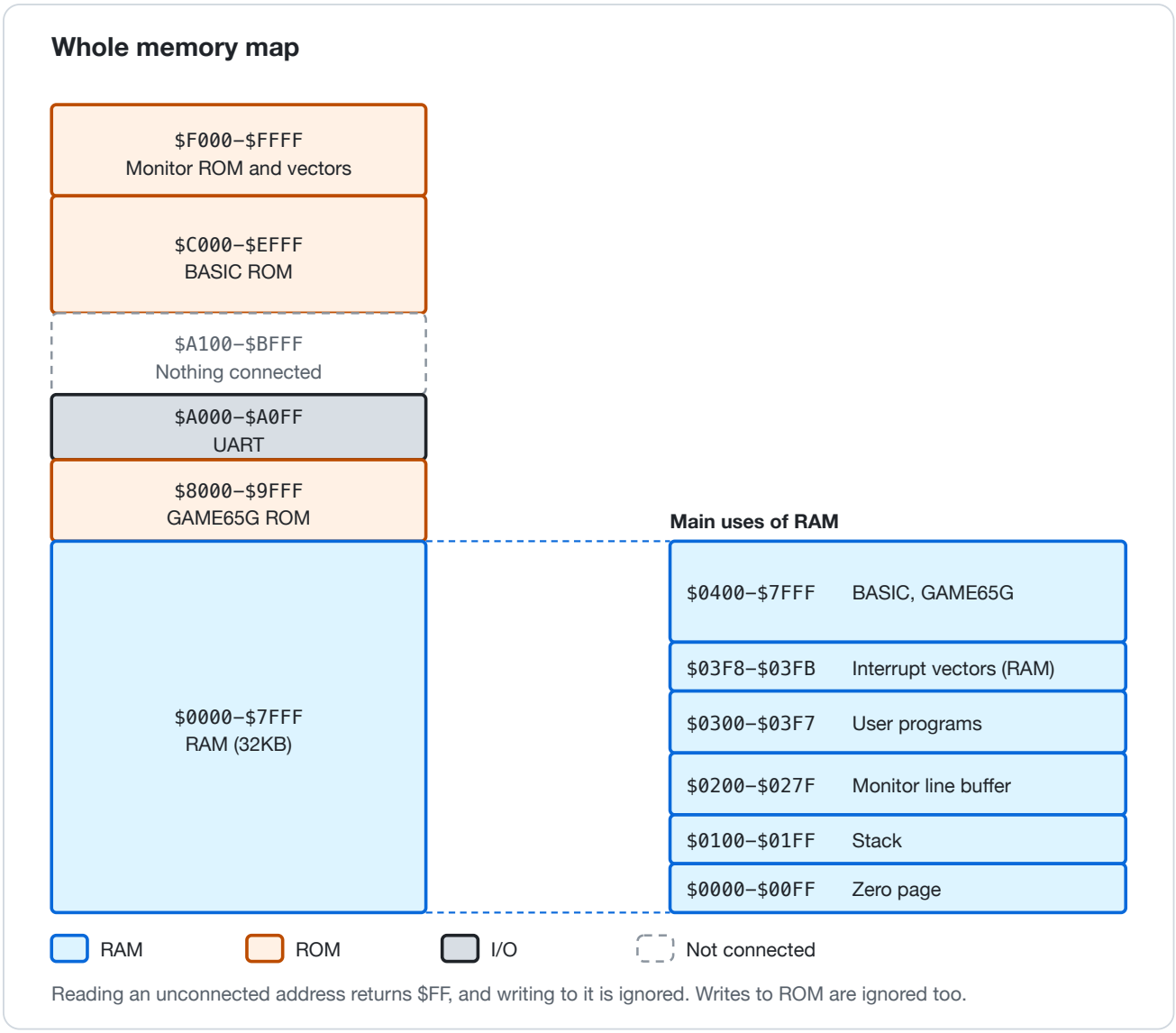


Figure 7-1 The Glow65 memory map.

Reading an address with nothing connected returns `$FF`. A write is ignored. A write to ROM is also ignored.

When the power is turned on, all of RAM becomes 0. The RESET switch keeps the contents of RAM.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

Read the end of RAM right after power-on, the unconnected `$A100` and `$B000`, and the start of ROM at `$F000`. Try writing to ROM with `E`.

```
*D 7FF0 7FFF
7FF0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
*D A100 A10F
A100: FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
*D B000 B00F
B000: FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
*E F000 00
*D F000 F00F
F000: 4C 0F F0 4C 28 F0 4C 37 F0 4C 4B F1 4C 1A F0 AD
```

`$7FF0` is filled with 0. `$A100` and `$B000` are all `FF`. Even after `E F000 00`, `$F000` is still `4C` (the `JMP` instruction).

The UART takes 256 addresses, `$A000` to `$A0FF`. Only the lowest bit of the address chooses between status and control (even) and data (odd). As a result, the two registers appear repeated 128 times each (**mirrors**). This is simple decoding that selects a part by looking at only a rough part of the address.

What you can try in Glow65

Read the status at `$A000` and at `$A0FE` (a mirror). Writing `M` to the odd address `$A003` sends it, the same as writing to `$A001`.

```
*E 0300 AD 00 A0 8D F0 03 AD FE A0 8D F1 03 A9 4D 8D 03
*E 0310 A0 A9 0D 8D 03 A0 A9 0A 8D 03 A0 60
*G 0300
M
*D 03F0 03F1
03F0: 02 02 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

Both statuses are `02`. `M` appears on the terminal.

7.1.3 How RAM Is Used

The RAM from `$0000` to `$7FFF` is divided up by purpose.

Address	Use
`$0000–$00FF`	Zero page. Can be addressed with a 1-byte address. The monitor uses `$F0–$FF`. BASIC keeps the USR vector (`$0A–$0C`) and the vector for the integer and floating-point conversion routines (`$06–$09`), among other things. GAME65G uses `$00–$2C`, and `$80–$B3` for the variables A-Z

(c) 2026 masanorisakai / MIT License

Address	Use
`$0100–$01FF`	The stack (Chapter 6)
`$0200–$027F`	The monitor's line input buffer
`$0300–$03F7`	248 bytes for user programs
`$03F8–$03F9`	Address of the NMI handler (low byte first)
`$03FA–$03FB`	Address of the IRQ and BRK handler (low byte first)
`$0400–$7FFF`	BASIC programs and variables. GAME65G uses `$0400–$05FF` for its input buffers, second stack, and type-ahead buffer, and stores its program from `$0600`

How BASIC uses the zero page is described in tutorial chapter 5.

7.2 Memory-Mapped I/O

I/O is operated the same way as memory: by reading and writing at an address. This method is called **memory-mapped I/O**. No special instructions are needed. `LDA` and `STA` work as they are.

However, I/O addresses behave differently from ordinary memory.

Difference	Example in the UART
Reading changes the state	Reading `$A001` removes the received byte from the queue
Reading and writing reach different registers	`$A000` is status when read and control when written. `$A001` is received data when read and transmit data when written
A written value may not read back	The value written to `$A001` does not come back when read
A value changes regardless of the program	When a character arrives, the status register changes

> **What you can try in Glow65**
>
> Read 16 bytes from `$A000` with `D`.
>
> ```
> *D A000 A00F
> A000: 02 00 02 00 02 00 02 00 02 00 02 00 02 00 02 00
> ```
>
> Even addresses are the status, `02` (only the transmit-ready bit is 1). Odd addresses are the data, `00` (no received data).

`D` reads every address in the range. Reading an odd address takes out the received byte, if there is one, and removes it from the UART. To observe I/O, have a program read the status and the data, write the results to memory, and then look at them with `D`.

(c) 2026 masanorisakai / MIT License

7.3 UART

The UART is a device that handles serial communication with the terminal. It sends and receives one byte at a time. The Glow65 UART is a minimal design similar to the MC6850, with two registers, `$A000` and `$A001`.

Address	Read	Write
`$A000`	Status register	Control register
`$A001`	Received data	Transmit data

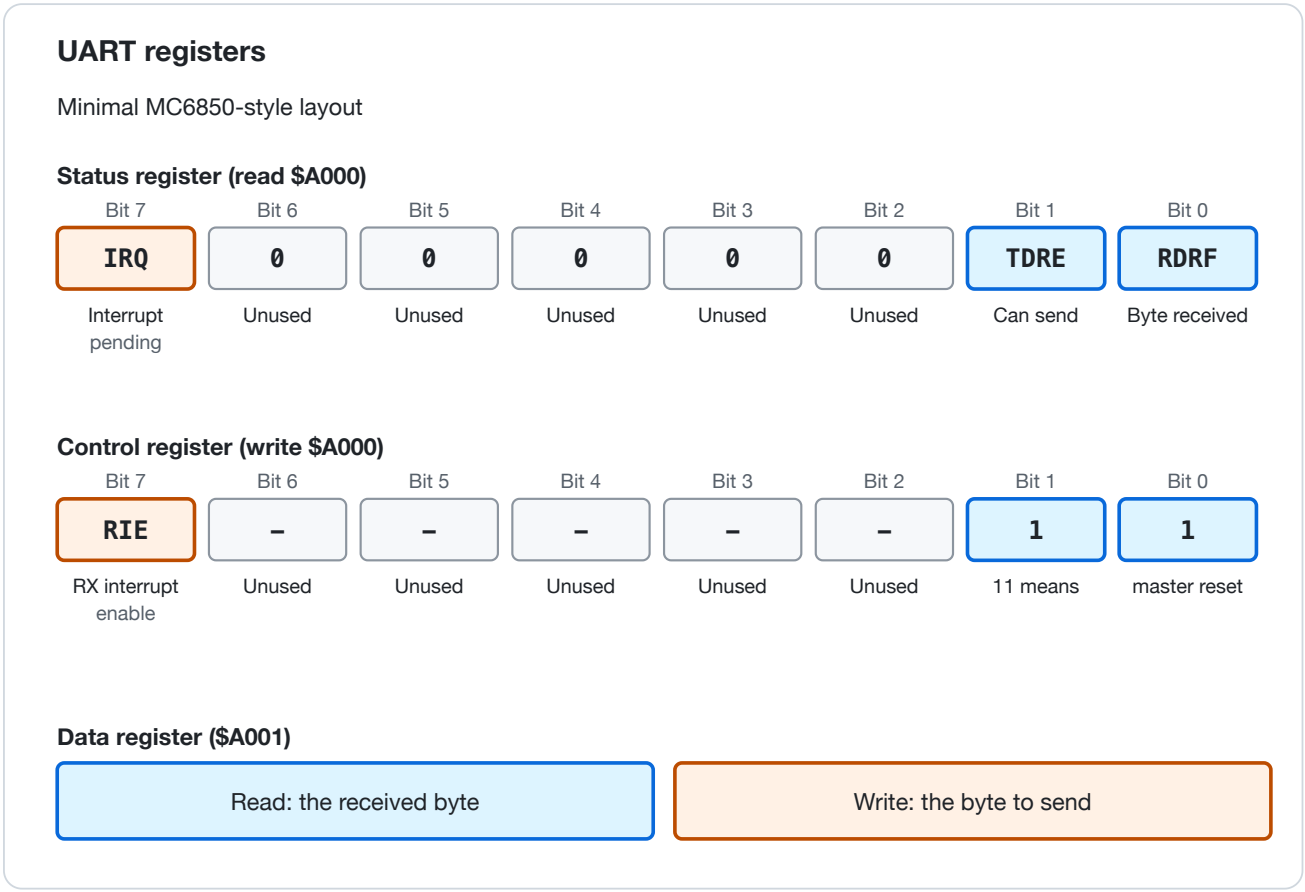


Figure 7-2 The UART registers.

Register	Bit	Name	Meaning
Status	0	RDRF	When 1, there is received data
Status	1	TDRE	When 1, the UART can transmit
Status	7	IRQ	When 1, an interrupt is being requested
Control	7	RIE	When 1, raises IRQ while there is received data
Control	0 and 1	-	When both are 1 (`%11`), master reset. Discards the received data and also turns off the interrupt enable

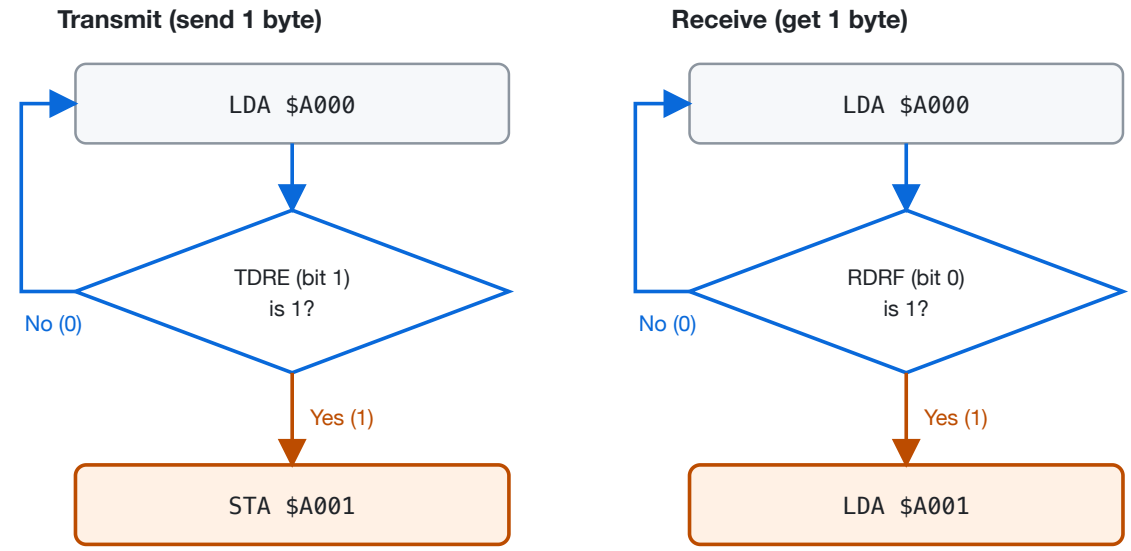
The Glow65 UART never makes transmission wait. TDRE is always 1. If you write your program to check TDRE before writing, it works the same way with a UART that transmits slowly.

(c) 2026 masanorisakai / MIT License

7.3.1 Polling

A program that reads the status register again and again, waiting for a bit to become 1, is using a method called **polling**.

Polling flow



Pick out the bit in the status register with AND, and read again while it is 0.
When it is 1, access the data register.
Transmit: write when TDRE is 1. Receive: read when RDRF is 1.
Reading removes the received byte from the queue.

Figure 7-3 The flow of polling.

Transmit and receive are written as follows. The monitor's `CHROUT` and `CHRIN` follow the same steps.

```
putc:   pha
wait:   lda $A000
        and #$02        ; TDRE
        beq wait
        pla
        sta $A001       ; transmit
        rts

getc:   lda $A000
        and #$01        ; RDRF
        beq getc
        lda $A001       ; receive
        rts
```

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

The program waits for a received character. Send `Q` while it runs. It reads `$A000`, `$A001`, and `$A000` in that order and writes the values from `$03F0`.

```
*E 0300 AD 00 A0 29 01 F0 F9 AD 00 A0 8D F0 03 AD 01 A0
*E 0310 8D F1 03 AD 00 A0 8D F2 03 60
*G 0300
*D 03F0 03F2
03F0: 03 51 02 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

The first status is `03` (RDRF and TDRE), the data is `51` (`Q`), and the status after reading the data is `02`. Reading removes the received data, and RDRF goes back to 0.

What you can try in Glow65

Set RIE (bit 7) of the control register to 1, and do the same thing. The CPU's I stays 1, so no interrupt occurs.

```
*E 0300 A9 80 8D 00 A0 AD 00 A0 29 01 F0 F9 AD 00 A0 8D
*E 0310 F0 03 AD 01 A0 8D F1 03 AD 00 A0 8D F2 03 A9 03
*E 0320 8D 00 A0 60
*G 0300
*D 03F0 03F2
03F0: 83 51 02 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

The status is `83`. IRQ (bit 7) is 1, which means an interrupt is being requested. After the data is read, the status goes back to `02`.

What you can try in Glow65

Send two characters (`X` and `Y`). When the UART is ready to receive, write `$03` to the control register and then read the status.

```
*E 0300 AD 00 A0 29 01 F0 F9 A9 03 8D 00 A0 AD 00 A0 8D
*E 0310 F0 03 60
*G 0300
*D 03F0 03F0
03F0: 02 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

The status is `02`. The master reset discarded the received data.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

You can follow how the status changes during polling on the panel. Use `echo.bin` from tutorial chapter 4.

1. Load `tutorial/echo.bin` and run `G 0300` in the monitor. The program repeats `LDA $A000`, `AND #$01`, and `BEQ` while it waits for input.
2. Set **RUN/STOP** to STOP to stop the CPU. Set **STEP MODE** to INSTR and press **STEP**.
3. At the STEP that ran `LDA $A000`, ADDRESS BUS is `$A000`, DATA BUS is `$02` (only D1 is lit), and A is also `$02`. Only transmit-ready (TDRE) is 1, and there is no received data (Figure 7-4).
4. While stopped, type `h` on the terminal. At the next STEP that runs `LDA $A000`, DATA BUS is `$03` (D1 and D0), and A is also `$03`. Received data (RDRF) is now 1 (Figure 7-5).
5. Step through `AND #$01` and `BEQ`. It does not branch and goes on to `LDA $A001`. At the STEP that runs `LDA $A001`, ADDRESS BUS is `$A001`, DATA BUS is `$68` (`h`), and A is also `$68` (Figure 7-6).
6. At the next `LDA $A000` in the wait for transmit, DATA BUS is back to `$02`. Reading the data made RDRF 0 (Figure 7-7).



Figure 7-4 With no received data. Reading `$A000` gives `$02`



Figure 7-5 After typing `h`. Reading `$A000` gives `$03`



Figure 7-6 Reading `$A001` puts the received `h` (`$68`) in A

(c) 2026 masanorisakai / MIT License



Figure 7-7 After the data is read. `$A000` is back to `$02`

Receiving with interrupts is covered in 6.3.6 and in tutorial chapter 6. Transmitting and receiving by polling are covered in tutorial chapter 3 and chapter 4.

7.4 Monitor Routines

The monitor keeps a table of five `JMP` instructions in ROM, starting at `$F000` (the jump table). Each one jumps to a routine. Even if the routine bodies move, these addresses do not change. Call them with `JSR`.

Address	Name	Input	Output	Action
`$F000`	CHRIN	None	A = character	Waits until a character arrives and reads one character
`$F003`	CHROUT	A = character	None (A changes)	Drops the most significant bit and sends one character. Waits until it can transmit
`$F006`	ISCNTC	None	C = 1 if Ctrl-C, C = 0 otherwise (A changes)	Does not wait. If a character is there, reads one and checks whether it is Ctrl-C (`$03`)
`$F009`	Warm start	None	None	Returns to the monitor. Jump with `JMP`, not `JSR`
`$F00C`	GETKEY	None	A = character, or A = 0 if none has arrived	Reads one character without waiting

X and Y are not changed by any of the routines.

What you can try in Glow65

Send `$48` (`H`) and `$C9` with `CHROUT`, then CR (`$0D`) and LF (`$0A`).

```
*E 0300 A9 48 20 03 F0 A9 C9 20 03 F0 A9 0D 20 03 F0 A9
*E 0310 0A 20 03 F0 60
*G 0300
HI
```

`$C9` is sent as `$49` (`I`), with its most significant bit dropped.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

Wait for one character with `CHRIN` and write it to `$03F0`. Send `z` while it runs.

```
*E 0300 20 00 F0 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 7A 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

You can read `$7A`, the ASCII code of `z`.

What you can try in Glow65

Call `GETKEY` once while no character has arrived and write the result to `$03F0`. Then call it repeatedly until a character arrives. Send `k` while it runs.

```
*E 0300 20 0C F0 8D F0 03 20 0C F0 F0 FB 8D F1 03 60
*G 0300
*D 03F0 03F1
03F0: 00 6B 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$03F0` is `00`. The routine returned without waiting, so A was 0. `6B` at `$03F1` is `k`.

What you can try in Glow65

Call `ISCNTC` repeatedly until Ctrl-C arrives, and send `!` when it does. While it runs, send `a` and then Ctrl-C (`$03`).

```
*E 0300 20 06 F0 90 FB A9 21 20 03 F0 A9 0D 20 03 F0 A9
*E 0310 0A 20 03 F0 60
*G 0300
!
```

`a` is read and discarded, the routine returns with C = 0, and the loop continues. Ctrl-C gives C = 1, and `!` appears.

(c) 2026 masanorisakai / MIT License

What you can try in Glow65

Write `$77` to `$03F0` and return to the monitor with `JMP $F009`.

```
*E 0300 A9 77 8D F0 03 4C 09 F0
*G 0300
*D 03F0 03F0
03F0: 77 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

You get back to the monitor prompt without using `RTS`.

The monitor commands are as follows.

Command	Action
`D aaaa [bbbb]`	Memory dump (128 bytes if the end address is omitted)
`E aaaa bb bb ...`	Write bytes
`L aaaa [bbbb]`	Disassemble (16 instructions if the end address is omitted)
`G aaaa`	Call the routine at `aaaa` (returns to the monitor with RTS)
`B`	Start BASIC
`T`	Start GAME65G
`?`	Help

7.5 The Vector Area

The last 6 bytes of ROM (`$FFFA` to `$FFFF`) are the vectors for NMI, reset, and IRQ and BRK. How they work is explained in section 3.7 of Chapter 3 and in section 6.3 of Chapter 6.

7.6 Practice Problems

The answers and explanations are in the next section.

Problem 1 What part is read, and what do you get, when you read each of these addresses? (1) `$0300` (2) `$A0F5` (3) `$B000` (4) `$F000`

Problem 2 Which UART registers are `$A0A0` and `$A0A1`?

Problem 3 What happens when you run `STA $F000`?

Problem 4 You read the status register and got `$03`. What does it tell you?

Problem 5 You read the status register and got `$83`. What does it tell you?

Problem 6 Write a routine that uses RDRF in the status register to read one byte into A by receive polling.

(c) 2026 masanorisakai / MIT License

Problem 7 What appears on the terminal after `LDA #$C1` and `JSR $F003`?

Problem 8 One byte of received data, `Q`, has arrived. The program reads `$A000`, `$A001`, and `$A000` in that order. What are the three values it read?

Problem 9 You call `JSR $F006` while `a` has arrived. What are C and `a` when it returns?

7.7 Answers and Explanations

Problem 1 (1) RAM. The value you wrote can be read back (`$00` right after power-on). (2) The UART data register. `$00` if there is no received data. (3) Nothing is connected. `$FF`. (4) The monitor ROM. The start of the jump table, `$4C` (`JMP`).

- (1) `$0300` is inside `$0000–$7FFF`. (2) `$A0F5` is inside `$A000–$A0FF`, and it is odd, so it is data. (3) `$B000` is inside `$A100–$BFFF`, which is unconnected. (4) `$F000` is ROM, and `D` shows `4C`.

Problem 2 `$A0A0` is status when read and control when written. `$A0A1` is received data when read and transmit data when written.

- The UART appears as mirrors in `$A000–$A0FF`. Even addresses are status and control, and odd addresses are data.

Problem 3 Nothing happens. ROM does not change, and the program goes on to the next instruction.

- Writes to ROM are ignored.

Problem 4 There is received data (RDRF = 1), and the UART can also transmit (TDRE = 1). No interrupt is requested.

- `$03` is `%00000011`. Bit 0 and bit 1 are 1. Bit 7 (IRQ) is 0.

Problem 5 There is received data, the UART can transmit, and an interrupt is being requested.

- `$83` is `%10000011`. Bit 7 (IRQ), bit 1 (TDRE), and bit 0 (RDRF) are 1. IRQ becomes 1 when RIE in the control register is 1 and there is unread received data.

Problem 6

```
getc:   lda $A000
        and #$01        ; RDRF
        beq getc        ; repeat while 0
        lda $A001       ; when 1, read the data
        rts
```

- Read the status and pick out bit 0 with AND. Repeat while it is 0, and read `$A001` when it becomes 1.

Problem 7 `A`

- `CHROUT` drops the most significant bit. `$C1` becomes `$41`, the ASCII code of `A`.

(c) 2026 masanorisakai / MIT License

Problem 8 `$03`, `$51`, `$02`

- The first status has RDRF and TDRE, so it is `$03`. Reading `$A001` takes out `Q` (`$51`) and removes the received data. In the second status, RDRF is back to 0, so it is `$02`.

Problem 9 C is 0, and `a` is discarded.

- `ISCNTC` reads one character if there is one. If it is not `$03`, it returns with C = 0. The character it read is not returned.

(c) 2026 masanorisakai / MIT License

Chapter 8 Glossary and Index

This chapter gives the definitions of the terms in Chapters 1 to 7, and an index to look up instructions, addressing modes, flags, addresses, and figures.

Section	Contents
8.1 Glossary	Definitions of terms in alphabetical order. 72 terms
8.2 Index	Instructions, addressing modes, flags and registers, monitor commands and routines, important addresses, figures
8.3 Correspondence with the Tutorial	A table matching the chapters of the manual to the chapters of the tutorial

8.3 Correspondence with the Tutorial

The manual is for reading and understanding, and the tutorial is for hands-on work. After you read a chapter of the manual, try the matching chapter of the tutorial, and you will remember the content better.

Tutorial	Contents	Matching manual chapters
1. LED counter	Load a 5-line program and follow the bus accesses	Chapter 2 Memory and Addresses, Chapter 3 Fetch and Execute, Chapter 3 Instruction Format
2. Addition and the carry flag	16-bit addition and C	Chapter 2 Range of Numbers, Chapter 5 Arithmetic and Comparison, Chapter 6 C (Carry)
3. Talking to the UART	Send characters with `CHROUT`	Chapter 7 UART, Chapter 7 Monitor Routines
4. Reading the keyboard	An echo program that receives by polling	Chapter 7 Polling, Chapter 7 Monitor Routines
5. USR	Call machine code from BASIC	Chapter 7 How RAM Is Used, Chapter 5 JSR and RTS
6. Interrupts	NMI and IRQ handlers	Chapter 6 Interrupts, Chapter 6 The Stack
7. Switching to the 65C02	The same program for the 6502 and the 65C02, and undefined opcodes	Chapter 4 Modes Added by the 65C02, Chapter 5 The 65C02

The addressing modes (Chapter 4) and the instruction reference (Chapter 5) work as a dictionary when you read the tutorial programs.

(c) 2026 masanorisakai / MIT License

8.1 Glossary

The terms from Chapters 1 to 7, in alphabetical order. The link after each definition goes to the section where the term is first explained.

A

- **Accumulator**: The A register. It is the center of calculation, and the target of additions and comparisons. (3.2 Registers)
- **Address**: A number given to each byte of memory. A 6502 address has 16 bits, so there are 65,536 of them, from `$0000` to `$FFFF`. (2.7 Memory and Addresses)
- **Address bus**: The 16 lines on which the CPU puts an address, A0 to A15. (3.3 Buses)
- **Address decoding**: The circuit (mechanism) that chooses which part, RAM, ROM, or I/O, to use for an address. (7.1.1 Assigning Addresses)
- **Address space**: The whole range of addresses the CPU can specify. For the 6502, it is 64KB. (2.7 Memory and Addresses)
- **Addressing mode**: The method that decides which value in memory the operand of an instruction refers to. The 6502 has 13 modes. (4.1 What Is an Addressing Mode)
- **ASCII** (American Standard Code for Information Interchange): A character code that assigns numbers to letters, digits, and symbols. `A` is `$41`, `0` is `$30`, and CR is `$0D`. (2.6 Character Codes (ASCII))
- **Assembler**: A program that turns a program written in assembly language into machine code bytes. It also converts the label of a branch target into a distance. (Instruction Syntax)
- **Assembly language**: A way of writing instructions as text, using a mnemonic and an operand, such as `LDA #$10`. (Instruction Syntax)

B

- **BCD** (Binary Coded Decimal): A format that treats each 4 bits as a digit from 0 to 9. ADC and SBC use it in decimal mode. (5.3 Arithmetic and Comparison)
- **Binary**: A way of writing numbers with only 0 and 1. Each digit is worth twice the digit to its right. (2.2 Binary)
- **Bit**: The smallest unit, holding either 0 or 1. (2.1 Bits and Bytes)
- **BRK** (Break): An instruction that causes an interrupt from a program. It uses the IRQ vector and is not masked by I. B is 1 in the P it pushes. (6.3.4 BRK)
- **Bus**: A bundle of lines that connects the CPU and memory. There is an address bus and a data bus. (3.3 Buses)
- **Byte**: A unit of 8 bits grouped together. The 6502 handles data one byte at a time. (2.1 Bits and Bytes)

C

- **Carry**: The C flag. It holds the digit that did not fit in 8 bits. In ADC it becomes 1 when the result exceeds `$FF`, and in SBC it becomes 1 when no borrow is needed. (6.1.2 C (Carry))

(c) 2026 masanorisakai / MIT License

- **Clock**: A signal at a fixed interval that drives the CPU forward. One period of the clock is one cycle. (3.4 Fetch and Execute)
- **Conditional branch**: An instruction that looks at a flag and jumps only when the condition holds. The instructions are BCC, BCS, BEQ, BNE, BMI, BPL, BVC, and BVS. (6.1.7 Flags Tested by Conditional Branches)
- **CPU** (Central Processing Unit): The part that takes instructions from memory one at a time and runs them. (3.1 The CPU and Memory)
- **Cycle**: One period of the clock. In each cycle the CPU makes one access to the bus. (3.4 Fetch and Execute)

D

- **Data bus**: The 8 lines that carry the byte being read or written, D0 to D7. (3.3 Buses)
- **Decimal mode**: The mode in which ADC and SBC calculate in BCD when the D flag is 1. `$09 + $01` becomes `$10`. (1.5 The Family Computer, 6.1.4 D (Decimal Mode))
- **Disassemble**: To read the bytes in memory as instructions and turn them back into written form. The monitor's `L` command does this. (3.5 Instruction Format)
- **Dummy read**: An extra read that a real 6502 performs and whose result is not used. Glow65 does not reproduce it. (Cycles and Bus Accesses in Glow65)

E

- **Edge**: The moment a signal changes from "none" to "present". NMI is accepted once for each edge. (6.3.3 The Difference between NMI and IRQ)
- **Effective address**: The address in memory that an instruction actually reads or writes. How it is worked out differs for each addressing mode. (4.1 What Is an Addressing Mode)

F

- **Fetch**: Taking the bytes of an instruction from the address in PC. (3.4 Fetch and Execute)
- **Flag**: One bit of the P register. It shows the result of the last instruction. (3.2 Registers)

H

- **Handler**: The program that deals with an interrupt. It ends with RTI. (6.3.2 The Acceptance Sequence)
- **Hexadecimal**: A way of writing numbers in which 4 bits make one digit (0 to F). On the 6502 it is written with a leading `$`. (2.3 Hexadecimal)

I

- **I/O** (Input/Output): The window to outside devices. A device is assigned to particular addresses in memory, and you talk to it by reading and writing those addresses. This is also called memory-mapped I/O. (2.8 ROM, RAM, and I/O)

(c) 2026 masanorisakai / MIT License

- **Interrupt**: Pausing the running program to move to other work. The causes are NMI, IRQ, and BRK. (6.3 Interrupts)
- **IRQ** (Interrupt Request): An interrupt raised by a device such as the UART. It is masked by the I flag. It is accepted again and again while the request continues (it works as a level). (6.3.1 The Four Interrupts)

L

- **Least significant bit** (LSB): Bit 0, the rightmost of the 8 bits. (2.1 Bits and Bytes)
- **Level**: A state that lasts while the signal is "present". IRQ is accepted again and again while the request continues. (6.3.3 The Difference between NMI and IRQ)
- **Little-endian**: A way of placing a value of 2 or more bytes with the low byte first. 6502 addresses are stored in this order. (2.7 Memory and Addresses)
- **Load and store**: A load puts a value from memory into a register. A store writes a register value to memory. (5.1 Load and Store)

M

- **Machine code**: A program written as opcode and operand bytes, which the CPU runs directly. (3.5 Instruction Format)
- **Mask**: To make the CPU refuse an interrupt. While the I flag is 1, IRQ is masked. NMI cannot be masked. (6.3.1 The Four Interrupts)
- **Master reset**: Writing `%11` to the UART control register to return the UART to its initial state. It discards received data and also turns off the interrupt enable. (7.3 UART)
- **Memory**: A shelf of boxes in a row, each holding one byte. The boxes are numbered with addresses from the start. (2.7 Memory and Addresses)
- **Memory map**: The assignment of the 64KB address space to RAM, ROM, and I/O parts. (7.1.1 Assigning Addresses)
- **Memory-mapped I/O**: A method of operating I/O the same way as memory, by reading and writing at an address. (7.2 Memory-Mapped I/O)
- **Mirror**: One register appearing repeatedly at several addresses. In the Glow65 UART, two registers repeat across `$A000` to `$A0FF`. (7.1.2 The Glow65 Memory Map)
- **Mnemonic**: The name of an instruction, such as `LDA` or `STA`. The 6502 has 56 official instructions. (List of All 56 Instructions)
- **Monitor**: A small program in the Glow65 ROM. With commands, you can show and write memory and run programs. (7.4 Monitor Routines)
- **Most significant bit** (MSB): Bit 7, the leftmost of the 8 bits. In a signed number, it is the sign bit. (2.1 Bits and Bytes)

N

- **NMI** (Non-Maskable Interrupt): An interrupt that is accepted regardless of the I flag. Its vector is `$FFFA` and `$FFFB`. In Glow65 it is raised with the NMI button on the panel. (6.3.1 The Four Interrupts)

(c) 2026 masanorisakai / MIT License

O

- **Opcode**: The first byte of an instruction. It decides the kind of instruction and the addressing mode. (3.4 Fetch and Execute)
- **Operand**: The bytes of an instruction that give a value or an address. There are 0 to 2 bytes. (3.4 Fetch and Execute)
- **Overflow**: The V flag. It shows overflow when the calculation is read as signed numbers. It becomes 1 when two numbers with the same sign are added and the sign of the result changes. (6.1.3 V (Overflow))

P

- **P register (status register)**: An 8-bit register that gathers the flags. The six real flags are N, V, D, I, Z, and C. (6.1 Flags (the P Register))
- **Page**: A block of 256 bytes with the same high byte. When an addition changes the high byte of the result, it is called "crossing a page". (4.5 Absolute Modes)
- **Polling**: A method in which the program reads a status register again and again, waiting for a bit to become 1. (7.3.1 Polling)
- **Program counter** (PC): A 16-bit register that holds the address of the next instruction to run. (3.1 The CPU and Memory)

R

- **RAM** (Random Access Memory): Memory that can be read and written. Its contents disappear when the power is turned off. (2.8 ROM, RAM, and I/O)
- **Register**: A storage area of a few bytes inside the CPU. It can be read and written faster than memory. (3.2 Registers)
- **Reset**: Throwing away the running program and starting again from a fixed state. PC is set from the vector at `$FFFC` and `$FFFD`. (6.4 Reset)
- **ROM** (Read Only Memory): Memory that can only be read. Its contents stay when the power is off. It holds the program that runs at start-up. (2.8 ROM, RAM, and I/O)

S

- **Shift**: Moving the bits one place sideways. ASL shifts left and LSR shifts right. (5.4 Logic, Shift, and Bit Test)
- **Sign bit**: In two's complement, the most significant bit (bit 7). 1 means negative, and 0 means zero or positive. (2.5 Negative Numbers and Two's Complement)
- **Signature byte**: The one byte after BRK. The CPU skips it without using it. A handler can read it to tell the kinds of BRK apart. (6.3.4 BRK)
- **Stack**: The memory from `$0100` to `$01FF`, used so that the value put in last is taken out first. (6.2.1 Location and SP)
- **Stack pointer** (SP): An 8-bit register. The address `$0100` plus SP is the place where the next value is written. (6.2.1 Location and SP)

(c) 2026 masanorisakai / MIT License

- **Subroutine**: A small program that you call and use. It is called with JSR and returns with RTS. (5.5 Branch and Jump)
- **SYNC**: A signal that is 1 while the CPU is reading the first byte of an instruction (the opcode). (3.3 Buses)

T

- **Transfer**: Copying the value of one register to another register. The instructions are TAX, TAY, TXA, TYA, TSX, and TXS. (5.2 Register Transfer)
- **Trigger**: The condition that causes an interrupt. NMI uses an edge and IRQ uses a level. (6.3.1 The Four Interrupts)
- **Two's complement**: A way of representing negative numbers. Invert the bits and add 1. If the most significant bit is 1, the number is negative. With 8 bits it represents -128 to 127. (2.5 Negative Numbers and Two's Complement)

U

- **UART** (Universal Asynchronous Receiver/Transmitter): A device that handles serial communication with the terminal. In Glow65 it has two registers, `$A000` and `$A001`. (7.3 UART)

V

- **Vector**: A place that holds the destination address in 2 bytes (low byte first). NMI uses `$FFFA`, reset uses `$FFFC`, and IRQ and BRK use `$FFFE`. (3.7 Vectors for Reset and Interrupts)

Z

- **Zero page**: The first 256 bytes (`$0000` to `$00FF`), where the high byte of the address is `$00`. It can be addressed with a 1-byte address, which makes instructions shorter and faster. (1.2 Specifications of the 6502)

(c) 2026 masanorisakai / MIT License

8.2 Index

8.2.1 Instruction Index

Alphabetical order. Each link goes to the heading of the instruction in Chapter 5.

Mnemonic	Name
ADC	Add with Carry
AND	Logical AND
ASL	Arithmetic Shift Left
BCC	Branch if Carry Clear
BCS	Branch if Carry Set
BEQ	Branch if Equal
BIT	Bit Test
BMI	Branch if Minus
BNE	Branch if Not Equal
BPL	Branch if Plus
BRK	Break
BVC	Branch if Overflow Clear
BVS	Branch if Overflow Set
CLC	Clear Carry Flag
CLD	Clear Decimal Mode
CLI	Clear Interrupt Disable
CLV	Clear Overflow Flag
CMP	Compare Accumulator
CPX	Compare X Register
CPY	Compare Y Register
DEC	Decrement Memory
DEX	Decrement X Register
DEY	Decrement Y Register
EOR	Exclusive OR
INC	Increment Memory
INX	Increment X Register
INY	Increment Y Register
JMP	Jump

(c) 2026 masanorisakai / MIT License

Mnemonic	Name
JSR	Jump to Subroutine
LDA	Load Accumulator
LDX	Load X Register
LDY	Load Y Register
LSR	Logical Shift Right
NOP	No Operation
ORA	Logical OR
PHA	Push Accumulator
PHP	Push Processor Status
PLA	Pull Accumulator
PLP	Pull Processor Status
ROL	Rotate Left
ROR	Rotate Right
RTI	Return from Interrupt
RTS	Return from Subroutine
SBC	Subtract with Carry
SEC	Set Carry Flag
SED	Set Decimal Mode
SEI	Set Interrupt Disable
STA	Store Accumulator
STX	Store X Register
STY	Store Y Register
TAX	Transfer A to X
TAY	Transfer A to Y
TSX	Transfer Stack Pointer to X
TXA	Transfer X to A
TXS	Transfer X to Stack Pointer
TYA	Transfer Y to A

Instructions Added by the 65C02

These are available in the Glow65 65C02 mode. Each links to its heading in section 5.7.

Mnemonic	Name
BBR	Branch on Bit Reset

(c) 2026 masanorisakai / MIT License

Mnemonic	Name
BBS	Branch on Bit Set
BRA	Branch Always
PHX	Push X Register
PHY	Push Y Register
PLX	Pull X Register
PLY	Pull Y Register
RMB	Reset Memory Bit
SMB	Set Memory Bit
STP	Stop the Clock
STZ	Store Zero
TRB	Test and Reset Bits
TSB	Test and Set Bits
WAI	Wait for Interrupt
INC A	New modes of INC and DEC
DEC A	New modes of INC and DEC

8.2.2 Addressing Mode Index

Mode	Example	Section
Implied	`INX`	Implied
Accumulator	`ASL A`	Accumulator
Immediate	`LDA #$10`	4.3 Immediate
Zero page	`LDA $10`	Zero Page
Zero page,X	`LDA $10,X`	Zero Page,X
Zero page,Y	`LDX $10,Y`	Zero Page,Y
Absolute	`LDA $0400`	Absolute
Absolute,X	`LDA $0410,X`	Absolute,X
Absolute,Y	`LDA $0420,Y`	Absolute,Y
Indirect	`JMP ($0430)`	4.6 Indirect
Indexed indirect	`LDA ($20,X)`	4.7 Indexed Indirect
Indirect indexed	`LDA ($30),Y`	4.8 Indirect Indexed
Relative	`BNE label`	4.9 Relative

(c) 2026 masanorisakai / MIT License

8.2.3 Flag and Register Index

Item	Section
N (negative)	6.1.1 N and Z (Result Flags)
Z (zero)	6.1.1 N and Z (Result Flags)
C (carry)	6.1.2 C (Carry)
V (overflow)	6.1.3 V (Overflow)
D (decimal mode)	6.1.4 D (Decimal Mode)
I (interrupt disable)	6.1.5 I (Interrupt Disable)
B (break) and bit 5	6.1.6 B and Bit 5
The flags as a whole (P register)	3.2 Registers, 6.1 Flags (the P Register)
A (accumulator)	3.2 Registers
X, Y (index registers)	3.2 Registers
SP (stack pointer)	6.2.1 Location and SP, 3.2 Registers
PC (program counter)	3.1 The CPU and Memory, 3.2 Registers
P (status register)	6.1 Flags (the P Register), 3.2 Registers

8.2.4 Monitor Command and Routine Index

Item	Description	Section
Command `D`	Memory dump	7.4 Monitor Routines
Command `E`	Write bytes	7.4 Monitor Routines
Command `L`	Disassemble	7.4 Monitor Routines
Command `G`	Call a routine	7.4 Monitor Routines
Command `B`	Start BASIC	7.4 Monitor Routines
Command `T`	Start GAME65G	7.4 Monitor Routines
Command `?`	Help	7.4 Monitor Routines
Routine `$F000` CHRIN	Read one character (waits)	7.4 Monitor Routines
Routine `$F003` CHROUT	Send one character	7.4 Monitor Routines
Routine `$F006` ISCNTC	Check for Ctrl-C	7.4 Monitor Routines
Routine `$F009` Warm start	Return to the monitor	7.4 Monitor Routines
Routine `$F00C` GETKEY	Read one character without waiting	7.4 Monitor Routines

(c) 2026 masanorisakai / MIT License

8.2.5 Important Address Index

Address	Contents	Section
`$0000–$00FF`	Zero page	7.1.3 How RAM Is Used
`$0006–$000C`	BASIC's USR and conversion routine vectors	7.1.3 How RAM Is Used
`$00F0–$00FF`	Zero page used by the monitor	7.1.3 How RAM Is Used
`$0100–$01FF`	Stack	6.2.1 Location and SP
`$0200–$027F`	The monitor's line input buffer	7.1.3 How RAM Is Used
`$0300–$03F7`	For user programs	7.1.3 How RAM Is Used
`$03F8–$03FB`	Addresses of the NMI, IRQ, and BRK handlers (RAM)	6.3.6 Interrupts in Glow65
`$0400–$7FFF`	BASIC programs and variables	7.1.3 How RAM Is Used
`$8000–$9FFF`	GAME65G ROM	7.1.2 The Glow65 Memory Map
`$A000` `$A001`	UART status and control, data	7.3 UART
`$A000–$A0FF`	UART (mirrors)	7.1.2 The Glow65 Memory Map
`$A100–$BFFF`	Nothing connected	7.1.2 The Glow65 Memory Map
`$C000–$EFFF`	BASIC ROM	7.1.2 The Glow65 Memory Map
`$F000–$FFFF`	Monitor ROM and vectors	7.1.2 The Glow65 Memory Map
`$F000–$F00C`	The monitor's jump table	7.4 Monitor Routines
`$FFFA–$FFFF`	Vectors for NMI, reset, and IRQ and BRK	3.7 Vectors for Reset and Interrupts, 6.3.1 The Four Interrupts

8.2.6 List of Figures

Figure	Title	Image	Section
Figure 1-1	The front panel while BASIC is running	frontpanel.png	1.2 Specifications of the 6502
Figure 2-1	After LDA #$A0	02-gui-lda-a0.png	2.3 Hexadecimal
Figure 2-2	After LDA #$FB	02-gui-lda-fb.png	2.5 Negative Numbers and Two's Complement
Figure 3-1	The panel right after reset	03-reset.png	3.2 Registers
Figure 3-2	1st STEP (opcode of SEI)	03-cycle-1.png	3.4 Fetch and Execute
Figure 3-3	3rd STEP (opcode of LDX)	03-cycle-3.png	3.4 Fetch and Execute
Figure 3-4	4th STEP (operand of LDX)	03-cycle-4.png	3.4 Fetch and Execute
Figure 3-5	5th STEP (result of LDX)	03-cycle-5.png	3.4 Fetch and Execute
Figure 3-6	After the 3rd STEP in INSTR	03-instr-3.png	3.6 An Example of Executing Instructions

(c) 2026 masanorisakai / MIT License

Figure	Title	Image	Section
Figure 3-7	After the 4th STEP in INSTR	03-instr-4.png	3.6 An Example of Executing Instructions
Figure 4-1	Implied (`INX`)	04-implied.svg	Implied
Figure 4-2	Accumulator (`ASL A`)	04-accumulator.svg	Accumulator
Figure 4-3	Immediate (`LDA #$10`)	04-immediate.svg	4.3 Immediate
Figure 4-4	Zero page (`LDA $10`)	04-zeropage.svg	Zero Page
Figure 4-5	Zero page,X (`LDA $10,X`)	04-zeropage-x.svg	Zero Page,X
Figure 4-6	Zero page,Y (`LDX $10,Y`)	04-zeropage-y.svg	Zero Page,Y
Figure 4-7	Absolute (`LDA $0400`)	04-absolute.svg	Absolute
Figure 4-8	Absolute,X (`LDA $0410,X`)	04-absolute-x.svg	Absolute,X
Figure 4-9	Absolute,Y (`LDA $0420,Y`)	04-absolute-y.svg	Absolute,Y
Figure 4-10	Indirect (`JMP ($0430)`)	04-indirect.svg	4.6 Indirect
Figure 4-11	Indexed indirect (`LDA ($20,X)`)	04-indexed-indirect.svg	4.7 Indexed Indirect
Figure 4-12	Indirect indexed (`LDA ($30),Y`)	04-indirect-indexed.svg	4.8 Indirect Indexed
Figure 4-13	Relative (`BNE`)	04-relative.svg	4.9 Relative
Figure 4-14	After LDA $0410,X	04-gui-absolute-x.png	4.5 Absolute Modes
Figure 4-15	After LDA $10	04-gui-zeropage.png	4.5 Absolute Modes
Figure 4-16	After LDA #$10	04-gui-immediate.png	4.5 Absolute Modes
Figure 5-1	How the bits move in ASL, LSR, ROL, and ROR	05-shift.svg	5.4 Logic, Shift, and Bit Test
Figure 5-2	How BIT sets N, V, and Z	05-bit.svg	5.4 Logic, Shift, and Bit Test
Figure 5-3	The target of a branch	05-branch.svg	5.5 Branch and Jump
Figure 5-4	JSR and RTS	05-jsr-rts.svg	5.5 Branch and Jump
Figure 5-5	BRK and RTI	05-brk-rti.svg	5.5 Branch and Jump
Figure 5-6	PHA and PLA	05-stack.svg	5.6 Stack and Flag Operations
Figure 5-7	After CLC (V is off)	05-gui-adc-before.png	5.3 Arithmetic and Comparison
Figure 5-8	After ADC #$50 (N and V are lit)	05-gui-adc-after.png	5.3 Arithmetic and Comparison
Figure 5-9	BBR opcode and bit	05-65c02-bbr.svg	BBR (Branch on Bit Reset)

(c) 2026 masanorisakai / MIT License

Figure	Title	Image	Section
Figure 5-10	Zero page indirect	05-65c02-zp-indirect.svg	Zero Page Indirect `($nn)`
Figure 5-11	TSB and TRB	05-65c02-tsb-trb.svg	TSB (Test and Set Bits)
Figure 6-1	The 8 bits of the P register	06-flags.svg	6.1 Flags (the P Register)
Figure 6-2	Flags and branches after CMP	06-compare.svg	6.1.8 After a Comparison (CMP)
Figure 6-3	The stack with two nested JSR calls	06-stack-nested.svg	6.2.3 Nested JSR
Figure 6-4	The flow of an interrupt	06-interrupt-flow.svg	6.3.2 The Acceptance Sequence
Figure 6-5	The difference between NMI and IRQ	06-nmi-irq.svg	6.3.3 The Difference between NMI and IRQ
Figure 6-6	Right after the NMI button is pressed	06-gui-nmi-pending.png	6.3.7 Watching Interrupts on the Panel
Figure 6-7	1st STEP (the high byte of PC is pushed)	06-gui-nmi-push1.png	6.3.7 Watching Interrupts on the Panel
Figure 6-8	3rd STEP (P is pushed)	06-gui-nmi-push3.png	6.3.7 Watching Interrupts on the Panel
Figure 6-9	The acceptance of NMI is complete	06-gui-nmi-entered.png	6.3.7 Watching Interrupts on the Panel
Figure 6-10	After PHA in the handler	06-gui-nmi-pha.png	6.3.7 Watching Interrupts on the Panel
Figure 6-11	After RTI	06-gui-nmi-rti.png	6.3.7 Watching Interrupts on the Panel
Figure 6-12	Right after IRQ is accepted	06-gui-irq-entered.png	6.3.7 Watching Interrupts on the Panel
Figure 6-13	After LDA $A001	06-gui-irq-read.png	6.3.7 Watching Interrupts on the Panel
Figure 6-14	The IRQ LED goes out	06-gui-irq-off.png	6.3.7 Watching Interrupts on the Panel
Figure 7-1	The Glow65 memory map	07-memory-map.svg	7.1.2 The Glow65 Memory Map
Figure 7-2	The UART registers	07-uart.svg	7.3 UART
Figure 7-3	The flow of polling	07-polling.svg	7.3.1 Polling
Figure 7-4	With no received data ($A000 is $02)	07-gui-status-02.png	7.3.1 Polling
Figure 7-5	After typing h ($A000 is $03)	07-gui-status-03.png	7.3.1 Polling
Figure 7-6	After reading $A001	07-gui-data-read.png	7.3.1 Polling
Figure 7-7	After the data is read ($A000 is $02)	07-gui-status-after.png	7.3.1 Polling

(c) 2026 masanorisakai / MIT License

Part 2: Tutorial

You load short programs into Glow65, run them, and check the result on the LEDs and in the monitor. The manual in Part 1 explains the ideas behind them. You can read either part first. The sample programs (`.s`) are in the `tutorial` directory of the repository.

(c) 2026 masanorisakai / MIT License

Tutorial: your first program, an LED counter

You will write a five-line 6502 program, load it into Glow65, and watch it count in binary on the LEDs. Everything below was run on Glow65 and checked by the tests in `Tests/Glow65Tests/TutorialTests.swift`.

1. The program

`counter.s` adds 1 to the A register forever and writes it to memory:

```
        lda #$00        ; A = 0
loop:   sta COUNTER     ; write A to memory (COUNTER = $03F0)
        clc             ; clear the carry flag before adding
        adc #$01        ; A = A + 1
        jmp loop        ; do it again, forever
```

Instruction	Machine code	What happens on the bus
`lda #$00`	`A9 00`	two reads: the opcode, then the value 0
`sta $03F0`	`8D F0 03`	three reads (opcode, low and high address byte), then one **write**
`clc`	`18`	one read: the opcode
`adc #$01`	`69 01`	two reads
`jmp loop`	`4C 02 03`	three reads. PC goes back to `$0302`

One trip around the loop takes 11 cycles.

2. Get it into the computer

You need to put the 11 bytes `A9 00 8D F0 03 18 69 01 4C 02 03` in RAM at `$0300`. `$0300–$03F7` is free for your programs (`$03F8–$03FB` hold the interrupt vectors of chapter 6). (`$0200–$027F` is the monitor's line buffer, so a program there would be overwritten by the command you type to start it.)

Option A: type the bytes. No tools needed. Start the app (`swift run Glow65App`), flip **POWER** up, click the terminal, and type:

```
E 0300 A9 00 8D F0 03 18 69 01 4C 02 03
L 0300 0308
```

`E` writes bytes. `L` lists them as instructions (a disassembler), so you can check that the bytes you typed are the program you meant. It should print:

(c) 2026 masanorisakai / MIT License

```
0300  A9 00     LDA   #$00
0302  8D F0 03  STA   $03F0
0305  18        CLC
0306  69 01     ADC   #$01
0308  4C 02 03  JMP   $0302
```

(`D 0300 030F` shows the same memory as raw bytes instead.)

Option B: assemble and load a file. This needs cc65 (`brew install cc65`). The repository already contains the result as `counter.bin`, so you can skip the first command.

```
Tools/assemble.sh tutorial/counter.s       # builds tutorial/counter.bin (11 bytes)
swift run Glow65App --load tutorial/counter.bin@0300
```

`--load` loads the file into RAM every time you turn POWER on. In a running app you can also use **File → Load Program…** (⌘O), pick `counter.bin`, and leave the address at `0300`. Turn POWER on first, because powering on clears RAM.

3. Run it

In the monitor, type:

```
G 0300
```

The prompt does not come back. The program never returns, so the CPU is now running your loop. (To get the monitor back, press **RESET**. RAM is kept.)

At 1 MHz this loop runs about 90,000 times a second, so you cannot follow the count. The A LEDs look random because the panel samples the registers about 60 times a second.

4. Slow it down and watch

1. Set **SPEED** to **10 Hz**. The loop now takes a little over a second, and the panel shows each bus access in turn.
2. Watch the address bus. It walks through `$0300`, `$0301`, `$0302` and so on, so the PC LEDs and the low address LEDs change as the program advances.
3. Watch **INSTRUCTION AT PC**. It shows the instruction being executed, for example `0302 8D F0 03 STA $03F0`, so you can match the LEDs to the program.
4. Watch **SYNC**. It lights on the first access of every instruction (the opcode fetch).
5. Watch **WRITE**. It lights once per loop, for the `sta`. At that moment the address bus shows `$03F0` and the data bus shows the value of A.
6. Switch **RUN/STOP** to STOP, set **STEP MODE** to CYCLE, and press **STEP** repeatedly. One press is one bus access.

(c) 2026 masanorisakai / MIT License

The first trip through the program, access by access:

#	Address	Data	Read/Write	SYNC	Instruction
1	`$0300`	`A9`	read	yes	`lda #$00`
2	`$0301`	`00`	read		
3	`$0302`	`8D`	read	yes	`sta $03F0`
4	`$0303`	`F0`	read		
5	`$0304`	`03`	read		
6	`$03F0`	`00`	**write**		
7	`$0305`	`18`	read	yes	`clc`
8	`$0306`	`69`	read	yes	`adc #$01`
9	`$0307`	`01`	read		
10	`$0308`	`4C`	read	yes	`jmp $0302`
11	`$0309`	`02`	read		
12	`$030A`	`03`	read		

Two things to know about this view:

- While an instruction is being replayed, the A, X, Y, PC and flag LEDs show the values from **before** it. They change when the instruction finishes.
- A real 6502 takes 2 cycles for `clc` and does a second, dummy read of the next byte. Glow65 does not reproduce dummy reads, so `clc` shows one access. The cycle counter still counts 2.

5. Look at the result

Press **RESET** to stop the loop and get the monitor back, then type:

```
D 03F0 03FF
```

The first byte of the line is the last value the program wrote to `$03F0`. The line holds 16 bytes; the four non-zero bytes in the middle (`$03F8–$03FB`, `24 F4 24 F4`) are the monitor's interrupt vectors, which chapter 6 explains.

6. Experiments

Press RESET, change a byte with `E`, check the result with `L 0300 0308`, and run it again with `G 0300`. The bytes are at these addresses:

(c) 2026 masanorisakai / MIT License

```
0300  A9 00        lda #$00
0302  8D F0 03     sta $03F0
0305  18           clc
0306  69 01        adc #$01
0308  4C 02 03     jmp $0302
```

1. **Count by 2.** `E 0307 02` changes `adc #$01` to `adc #$02`. The A LEDs now skip the lowest bit.
2. **Write somewhere else.** `E 0303 F1` makes `sta` write to `$03F1`. Check with `D 03F0 03FF`.
3. **Count down.** `E 0305 38 E9 01` replaces `clc` / `adc #$01` with `sec` / `sbc #$01`. Same length, opposite direction. `L 0300 0308` now shows `SEC` and `SBC #$01` at `$0305` and `$0306`.

When you are ready for more, change `counter.s`, run `Tools/assemble.sh` again, and load the new file. Programs must fit in the 248 bytes at `$0300–$03F7` (see `ROM/user.cfg`).

(c) 2026 masanorisakai / MIT License

Tutorial 2: addition and the carry flag

The 6502 adds eight bits at a time. To add numbers bigger than 255 you add the low bytes, then the high bytes, and pass the **carry** from one step to the next. In this chapter you write a 16-bit addition and watch the carry flag move between the two halves. Everything below was run on Glow65 and checked by `Tests/Glow65Tests/TutorialChapterTests.swift`.

1. The program

`add.s` computes `SUM = A + B`. Each number takes two bytes, **low byte first**:

Number	Low byte	High byte
A	`$03F0`	`$03F1`
B	`$03F2`	`$03F3`
SUM	`$03F4`	`$03F5`

```
        clc             ; no carry goes into the low byte
        lda A_LO        ; low bytes first
        adc B_LO        ; A = A_LO + B_LO
        sta SUM_LO
        lda A_HI        ; then the high bytes
        adc B_HI        ; A = A_HI + B_HI + the carry from the low bytes
        sta SUM_HI
        rts             ; back to the monitor
```

`adc` means "add with carry": it adds the memory byte **and** the carry flag. `clc` clears the carry first, so the low-byte addition starts clean. If the low-byte addition overflows past 255, the CPU sets the carry flag, and the second `adc` adds it into the high byte.

The program ends with `rts`. The monitor's `G` command calls your program as a subroutine, so `rts` brings you back to the `*` prompt.

2. Load it

Load `add.bin` the way you did in chapter 1 (`swift run Glow65App --load tutorial/add.bin@0300`, or **File → Load Program…**), or type the 20 bytes:

```
E 0300 18 AD F0 03 6D F2 03 8D F4 03 AD F1 03 6D F3 03 8D F5 03 60
L 0300 0313
```

`L` should print:

(c) 2026 masanorisakai / MIT License

```
0300  18        CLC
0301  AD F0 03  LDA   $03F0
0304  6D F2 03  ADC   $03F2
0307  8D F4 03  STA   $03F4
030A  AD F1 03  LDA   $03F1
030D  6D F3 03  ADC   $03F3
0310  8D F5 03  STA   $03F5
0313  60        RTS
```

If you list further than `$0313`, you will see `BRK` lines. Those are just zero bytes in empty memory: `L` cannot tell code from data.

3. Run it

Put numbers in A and B, run the program, and look at the result:

```
E 03F0 05 00 07 00
G 0300
D 03F0 03F5
```

A is `$0005`, B is `$0007`. The first six bytes of the dump are `05 00 07 00 0C 00`, so SUM is `$000C` (12). The `*` prompt comes back by itself.

Now an addition that needs the carry. `$01FF + $0001`:

```
E 03F0 FF 01 01 00
G 0300
D 03F0 03F5
```

The first six bytes are `FF 01 01 00 00 02`: SUM is `$0200`. The low bytes `$FF + $01` do not fit in a byte, so the low byte of the sum is `$00` and the carry is passed on. The high bytes then add up to `$01 + $00 + 1 = $02`.

4. Watch the carry

The program is over in a few microseconds. To watch it, turn it into a loop: replace the final `rts` with `jmp $0300`.

```
E 03F0 FF 01 01 00
E 0313 4C 00 03
G 0300
```

The prompt does not come back, because the program now repeats forever. Then:

(c) 2026 masanorisakai / MIT License

1. Set **SPEED** to **10 Hz**, and switch **RUN/STOP** to STOP.
2. Set **STEP MODE** to INSTR, and press **STEP** until **INSTRUCTION AT PC** shows `0300 18 CLC`. (The CPU was stopped somewhere inside the loop, so it may take up to eight presses.)
3. Keep pressing STEP. After each press, look at the **A** LEDs and at the **C** flag in the STATUS P row (the rightmost LED):

Press	Instruction at PC before the press	A after	C after	What happened
1	`0300 CLC`	unchanged	0	the carry is cleared
2	`0301 LDA $03F0`	`$FF`	0	A = low byte of A
3	`0304 ADC $03F2`	`$00`	**1**	`$FF + $01 = $100`: the low 8 bits are 0, the carry is 1. The Z flag lights too
4	`0307 STA $03F4`	`$00`	1	the low byte of the sum is stored
5	`030A LDA $03F1`	`$01`	1	A = high byte of A. `lda` does not change C
6	`030D ADC $03F3`	`$02`	0	`$01 + $00 +` the carry (1) `= $02`, and no new carry
7	`0310 STA $03F5`	`$02`	0	the high byte of the sum is stored
8	`0313 JMP $0300`	`$02`	0	back to the start

The carry lights at press 3 and is used up at press 6. That is the whole idea of multi-byte arithmetic.

5. Experiments

Press RESET first, and put the `rts` back with `E 0313 60` (the loop version never returns to the monitor). The numbers in `$03F0–$03F5` are kept.

1. **Overflow.** `E 03F0 FF FF 01 00`, then `G 0300` and `D 03F0 03F5`. SUM is `$0000`. The carry out of the high byte is not stored anywhere, so a 16-bit sum wraps around.
2. **Subtract.** Subtraction works the same way, with the carry flag acting as "no borrow": `sec` before the first `sbc`. Change the opcodes with `E 0300 38` (`sec`), `E 0304 ED F2 03` (`sbc $03F2`) and `E 030D ED F3 03` (`sbc $03F3`). With A = `$0200` and B = `$0001` (`E 03F0 00 02 01 00`), SUM becomes `$01FF`.
3. **Make it 24 bits.** Add a third byte to each number, and one more `lda` / `adc` / `sta` to the program. Keep the order: low byte first, carry passed along.

(c) 2026 masanorisakai / MIT License

Tutorial 3: talking to the UART

So far your programs only changed memory. In this chapter a program prints text in the terminal, by writing to the UART directly. Everything below was run on Glow65 and checked by `Tests/Glow65Tests/TutorialChapterTests.swift`.

1. The UART is two addresses

An address on the bus does not always lead to RAM. Glow65 has a serial port (a UART, modelled on the MC6850) at two addresses. Reading or writing them talks to the device instead:

Address	Read	Write
`$A000`	status: bit 0 = a received character is waiting, **bit 1 = ready to send**	control
`$A001`	the received character	**send this character**

The terminal in the app is on the other end of that port. To print a character, a program waits until bit 1 of the status is set, then writes the character to `$A001`. (In Glow65 the UART is never busy, so the wait always ends at once. A real UART sends bits slowly, so the wait matters there. Write it anyway.)

2. The program

`hello.s` sends "HELLO, GLOW65" and a line break:

```
        ldx #$00            ; X = index into the message
next:   lda message,x       ; fetch the next character
        beq done            ; a 0 byte marks the end of the message
        tay                 ; keep the character in Y while we check the UART
wait:   lda UART_STATUS
        and #$02            ; is the transmit register empty?
        beq wait            ; no: look again
        tya                 ; yes: get the character back
        sta UART_DATA       ; send it
        inx
        jmp next
done:   rts
message:
        .byte "HELLO, GLOW65", $0D, $0A, $00
```

- `lda message,x` reads the byte at `message + X`. Counting X up walks through the text.
- Loading the status into A would destroy the character, so the program parks the character in Y with `tay` and gets it back with `tya`.
- `and #$02` keeps only bit 1. If the result is 0 the UART is not ready, and `beq wait` tries again.

(c) 2026 masanorisakai / MIT License

- `$0D, $0A` are the carriage return and line feed, so the next prompt starts on a new line. The final `$00` ends the message.

3. Load and run it

Load `hello.bin` (`swift run Glow65App --load tutorial/hello.bin@0300`, or **File** → **Load Program…**), or type the code and the message separately (40 bytes in all):

```
E 0300 A2 00 BD 18 03 F0 10 A8 AD 00 A0 29 02 F0 F9 98 8D 01 A0 E8 4C 02 03 60
E 0318 48 45 4C 4C 4F 2C 20 47 4C 4F 57 36 35 0D 0A 00
L 0300 0317
```

`L` prints:

```
0300  A2 00     LDX   #$00
0302  BD 18 03  LDA   $0318,X
0305  F0 10     BEQ   $0317
0307  A8        TAY
0308  AD 00 A0  LDA   $A000
030B  29 02     AND   #$02
030D  F0 F9     BEQ   $0308
030F  98        TYA
0310  8D 01 A0  STA   $A001
0313  E8        INX
0314  4C 02 03  JMP   $0302
0317  60        RTS
```

The message starts at `$0318`. If you list past `$0317`, the text shows up as nonsense instructions (`PHA`, ...): `L` cannot tell code from data.

Run it:

```
G 0300
```

```
HELLO, GLOW65
*
```

The text comes out of your program, and `rts` returns to the monitor.

4. See it on the panel

At full speed you can already see it work: each character makes the **WRITE** and **TX** LEDs flash. To look at one character, turn the program into a loop (`E 0317 4C 00 03` replaces `rts` with `jmp $0300`), run it with `G`

(c) 2026 masanorisakai / MIT License

`0300`, and then set **SPEED** to **10 Hz** and **RUN/STOP** to STOP. The terminal fills with lines at first, because the loop prints the message over and over before you slow it down. Then set **STEP MODE** to CYCLE and press **STEP** until the **WRITE** LED lights with `STA $A001` under INSTRUCTION AT PC:

You stopped the loop at an arbitrary place, so the character you see first may be any letter of the message. Keep pressing STEP until WRITE lights, and read the rows:

LED row	What you see when the program writes a character
ADDRESS BUS	always `$A001`: **A15**, **A13** and **A0** lit
DATA BUS	the ASCII code of the character. `H` is `$48` (**D6** and **D3** lit); then `E`, `L`, `L`, `O` are `$45`, `$4C`, `$4C`, `$4F`
WRITE	lit

Before each write there is a read of `$A000`: the program asking the UART whether it is ready.

5. A shorter way: use the monitor's routine

The monitor already has a routine that waits and sends one character: `CHROUT` at `$F003` (its address is fixed, so your programs can call it). `hello_monitor.s` is hello.s without the waiting loop:

```
        ldx #$00
next:   lda message,x
        beq done
        jsr $F003           ; the monitor waits and sends A
        inx
        jmp next
done:   rts
```

It is 31 bytes. To type it in, with the message at `$030F`:

```
E 0300 A2 00 BD 0F 03 F0 07 20 03 F0 E8 4C 02 03 60
E 030F 48 45 4C 4C 4F 2C 20 47 4C 4F 57 36 35 0D 0A 00
G 0300
```

It prints the same text. BASIC's output goes through the same `CHROUT`.

6. Experiments

1. **Your own message.** Press RESET, then `E 0318 47 4F 4F 44 0D 0A 00` (GOOD, a line break, and the end marker), and `G 0300`. Leave out `0D 0A` and the next prompt appears right after the text. Keep the message short: it must not run into `$03F0`.
2. **Another ASCII code.** `E 0318 41 42 43 0D 0A 00` prints `ABC`. Letters go up one code at a time, so you can print `DEF` by adding 3 to each byte.

(c) 2026 masanorisakai / MIT License

3. **Receiving.** Bit 0 of the status says that a character has arrived, and reading `$A001` returns it. A program that waits for bit 0, reads the character and sends it back is an echo. Try writing it with the same pieces.

(c) 2026 masanorisakai / MIT License

Tutorial 4: reading the keyboard, an echo program

Chapter 3 sent characters out through the UART. This chapter reads them in. The program waits for a key, reads it, and sends it back, so what you type appears on the screen. Everything below was run on Glow65 and checked by `Tests/Glow65Tests/TutorialChapterTests.swift`.

1. Why the screen needs an echo

The terminal in the app does not show your keys by itself. A key goes to the UART, and the terminal shows only what the UART sends out. When you type in the monitor, the **monitor** reads each key and sends it back; that is why you see it.

So a program that wants to show typed text has to do the same: read, then send. That is an echo.

Receiving uses the same two registers as sending:

Address	Read	Write
`$A000`	status: **bit 0 = a received character is waiting**, bit 1 = ready to send	control
`$A001`	**the received character** (reading it removes it from the UART)	send a character

A program cannot know when a key will be pressed, so it **polls**: it reads the status again and again until bit 0 is set.

2. The program

`echo.s`:

```
loop:   lda UART_STATUS
        and #$01            ; has a character arrived?
        beq loop            ; no: look again
        lda UART_DATA       ; yes: read it
        cmp #$0D            ; is it Enter?
        beq done            ; yes: finish
        tay                 ; keep the character in Y
send:   lda UART_STATUS
        and #$02            ; ready to send?
        beq send            ; no: look again
        tya
        sta UART_DATA       ; send it back
        jmp loop
done:   lda #$0D            ; send a line break
        jsr CHROUT
        lda #$0A
        jsr CHROUT
        rts
```

- The first three lines are the waiting loop. `and #$01` keeps bit 0 of the status. While it is 0, `beq loop` goes round again.
- `cmp #$0D` compares A with the code of Enter. If they are equal, the Z flag is set and `beq done` jumps. Without this exit the program would never return to the monitor.
- The middle part is the sending code from chapter 3.
- At the end it sends a carriage return and a line feed through the monitor's `CHROUT` (`$F003`), so the prompt starts on a new line.

3. Load and run it

Load `echo.bin` (`swift run Glow65App --load tutorial/echo.bin@0300`, or **File → Load Program…**), or type the 40 bytes:

```
E 0300 AD 00 A0 29 01 F0 F9 AD 01 A0 C9 0D F0 0F A8 AD 00 A0 29 02 F0 F9 98 8D 01 A0 4C
00 03 A9 0D 20 03 F0 A9 0A 20 03 F0 60
L 0300 0327
```

`L` prints:

(c) 2026 masanorisakai / MIT License

```
0300  AD 00 A0  LDA   $A000
0303  29 01     AND   #$01
0305  F0 F9     BEQ   $0300
0307  AD 01 A0  LDA   $A001
030A  C9 0D     CMP   #$0D
030C  F0 0F     BEQ   $031D
030E  A8        TAY
030F  AD 00 A0  LDA   $A000
0312  29 02     AND   #$02
0314  F0 F9     BEQ   $030F
0316  98        TYA
0317  8D 01 A0  STA   $A001
031A  4C 00 03  JMP   $0300
031D  A9 0D     LDA   #$0D
031F  20 03 F0  JSR   $F003
0322  A9 0A     LDA   #$0A
0324  20 03 F0  JSR   $F003
0327  60        RTS
```

Run it and type:

```
G 0300
hello
```

Each letter is echoed by your program, not by the monitor. Press **Enter**: the program sends a line break, returns, and the `*` prompt comes back:

```
hello
*
```

4. Watch it on the panel

1. Run the program with `G 0300` at full speed. The prompt does not come back, because the program is waiting for a key.
2. Set **SPEED** to **10 Hz**, and type a letter in the terminal. Nothing appears yet: the CPU is now so slow that it has not looked at the UART. The letter is waiting inside the UART.
3. Watch **INSTRUCTION AT PC**. It cycles through `LDA $A000`, `AND #$01`, `BEQ $0300`: the polling loop. Whenever the program reads `$A000`, the address bus shows `$A000` (**A15** and **A13** lit), and **READ** is lit.
4. The next time round the loop, the status has bit 0 set, and `beq` no longer jumps. The program reads `$A001`: the address bus shows `$A001` (**A15**, **A13**, **A0**), the data bus shows the ASCII code of the letter (`h` is `$68`: **D6**, **D5**, **D3**), and the **RX** LED lights.

(c) 2026 masanorisakai / MIT License

5. A few instructions later the program writes the same code to `$A001`. This time **WRITE** and **TX** light, and the letter appears in the terminal.

Before you press Enter, set **SPEED** back to 1 MHz, so the monitor is fast again.

5. Experiments

Press RESET first. A program loaded in RAM is still there.

1. **Finish with another key.** `E 030B 2E` changes `cmp #$0D` to `cmp #$2E`, so the program ends at `.` (ASCII `$2E`) instead of Enter. Now Enter is echoed like any other key: it moves the cursor back to the start of the line, without a line feed.

2. **Capital letters.** `echo_upper.s` (50 bytes, `echo_upper.bin`) echoes `a-z` as `A-Z`. Between reading the key and sending it, it adds:

   ```
   cmp #$61            ; 'a'
   bcc send_it         ; below 'a': leave it alone
   cmp #$7B            ; one past 'z'
   bcs send_it         ; 'z' or above: leave it alone
   and #$DF            ; clear bit 5
   ```

   In ASCII, `a-z` are `$61-$7A` and `A-Z` are `$41-$5A`. They differ only in bit 5 (`$20`), so `and #$DF` turns a lower-case letter into a capital. `cmp` sets the carry flag when A is greater than or equal to the operand, which is what `bcc` and `bcs` test. Load `echo_upper.bin`, run it, and type `Hello, World 123`.

3. **Count the keys.** Keep a counter in a spare memory cell (for example `$03F0`) and add 1 to it every time a character is read. After you press Enter, look at the count with `D 03F0 03FF`.

(c) 2026 masanorisakai / MIT License

Tutorial 5: calling machine code from BASIC (USR)

BASIC is easy to write and slow to run. Machine code is the opposite. BASIC's `USR` function lets you use both: write the program in BASIC, and call a machine-code routine when you need one. In this chapter the routine doubles a number. Everything below was run on Glow65 and checked by `Tests/Glow65Tests/TutorialUSRTests.swift`.

1. How USR works

`USR(X)` does three things:

1. BASIC works out `X` and leaves it in the **FAC**, its floating-point accumulator.
2. BASIC jumps to the **USR vector** in page zero: a `JMP` instruction at address 10 (`$0A`). The two bytes after it, at addresses 11 and 12, are the address it jumps to.
3. When your routine ends with `rts`, BASIC takes whatever is in the FAC as the value of `USR(X)`.

You can look at the vector yourself. Right after BASIC starts:

```
PRINT PEEK(10);PEEK(11);PEEK(12)
 76  241  207
```

76 is `$4C`, the opcode of `JMP`. 241 and 207 are the low and high bytes of `$CFF1`, an error routine inside BASIC. That is why `PRINT USR(21)` at this point prints `?ILLEGAL QUANTITY ERROR`. To use your own routine, **POKE its address into 11 (low byte) and 12 (high byte)**.

The FAC holds a floating-point number, which is awkward to work with in machine code. BASIC offers two helper routines, reached through vectors in page zero:

Vector	Points to	Does
`$06–$07`	AYINT	FAC → 16-bit signed integer, high byte in `FACMO` (`$B4`), low byte in `FACLO` (`$B5`)
`$08–$09`	GIVAYF	16-bit signed integer (A = high byte, Y = low byte) → FAC

2. The program

`usr_double.s`:

```
        jsr to_integer  ; the argument, as an integer, is now in FACMO:FACLO
        asl FACLO       ; shift the low byte left; bit 7 goes into the carry
        rol FACMO       ; rotate the carry into the high byte: 16-bit X * 2
        lda FACMO       ; A = high byte
        ldy FACLO       ; Y = low byte
        jsr to_fac      ; put the integer back into the FAC
        rts

to_integer:
        jmp (ADRAYI)
to_fac: jmp (ADRGAY)
```

- `jsr` cannot jump through a vector, but `jmp (vector)` can. So the program calls a small stub with `jsr`, and the stub does `jmp (ADRAYI)`. When BASIC's routine ends with `rts`, it returns to the instruction after your `jsr`.
- `asl FACLO` / `rol FACMO` is a 16-bit shift left, which doubles the number. This is the same carry idea as in chapter 2.

3. Load it

The routine goes at `$0300`, as before. BASIC keeps its own program and variables from `$0400` upwards, so it does not touch `$0300–$03F7`. Load the file first, then start BASIC:

```
swift run Glow65App --load tutorial/usr_double.bin@0300
```

Turn **POWER** on, and in the monitor type `B` to start BASIC. Press Return at the two prompts. (Or type the 21 bytes in the monitor first: `E 0300 20 0F 03 06 B5 26 B4 A5 B4 A4 B5 20 12 03 60 6C 06 00 6C 08 00`, then `L 0300 0313` to check, then `B`.)

`L 0300 0313` shows:

```
0300  20 0F 03  JSR   $030F
0303  06 B5     ASL   $B5
0305  26 B4     ROL   $B4
0307  A5 B4     LDA   $B4
0309  A4 B5     LDY   $B5
030B  20 12 03  JSR   $0312
030E  60        RTS
030F  6C 06 00  JMP   ($0006)
0312  6C 08 00  JMP   ($0008)
```

(c) 2026 masanorisakai / MIT License

4. Call it

Tell BASIC where the routine is. `$0300` is 768, so the low byte is 0 and the high byte is 3:

```
POKE 11,0:POKE 12,3
PRINT USR(21)
 42
PRINT USR(1000)
 2000
PRINT USR(-5)
-10
```

It works inside a program too:

```
FOR I=1 TO 5:PRINT USR(I);:NEXT
 2  4  6  8  10
```

BASIC is now calling your routine from its own loop. Note that BASIC starts with the vector set to the error routine every time: after you start BASIC again (or press RESET and type `B`), POKE 11 and 12 again.

5. The limits of the routine

The integer is a 16-bit signed number, from -32768 to 32767, and `asl` / `rol` just shifts bits:

```
PRINT USR(16383)
 32766
PRINT USR(16384)
-32768
PRINT USR(20000)
-25536
```

`16384 × 2 = 32768` does not fit, so the top bit becomes the sign bit and the result is negative. A machine-code routine does exactly what its instructions say, and nothing checks the range for you.

6. Printing from machine code

`usr_hello.s` (37 bytes) sends a message through the monitor's `CHROUT` (`$F003`), as in chapter 3. It ignores the argument and leaves the FAC alone. Load it the same way (`--load tutorial/usr_hello.bin@0300`, or `E 0300 A2 00 BD 0F 03 F0 07 20 03 F0 E8 4C 02 03 60` and the message with `E 030F 48 45 4C 4C 4F 20 46 52 4F 4D 20 41 53 53 45 4D 42 4C 59 0D 0A 00`), then:

(c) 2026 masanorisakai / MIT License

```
POKE 11,0:POKE 12,3
A=USR(0)
HELLO FROM ASSEMBLY
10 FOR I=1 TO 3:A=USR(0):NEXT
RUN
HELLO FROM ASSEMBLY
HELLO FROM ASSEMBLY
HELLO FROM ASSEMBLY
```

`A=USR(0)` is the usual way to call a routine whose result you do not need.

7. Experiments

Load `usr_double` again first.

1. **Halve instead of double.** Change the four bytes at `$0303–$0306` (`asl $B5` / `rol $B4`) to `lsr $B4` / `ror $B5` (`46 B4 66 B5`). From BASIC, with `POKE`, that is addresses 771 to 774:

   ```
   POKE 771,70:POKE 772,180:POKE 773,102:POKE 774,181
   PRINT USR(42)
    21
   ```

   Your routine is in RAM, so BASIC can change it. `lsr` shifts the high byte right and puts bit 0 into the carry; `ror` rotates that carry into the low byte's top bit.

2. **Find the limit.** `PRINT USR(32767)` prints `-2` (the doubled value, 65534, wraps around). `PRINT USR(40000)` prints `?ILLEGAL QUANTITY ERROR`: the argument itself is outside -32768 to 32767, so the first helper call, FAC to integer, raises the error before your routine can use the number.

3. **Add 100.** Replace the shift with an addition: `clc`, `lda FACLO`, `adc #100`, `sta FACLO`, then add the carry into `FACMO` (`bcc` over an `inc FACMO`). The routine gets longer, so write it as a new `.s` file, assemble it with `Tools/assemble.sh`, and load the new `.bin`.

8. Things to remember

- Your routine may use `$0300–$03F7` freely, and the FAC and the other BASIC zero-page locations only as BASIC expects. BASIC itself uses almost all of page zero, so avoid other zero-page addresses.
- A routine called with `USR` must end with `rts`.
- From machine code you can call the monitor's routines (`CHROUT` at `$F003`, `GETKEY` at `$F00C`).

(c) 2026 masanorisakai / MIT License

Tutorial 6: interrupts

Until now the program asked for everything: it polled the UART status in a loop and waited for a key. An **interrupt** turns that around. Hardware raises a signal, the CPU stops what it is doing, runs a short routine (the **handler**), and then goes back to exactly where it was. In this chapter you use both interrupt lines of the 6502, **NMI** and **IRQ**, and watch the CPU enter and leave a handler on the panel. Everything below was run on Glow65 and checked by `Tests/Glow65Tests/TutorialInterruptTests.swift`.

1. What the CPU does on an interrupt

Before each instruction the CPU checks two inputs:

Input	Raised by	Can the program block it?
NMI (non-maskable)	The panel's **NMI** button	No. It is taken even when the I flag is set. It triggers once per press (on the edge)
IRQ	The UART, when it has a received character and you have enabled its interrupt	Yes: while the **I** flag is 1, IRQ is ignored. `sei` sets I, `cli` clears it. IRQ is **level**-triggered: as long as the line stays high, the CPU keeps coming back

When the CPU takes an interrupt it:

1. pushes the **PC** high byte, the PC low byte, then the **P** register onto the stack (three writes to `$01xx`),
2. sets the **I** flag, so a handler is not interrupted by IRQ again,
3. reads the handler address from a vector: `$FFFA/$FFFB` for NMI, `$FFFE/$FFFF` for IRQ,
4. jumps there.

The handler ends with `rti`. It pulls P and then the PC back, so the program resumes with its flags as they were. Unlike `rts`, `rti` does not add 1 to the address, because the pushed PC already points at the next instruction.

The CPU saves only PC and P. If the handler uses A, X or Y, it must save and restore them itself (`pha` ... `pla`), or the main program's registers are destroyed.

2. The vectors on this board

The vectors at `$FFFA` and `$FFFE` are in ROM, so you cannot change them. The monitor therefore points them at two small jump instructions, which jump through two addresses in RAM:

RAM address	Contents	Used for
`$03F8/$03F9`	NMI handler address (low, high)	NMI
`$03FA/$03FB`	IRQ handler address (low, high)	IRQ and `BRK`

After a reset both hold the address of a lone `rti`, so an interrupt you did not expect does nothing. Your program installs its own handler by writing its address there. (The memory for your programs is now

(c) 2026 masanorisakai / MIT License

`$0300–$03F7`.)

3. NMI: count button presses

`tutorial/nmi_count.s` installs a handler and then spins in `jmp loop`. Each NMI runs the handler, which adds 1 to `COUNT` (`$03F0`) and prints `!`:

```
start:  lda #<handler           ; point the NMI vector at our handler
        sta NMIVEC
        lda #>handler
        sta NMIVEC+1
        lda #$00
        sta COUNT
loop:   jmp loop                ; the main program does nothing

handler:
        pha                     ; the CPU saved only PC and P: save A ourselves
        inc COUNT
wait:   lda UART_STATUS
        and #$02                ; transmit ready?
        beq wait
        lda #'!'
        sta UART_DATA
        pla
        rti                     ; restores P and PC (and the I flag with it)
```

Run it:

```
swift run Glow65App --load tutorial/nmi_count.bin@0300
```

Turn POWER on and type `G 0300`. The terminal shows no prompt: the CPU is busy spinning. Press the **NMI** button on the panel. Each press prints one `!`. Press RESET to stop (RESET keeps RAM), then check the count:

```
D 03F0 03F0
```

The first byte is the number of presses.

4. Watch an interrupt on the panel

Set the SPEED to 1 Hz or 10 Hz and STEP MODE to CYCLE, or stop the CPU with RUN/STOP and use STEP. While the program spins in `jmp loop` at `$030F`, press **NMI**, then step. You see the interrupt entry as bus accesses:

(c) 2026 masanorisakai / MIT License

#	Address	Data	R/W	What it is
1	`$01FD`	`03`	write	PC high byte pushed
2	`$01FC`	`0F`	write	PC low byte pushed
3	`$01FB`	`26`	write	P pushed (bit 5 is always 1 on the stack)
4	`$FFFA`	`1E`	read	NMI vector, low byte
5	`$FFFB`	`F4`	read	NMI vector, high byte

The values are from a run of `nmi_count`: the stack pointer was `$FD` (the monitor's `G` command had already pushed its return address) and the PC `$030F` (the `jmp loop`). After step 5 the PC is `$F41E`, the monitor's `jmp (NMIVEC)`; the next steps read `$03F8/$03F9`, and the CPU lands in your `handler` at `$0312`. The first access is not marked SYNC: an interrupt entry does not fetch an opcode. A real 6502 spends 7 cycles here, with two extra dummy reads that Glow65 does not reproduce, so you see five accesses.

After `rti` the P LEDs and the PC LEDs return to what they were, and the stack pointer is back at `$FD`.

5. IRQ: echo by interrupt

In chapter 4 the program waited in a loop for each character. With IRQ the main program can do something else, and the UART interrupts it when a character arrives. `tutorial/irq_echo.s` counts in a 16-bit counter (`$03F0` low, `$03F1` high) while the handler echoes what you type.

The UART's control register (`$A000`) has one new bit:

Bit	Meaning
bit 7 of the control register	1 = raise IRQ while a received character is waiting
bit 7 of the status register	1 = this UART is requesting an interrupt

The setup, in order:

```
        sei                     ; no interrupts while we set up
        ...                     ; install the handler address in IRQVEC ($03FA)
        lda #$80                ; UART control bit 7: raise IRQ when a character arrives
        sta UART_STATUS         ; ($A000 is status when read and control when written)
        cli                     ; let the CPU accept IRQ
```

The handler reads the character, echoes it, and treats Enter as the signal to finish:

(c) 2026 masanorisakai / MIT License

```
handler:
        pha
        lda UART_DATA           ; reading the data register clears the IRQ request
        cmp #$0D
        beq enter
        pha
wait:   lda UART_STATUS
        and #$02
        beq wait
        pla
        sta UART_DATA           ; echo
        pla
        rti
enter:  sta DONE                ; A = $0D, not zero: tell the main program to stop
        pla
        rti
```

Run it with `swift run Glow65App --load tutorial/irq_echo.bin@0300`, then `G 0300`. Whatever you type is echoed. Press Enter and the monitor's prompt comes back. Check the counter:

```
D 03F0 03F2
```

`$03F0/$03F1` is how far the main program counted while you typed, and `$03F2` is `0D`, the flag the handler set.

The line `lda UART_DATA` matters. IRQ is level-triggered: the UART keeps the line high while a character is unread. A handler that returns without reading `$A001` is interrupted again at once, forever.

6. BRK is an interrupt too

`brk` (opcode `$00`) pushes the PC plus 2 and P, sets I, and jumps through the **IRQ** vector. It is the same path as a hardware IRQ, so `$03FA/$03FB` serve both. The byte after `brk` is skipped on return: it is a free "signature" byte, which a handler can read to learn which `brk` it came from. (The B flag in the pushed P tells a handler whether it came from `brk` or from IRQ.)

7. Things to try

1. **Watch SP.** Press NMI while stepping. The SP LEDs go from `$FD` to `$FA` after the three pushes and back to `$FD` after `rti`.
2. **Count differently.** In `nmi_count.s` print the count as a letter instead of `!`: add `#'0'` to COUNT and print it (up to 9 presses).
3. **Forget the read.** In `irq_echo.s` make the handler's first instruction `rti`, so it never reads `$A001`. After you type one character, the main program stops counting and nothing is echoed. Why? (Slow the

(c) 2026 masanorisakai / MIT License

CPU down and watch the PC: it only walks through the interrupt entry, the handler and `rti`, over and over.) Press RESET to recover.

4. **Use BRK.** Enter `E 0300 A9 01 00 60 8D F0 03 60`, then `G 0300`. The `brk` at `$0302` goes through the default `rti` handler and the program resumes at `$0304`, skipping the `60` at `$0303`: `D 03F0 03F0` shows `01`.
5. **Turn it off.** `sei` blocks IRQ but not NMI. Put `sei` before the `jmp loop` in `nmi_count.s` and press NMI anyway.

8. Things to remember

- NMI cannot be masked; IRQ is masked by the I flag.
- The CPU saves only PC and P. Save A, X and Y in the handler if you use them.
- End a handler with `rti`, not `rts`.
- A level-triggered IRQ must be acknowledged at the source (here: read `$A001`).
- The monitor's warm start switches the UART interrupt off, so returning to the monitor with `rts` is safe even if your program left it on.

(c) 2026 masanorisakai / MIT License

Tutorial 7: Switching to the 65C02

The 65C02 is the successor of the 6502. It runs the same programs and adds instructions that make them shorter. Glow65 runs as a 6502 by default and as a 65C02 when you ask for it. In this chapter you write one small program twice, once for each CPU, compare their sizes, and then run the 65C02 version on both machines to see what an undefined instruction does. Everything below was run on Glow65 and checked by `Tests/Glow65Tests/TutorialChapter65C02Tests.swift`.

1. What the 65C02 adds

This chapter uses four of the new instructions:

Instruction	Bytes	What it does	The 6502 way
`stz addr`	2 to 3	Stores 0 to memory	`lda #$00` first, then `sta addr`
`inc a`	1	Adds 1 to A	`clc` and `adc #$01` (3 bytes)
`bra label`	2	Branches always	`jmp label` (3 bytes)
`phx` / `plx`	1	Pushes / pulls X (`phy` / `ply` for Y)	`txa`, `pha` and `pla`, `tax` (through A)

The 65C02 also has more addressing modes and a few more instructions (`wai`, `stp` and the bit instructions among them). The details are in the manual: 4.11 Modes Added by the 65C02 and 5.7 The 65C02. How Glow65 implements it is written in the implementation spec.

2. The same program twice

Both programs do two things. They clear the 16 bytes at `$03D0–$03DF`, and then they count a 16-bit value from 0 up to `$0300` in `COUNT` (`$03F0` low, `$03F1` high). The count is kept in A (high byte) and in memory (low byte).

`clear_6502.s` is the version for the 6502:

(c) 2026 masanorisakai / MIT License

```
        ldx #$0F        ; X = 15: index of the last byte of the area
        lda #$00        ; a 6502 cannot store a constant: A must hold the 0
clr:    sta BUF,x       ; clear one byte
        dex
        bpl clr         ; repeat until X wraps from 0 to $FF
        sta COUNT       ; A is still 0
loop:   cmp #$03        ; A = high byte of the count: finished?
        beq done
        inc COUNT       ; low byte + 1
        bne loop        ; no wrap: next
        clc             ; the low byte wrapped: high byte + 1
        adc #$01
        jmp loop        ; the 6502 has no unconditional short branch
done:   sta COUNT+1
        rts
```

`clear_65c02.s` is the version for the 65C02:

```
        ldx #$0F        ; X = 15: index of the last byte of the area
clr:    stz BUF,x       ; STZ stores a zero without using A
        dex
        bpl clr         ; repeat until X wraps from 0 to $FF
        stz COUNT       ; the low byte of the count
        lda #$00        ; A = high byte of the count
loop:   cmp #$03        ; finished?
        beq done
        inc COUNT       ; low byte + 1
        bne loop        ; no wrap: next
        inc a           ; the low byte wrapped: INC A adds 1 to A
        bra loop        ; BRA: unconditional branch, 2 bytes
done:   sta COUNT+1
        rts
```

The 65C02 version uses `stz` instead of loading a zero into A, `inc a` instead of `clc` and `adc #$01`, and `bra` instead of `jmp`. The loop in the middle is the same.

3. Assemble them

`Tools/assemble.sh` needs the `cc65` package, as in the earlier chapters. The 65C02 source begins with `.setcpu "65C02"`, which tells `ca65` to accept the new instructions. Without it, `stz` is an error. The 6502 source has `.setcpu "6502"`.

(c) 2026 masanorisakai / MIT License

```
Tools/assemble.sh tutorial/clear_6502.s
Tools/assemble.sh tutorial/clear_65c02.s
```

```
tutorial/clear_6502.bin: 32 bytes (load at $0300)
tutorial/clear_65c02.bin: 29 bytes (load at $0300)
```

The 65C02 version is 3 bytes shorter: 2 for `inc a` and 1 for `bra`. `stz` saves nothing in this program, because the 65C02 version still needs its own `lda #$00` for the high byte, while the 6502 version gets by with one. The `.bin` files are committed in `tutorial/`, so you can skip this step.

4. Run it on a 65C02

Start Glow65 as a 65C02 and load the 65C02 binary. `--cpu 65c02` works for both programs:

```
swift run glow65-cli --cpu 65c02 --load tutorial/clear_65c02.bin@0300
```

or

```
swift run Glow65App --cpu 65c02 --load tutorial/clear_65c02.bin@0300
```

In the app, flip POWER up and click the terminal. First fill the area with `$AA`, so you can see it being cleared:

```
E 03D0 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
```

List the program. The monitor's `L` command detects the CPU at run time, so on a 65C02 it shows the new instructions by name:

```
L 0300 031C
```

(c) 2026 masanorisakai / MIT License

```
0300  A2 0F     LDX   #$0F
0302  9E D0 03  STZ   $03D0,X
0305  CA        DEX
0306  10 FA     BPL   $0302
0308  9C F0 03  STZ   $03F0
030B  A9 00     LDA   #$00
030D  C9 03     CMP   #$03
030F  F0 08     BEQ   $0319
0311  EE F0 03  INC   $03F0
0314  D0 F7     BNE   $030D
0316  1A        INC   A
0317  80 F4     BRA   $030D
0319  8D F1 03  STA   $03F1
031C  60        RTS
```

Run it and dump the area and the counter:

```
G 0300
D 03D0 03F1
```

```
03D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
03E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
03F0: 00 03 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

The 16 bytes at `$03D0` are zero again, and `$03F0/$03F1` hold `00 03`, which is `$0300` (low byte first). The bytes at `$03F8–$03FB` are the interrupt vectors of chapter 6.

5. Run it on a 6502

Now start the same binary on the default CPU. Leave out `--cpu` (or write `--cpu 6502`):

```
swift run glow65-cli --load tutorial/clear_65c02.bin@0300
```

Fill the area with `$AA` again and list the program:

```
E 03D0 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
L 0300 0318
```

(c) 2026 masanorisakai / MIT License

```
0300  A2 0F     LDX   #$0F
0302  9E        .BYTE $9E
0303  D0 03     BNE   $0308
0305  CA        DEX
0306  10 FA     BPL   $0302
0308  9C        .BYTE $9C
0309  F0 03     BEQ   $030E
030B  A9 00     LDA   #$00
030D  C9 03     CMP   #$03
030F  F0 08     BEQ   $0319
0311  EE F0 03  INC   $03F0
0314  D0 F7     BNE   $030D
0316  1A        .BYTE $1A
0317  80        .BYTE $80
0318  F4        .BYTE $F4
```

The bytes are the same, but the 6502 does not know `$9E`, `$9C`, `$1A` and `$80`, and `L` prints them as `.BYTE`. The 6502 in Glow65 runs such an undefined opcode as a one-byte NOP (2 cycles). The bytes after it, which were the operands of `stz` and `bra`, are then read as instructions of their own: `D0 03` becomes a `BNE`, `F0 03` a `BEQ`. Run it:

```
G 0300
D 03D0 03F1
```

```
03D0: AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
03E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
03F0: 00 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

Nothing was cleared: the area still holds `$AA`. The counter is `00 00` instead of `00 03`. `inc a` is a NOP here, so A never reaches 3: the low byte counts through 256 values once and the program falls out of the loop with A still 0. The program did not crash and did not print an error. It ran to the end and returned to the monitor with the wrong answer. This is why a 65C02 program must not be loaded on a 6502 machine unless you want to see this. The CPU does not warn you.

The other direction is harmless. The 6502 program uses only instructions the 65C02 also has, so `clear_6502.bin` produces the same dump on both machines:

```
03D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
03E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
03F0: 00 03 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

For reference, `L 0300 031F` on `clear_6502.bin` prints 6502 instructions only, and it prints the same lines on either CPU:

(c) 2026 masanorisakai / MIT License

```
0300  A2 0F     LDX   #$0F
0302  A9 00     LDA   #$00
0304  9D D0 03  STA   $03D0,X
0307  CA        DEX
0308  10 FA     BPL   $0304
030A  8D F0 03  STA   $03F0
030D  C9 03     CMP   #$03
030F  F0 0B     BEQ   $031C
0311  EE F0 03  INC   $03F0
0314  D0 F7     BNE   $030D
0316  18        CLC
0317  69 01     ADC   #$01
0319  4C 0D 03  JMP   $030D
031C  8D F1 03  STA   $03F1
031F  60        RTS
```

6. The CPU switch on the panel

In the app, the **CPU** switch on the panel chooses between the 6502 and the 65C02. It works only while POWER is off, and the `--cpu` option chooses the CPU at start. To change the CPU, turn POWER off, move the switch, and turn POWER on again. Powering on clears RAM, so load the program after POWER is on: `--load` does it every time, or use **File** → **Load Program…**, or type the bytes.

7. Things to try

1. **Another limit.** On the 65C02 machine, change the `cmp` operand with `E 030E 01` and run `G 0300` again. The counter stops at `$0100`: the row printed by `D 03F0 03F1` starts with `00 01`.
2. **Push X.** Enter `E 0300 A2 07 DA A2 00 FA 8E F0 03 60`. The program is `ldx #$07`, `phx`, `ldx #$00`, `plx`, `stx $03F0`, `rts`. Run it with `G 0300` and look at `D 03F0 03F0`. On a 65C02 the first byte is `07`: `plx` brought the 7 back. On a 6502 it is `00`, because `phx` and `plx` do nothing there.
3. **Write your own.** Use `dec a` and `bra` to count down from 10 in A, and store the 0 at the end with `stz`.

8. Things to remember

- The 65C02 runs all 6502 programs, and adds instructions such as `stz`, `inc a`, `bra`, `phx` and `plx`.
- Tell `ca65` which CPU you mean with `.setcpu "6502"` or `.setcpu "65C02"`.
- On a 6502, the new opcodes are undefined. Glow65 runs them as NOPs, so a 65C02 program gives wrong results without any error.
- `L` shows `STZ`, `BRA` and the other new instructions by name only on a 65C02 machine. On a 6502 it prints them as `.BYTE`.
- The CPU can be changed only while POWER is off.

(c) 2026 masanorisakai / MIT License